GOOGLE | ALL SOURCES | TOPIC 5 OF 10 | CORE TOPIC

# Dynamic Programming

23 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

| 3 | 17 | 3 |
|---|---|---|
| EASY | MEDIUM | HARD |

**How to use**

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Dynamic Programming | 23 questions | tap a title to jump to it**


| | | | |
|---|---|---|---|
| **E** | 1. Maximum Profit | **Exponent** | **no date** |
| **E** | 2. Climbing Stairs | **Exponent** | **no date** |
| **E** | 3. Fibonacci Numbers | **Exponent** | **no date** |
| **M** | 4. Bounded Knapsack Variant | **1Point3Acres** | **2026-10-16** |
| **M** | 5. Longest Non-Decreasing Subarray with One Replacement | **1Point3Acres** | **2026-10-04** |
| **M** | 6. Grid Path Count with Diagonal Moves and Checkpoints | **1Point3Acres** | **2026-09-29** |
| **M** | 7. Minimum-Cost Tree Disconnection | **1Point3Acres** | **2026-09-09** |
| **M** | 8. Target Expressions from Ordered Digits | **1Point3Acres** | **2026-08-13** |
| **M** | 9. Count Word Decompositions into Chemical Element Symbols | **1Point3Acres** | **2026-07-14** |
| **M** | 10. Fence Painting Minimum Operations (Gemini Team) | **1Point3Acres** | **2026-04-27** |
| **M** | 11. Maximize sum without choosing adjacent elements | **PracHub** | **2026-01-06** |
| **M** | 12. Jump Game with Profit | **1Point3Acres** | **no date** |
| **M** | 13. Count Same-Color Squares in Matrix | **1Point3Acres** | **no date** |
| **M** | 14. Russian Doll Envelopes | **1Point3Acres** | **no date** |
| **M** | 15. Find the maximum subarray sum. | **Exponent** | **no date** |
| **M** | 16. Partition an array into two sub-arrays with equal sum. | **Exponent** | **no date** |
| **M** | 17. Given n houses in a line with money in each, find the maximum amount a robber can steal, without st... | **Exponent** | **no date** |
| **M** | 18. Find the longest palindromic subsequence using dynamic programming. | **Exponent** | **no date** |
| **M** | 19. Count the number of ways to split a string. | **Exponent** | **no date** |
| **M** | 20. Reverse Coin Change | **HackTheRounds** | **no date** |
| **H** | 21. Count Perfect Wake Numbers up to N | **1Point3Acres** | **2026-05-07** |
| **H** | 22. Tree Distance Sum (Re-root DP) | **1Point3Acres** | **2026-01-19** |
| **H** | 23. Determine winner in stack-merging game | **PracHub** | **2026-01-06** |

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

**EASY | Exponent | 1 of 23**

**★ TOP 10 EASY: rank 2 among Easy problems**

# ★ Maximum Profit

**ASKED (ANY CO.) ~4 MONTHS AGO | LAST 6 MONTHS**

**STUDY: TYPE** **ALL TIME 72** **30 DAYS 61** **PRIORITY 6/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** written and tested here for this exact question

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #121 Best Time to Buy and Sell Stock

**LeetCode match:** LC 121 Best Time to Buy and Sell Stock (likely) (inferred from short statement)

Last asked (any company): 4 months ago, about 2026-06 (Exponent shows only a rough age) | 4 interviews

**Original:** https://www.aced.io/questions/3630/maximum-profit

## QUESTION

You and your friend have decided to start an international trading business. You want to earn a profit through some ol' fashioned arbitrage: You'll buy widgets in one country and sell them in another at a higher price. Your friend has already found the current market prices for widgets in cities around the world, and you'd like to use this data to pick which cities to buy and sell widgets in. Write a function max_profit(prices) that efficiently finds the two cities that maximize your profit (i.e. largest difference in prices).
Input: A dictionary with cities as keys and prices as values
Output: An array containing the names of the cities (min, max) If the prices dictionary is empty, return None/null (depending on the programming language used). If there is 0 profit to be earned, return None/null as well. If you are using C++, return {"", ""} in such cases. Example Pythonprices = {'London': 72, 'New York': 70, 'Tokyo': 67, 'Miami': 62} max_profit(prices)

**=> ['Miami', 'London']Hint 1You can think of this as two sub-problems**

: Finding the lowest price, and finding the highest price.Hint 2While you could compare every pair of cities, it's more efficient to find the min and max values.Hint 3You can keep track of the lowest and highest values and update them as you iterate through the list.Challenge 1Does your solution work in O(N) time and O(1) space? If not, you can be more efficient!Challenge 2What happens if the price is the same in every city? Does your function still return

## APPROACH

Single pass tracking the minimum price seen so far and the best difference; adapt if the statement allows multiple trades. O(n).

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

The statement gives a dictionary city to price: ask whether the order of cities matters (it does not, since it is a dictionary), whether you may buy and sell in the same city, and what to return when fewer than two cities exist or all prices are equal.

**BRUTE FORCE TO SAY OUT LOUD**

Compare every ordered pair of cities. O(n^2).

**OPTIMAL IDEA**

Since there is no time order, the maximum profit is the largest price minus the smallest price, so find the city with the minimum price and the city with the maximum price in a single pass. Return [buy city, sell city]. O(n) time, O(1) space. If cities were a time series you would track the running minimum instead (LeetCode 121).

**DRY-RUN THESE CASES**

Two cities, ties for the minimum or maximum, all equal prices, a single city, negative prices.

**LIKELY FOLLOW-UPS**

Multiple transactions, a transaction fee, or the same problem on an ordered list.

**PITFALLS**

Using a time-order solution on an unordered dictionary. Returning the cities in the wrong order.

## HOW TO THINK ABOUT IT

Check whether order matters. A dictionary has none, so the profit is largest price minus smallest price, and the answer is just the two extremes. If this were a list of prices over time you would have to buy before you sell, which is a different one-pass problem with a running minimum.

**Basis:** A dictionary of city to price has no time order, so the best trade is the cheapest city to the most expensive one (compare LeetCode 121, where order matters). Tie rule assumed: smallest city name; with all prices equal, the first two names alphabetically.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(price, cities):
    best = (-INF, None, None)
    for buy in cities:
        for sell in cities:
            if buy == sell:
                continue
            profit = price[sell] - price[buy]
            best = max(best, (profit, buy, sell))
    return best
```

## BRUTE FORCE

| TIME | SPACE |
|---|---|
| O(n^2) | O(1) |

**WHY IT IS SLOW**

n^2 pairs. Since city order does not matter, the best pair is the cheapest city and the dearest city, found in one O(n) scan.

## OPTIMAL SOLUTION · One pass for the cheapest and the dearest city

| TIME | SPACE |
|---|---|
| O(n) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The prices are a time series: you must buy before you sell (LeetCode 121).

Keep the minimum price seen so far and the best difference: O(n), O(1). The dictionary version ignores order, so the two extremes are enough.

## FOLLOW-UP 2 Allow many trades (LeetCode 122) or a limit of k trades.

Many trades: add every positive day-to-day rise. At most k trades: DP over (trades used, holding or not), O(n * k).

## FOLLOW-UP 3 Transaction fee or cooldown.

A two-state DP (holding, not holding) per day: hold = max(hold, free - price), free = max(free, hold + price - fee).

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Maximum Profit
#
# You and your friend have decided to start an
# international trading business. You want to earn a
# profit through some ol' fashioned arbitrage: You'll buy
# widgets in one country and sell them in another at a
# higher price. Your friend has already found the current
# market prices for widgets in cities around the world,
# and you'd like to use this data to pick which cities to
# buy and sell widgets in. Write a function
# max_profit(prices) that efficiently finds the two cities
# that maximize your profit (i.e. largest difference in
# prices). Input: A dictionary with cities as keys and
# prices as values Output: An array containing the names
# of the cities (min, max) If the prices dictionary is
# empty, return None/null (depending on the programming
# language used). If there is 0 profit to be earned,
# return None/null as well. If you are using C++, return
# {"", ""} in such cases.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The prices are a time series: you must buy before you
# sell (LeetCode 121).
# 2. Allow many trades (LeetCode 122) or a limit of k
# trades.
# 3. Transaction fee or cooldown.
#
# INTERVIEW NOTES
# Check whether order matters. A dictionary has none, so
# the profit is largest price minus smallest price, and
# the answer is just the two extremes. If this were a list
# of prices over time you would have to buy before you
# sell, which is a different one-pass problem with a
# running minimum.
#
# BRUTE FORCE (say it first): Try every ordered pair of
# cities
# Compute price[sell] - price[buy] for every pair of
# different cities and keep the best. Correct, and
# quadratic. Time O(n^2). Space O(1).


import itertools
```

```
# BRUTE FORCE (say it first): Try every ordered pair of
# cities | time O(n^2)
def max_profit_brute(prices):
    best = None
    for buy, sell in itertools.permutations(
        sorted(prices), 2
    ):
        key = (-(prices[sell] - prices[buy]), buy, sell)
        if best is None or key < best:
            best = key
    return [best[1], best[2]]


# BASE QUESTION (optimal): One pass for the cheapest and
# the dearest city
# Idea: Find the city with the lowest price and the city
# with the highest price in one scan (ties by name). They
# are the best pair because profit is sell minus buy and
# the order of cities does not matter.
def max_profit_prices(prices):
    # a dictionary city -> price: buy at the cheapest city
    # and
    # sell at the dearest, order does not matter
    if len(prices) < 2:
        return 0
    values = list(prices.values())
    return max(values) - min(values)


# FOLLOW-UP 1: The prices are a time series: you must buy
# before you sell (LeetCode 121).
# Answer: Keep the minimum price seen so far and the best
# difference: O(n), O(1). The dictionary version ignores
# order, so the two extremes are enough.
def best_one_trade(series):
    # a time series: buy BEFORE you sell (LeetCode 121)
    low, best = float("inf"), 0
    for p in series:
        low = min(low, p)
        best = max(best, p - low)
    return best


# FOLLOW-UP 2: Allow many trades (LeetCode 122) or a limit
# of k trades.
```

```
# Answer: Many trades: add every positive day-to-day rise.
# At most k trades: DP over (trades used, holding or not),
```

```
# O(n * k).
def best_many_trades(series):
    # unlimited trades (LeetCode 122): take every rise
    pairs = zip(series, series[1:])
    return sum(max(0, b - a) for a, b in pairs)


# FOLLOW-UP 2: Allow many trades (LeetCode 122) or a limit
# of k trades.
# Answer: Many trades: add every positive day-to-day rise.
# At most k trades: DP over (trades used, holding or not),
# O(n * k).
def best_k_trades(series, k):
    # at most k trades: dp[t][holding]
    if not series or k == 0:
        return 0
    hold = [float("-inf")] * (k + 1)
    cash = [0] * (k + 1)
    for p in series:
        for t in range(1, k + 1):
            # buy with the cash of t - 1 finished trades,
            # or sell
            hold[t] = max(hold[t], cash[t - 1] - p)
            cash[t] = max(cash[t], hold[t] + p)
    return cash[k]


# FOLLOW-UP 3: Transaction fee or cooldown.
# Answer: A two-state DP (holding, not holding) per day:
# hold = max(hold, free - price), free = max(free, hold +
# price - fee).
def best_with_fee(series, fee):
    # a fee per trade: two states each day
    hold, cash = float("-inf"), 0
    for p in series:
        hold = max(hold, cash - p)
        cash = max(cash, hold + p - fee)
    return cash


if __name__ == "__main__":
    cities = {"a": 10, "b": 4, "c": 9}
    assert max_profit_prices(cities) == 6
```

```
    assert max_profit_prices({"a": 3}) == 0
    assert best_one_trade([7, 1, 5, 3, 6, 4]) == 5
    assert best_one_trade([7, 6, 4, 3, 1]) == 0
    assert best_many_trades([7, 1, 5, 3, 6, 4]) == 7
    assert best_k_trades([3, 2, 6, 5, 0, 3], 2) == 7
    assert best_k_trades([1, 2, 3], 1) == 2
    assert best_k_trades([], 3) == 0
    assert best_with_fee([1, 3, 2, 8, 4, 9], 2) == 8
```

**OFFICIAL PAGE TEXT**

You and your friend have decided to start an international trading business. You want to earn a profit through some ol' fashioned arbitrage: You'll buy widgets in one country and sell them in another at a higher price. Your friend has already found the current market prices for widgets in cities around the world, and you'd like to use this data to pick which cities to buy and sell widgets in. Write a function max_profit(prices) that efficiently finds the two cities that maximize your profit (i.e. largest difference in prices).
Input: A dictionary with cities as keys and prices as values
Output: An array containing the names of the cities (min, max) If the prices dictionary is empty, return None/null (depending on the programming language used). If there is 0 profit to be earned, return None/null as well. If you are using C++, return {"", ""} in such cases. Example

```
Pythonprices = {'London': 72, 'New York': 70, 'Tokyo':
 67, 'Miami': 62}

max_profit(prices) # => ['Miami', 'London']
```

Hint 1You can think of this as two sub-problems: Finding the lowest price, and finding the highest price.Hint 2While you could compare every pair of cities, it's more efficient to find the min and max values.Hint 3You can keep track of the lowest and highest values and update them as you iterate through the list.Challenge 1Does your solution work in O(N) time and O(1) space? If not, you can be more efficient!Challenge 2What happens if the price is the same in every city? Does your function still return different cities?For our solution, we'll simply iterate through the prices list and simultaneously keep track of the minimum and maximum values we've seen so far. This only requires one iteration, so it's more efficient than the 'brute force' approach of calculating the profit for each pair of cities, which would require O(n^2) time. A more concise—but slightly less efficient solution—would be to sort the cities according to their prices and then return the first and last elements. In some languages like Python, you can also use the built-in min and max functions to compare dictionary values:

```
Pythondef max_profit(prices):

  # Check if the dictionary is empty
  if not prices:
    return None

  min_city = min(prices, key=prices.get)
  max_city = max(prices, key=prices.get)

  # Check if no profits earned
  if min_city == max_city:
    return None

  return [min_city, max_city]
```

**OFFICIAL PAGE TEXT**

Time Complexity: Iterating over prices.items() takes O(n) time, where n is the number of cities in the prices dictionary. Within the loop, there are constant-time operations, so the overall time complexity is O(n). Space Complexity: The space complexity is also O(1) because the size of the auxiliary array returned ([min_city, max_city]) is constant and independent of the input size.No answers contributed by the community yet.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

**EASY | IN APP LC 70 | Exponent | 2 of 23**

**★ TOP 10 EASY: rank 6 among Easy problems**

# ★ Climbing Stairs

**ASKED (ANY CO.) ~4 MONTHS AGO | LAST 6 MONTHS**

**STUDY: TYPE** **ALL TIME 62.9** **30 DAYS 51.4** **PRIORITY 8/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** your app's interview approach (LeetCode #70)

**LeetCode match:** LC 70 Climbing Stairs

Last asked (any company): 4 months ago, about 2026-06 (Exponent shows only a rough age) | +38 interviews

**Original:** https://www.aced.io/questions/3255/climbing-stairs

## QUESTION

Given an integer n, representing the number of steps in a staircase, write a function climbStairs that returns the number of distinct ways to climb to the top of the staircase. Each time you can either climb 1 step or 2 steps. You may assume that n is a non-negative integer. Examples Input: n = 1 Output: 1 Explanation: There is only 1 way to climb a staircase with 1 step.
Input: n = 2 Output: 2 Explanation: There are 2 ways to climb a staircase with 2 steps:

1. Climb 1 step + 1 step 2. Climb 2 steps directly Input: n = 3 Output: 3 Explanation: There are 3 ways to climb a staircase with 3 steps:
1. Climb 1 step + 1 step + 1 step 2. Climb 1 step + 2 steps 3. Climb 2 steps + 1 step Input: n = 4 Output: 5 Explanation: There are 5 ways to climb a staircase with 4 steps:
1. Climb 1 step + 1 step + 1 step + 1 step 2. Climb 1 step + 1 step + 2 steps 3. Climb 1 step + 2 steps + 1 step 4. Climb 2 steps + 1 step + 1 step 5. Climb 2 steps + 2 stepsChallenge 1Can you come up with a solution that has a O(n) time complexity?Challenge 2Can you come up with a solution that has a O(1) space complexity?Solution 1: Dynamic Programming Approach In this solution, we use dynamic programming to compute the number of distinct ways to climb n stairs. The idea is to build up the solution incrementally by solving subproblems. We initialize a list ways where ways[i] represents the number of ways to reach the ith step. We start by setting the base cases: ways[0] = 1 (1 way to stay at the ground) ways[1] =

## APPROACH

dp[i] = dp[i-1] + dp[i-2] with two variables. O(n), O(1).

## HOW TO THINK ABOUT IT

The last move was either a 1-step from n - 1 or a 2-step from n - 2, so ways(n) = ways(n - 1) + ways(n - 2): the Fibonacci recurrence. Only the previous two values matter, so two variables replace the whole table.

**Basis:** LeetCode 70, Climbing Stairs: ways to climb n steps taking 1 or 2 at a time (a Fibonacci number).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def ways(n):
    if n <= 1:
        return 1
    return ways(n - 1) + ways(n - 2)
```

| TIME | SPACE |
|---|---|
| O(2^n) | O(n) |

**WHY IT IS SLOW**

The same values are computed over and over: O(2^n). Keeping the last two values in a loop is O(n) time and O(1) space (and a matrix power gives O(log n)).

## OPTIMAL SOLUTION · Two running values

| TIME | SPACE |
|---|---|
| O(n) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 n is huge (10^18).

Use matrix exponentiation of [[1, 1], [1, 0]] (or fast doubling) for O(log n) multiplications; take the result modulo a prime if asked.

## FOLLOW-UP 2 You can take 1 to k steps (or a given set of step sizes).

ways[i] = sum of ways[i - s] over allowed step sizes s; for 1..k keep a sliding-window sum for O(n).

## FOLLOW-UP 3 Some steps are broken (cannot be stood on).

Set ways[i] = 0 for broken steps while filling the table; everything else is unchanged.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Climbing Stairs
#
# Given an integer n, representing the number of steps in
# a staircase, write a function climbStairs that returns
# the number of distinct ways to climb to the top of the
# staircase. Each time you can either climb 1 step or 2
# steps. You may assume that n is a non-negative integer.
# Examples Input: n = 1 Output: 1 Explanation: There is
# only 1 way to climb a staircase with 1 step. Input: n =
# 2 Output: 2 Explanation: There are 2 ways to climb a
# staircase with 2 steps: 1. Climb 1 step + 1 step 2.
# Climb 2 steps directly Input: n = 3 Output: 3
# Explanation: There are 3 ways to climb a staircase with
# 3 steps: 1. Climb 1 step + 1 step + 1 step 2. Climb 1
# step + 2 steps 3. Climb 2 steps + 1 step Input: n = 4
# Output: 5 Explanation: There are 5 ways to climb a
# staircase with 4 steps: 1. Climb 1 step + 1 step + 1
# step + 1 step 2. Climb 1 step + 1 step + 2 steps 3.
# Climb 1 step + 2 steps + 1 step 4.
#
# FOLLOW-UPS (answered below, in this order):
# 1. n is huge (10^18).
# 2. You can take 1 to k steps (or a given set of step
# sizes).
# 3. Some steps are broken (cannot be stood on).
#
# INTERVIEW NOTES
# The last move was either a 1-step from n - 1 or a 2-step
# from n - 2, so ways(n) = ways(n - 1) + ways(n - 2): the
# Fibonacci recurrence. Only the previous two values
# matter, so two variables replace the whole table.
#
# BRUTE FORCE (say it first): Plain recursion
# ways(n) = ways(n - 1) + ways(n - 2) computed
# recursively. Correct and the oracle; the same values are
# recomputed exponentially often. Time O(2^n). Space O(n).


# BRUTE FORCE (say it first): Plain recursion | time
# O(2^n)
def climb_recursive(n):
    return (
        1
        if n <= 1
        else climb_recursive(n - 1)
        + climb_recursive(n - 2)
    )


# HELPERS for follow-up 1
def mat_mult(x, y, mod):
    out = [[0, 0], [0, 0]]
    for i in range(2):
        for j in range(2):
            cells = (x[i][k] * y[k][j] for k in range(2))
            total = sum(cells)
            out[i][j] = total % mod
    return out


# BASE QUESTION (optimal): Two running values
# Idea: Keep ways for the previous two steps and advance:
# (a, b) = (b, a + b). Start with ways(0) = ways(1) = 1.
# No table and no recursion.
def climb(n):
    # ways(n) = ways(n - 1) + ways(n - 2): O(n), O(1)
    # space
    a, b = 1, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b if n >= 1 else 1


# FOLLOW-UP 1: n is huge (10^18).
# Answer: Use matrix exponentiation of [[1, 1], [1, 0]]
# (or fast doubling) for O(log n) multiplications; take
# the result modulo a prime if asked.
def climb_huge(n, mod=10 ** 9 + 7):
    # n up to 10^18: matrix power of [[1, 1], [1, 0]]
    result, base = [[1, 0], [0, 1]], [[1, 1], [1, 0]]
    e = n
    while e:
        if e & 1:
            result = mat_mult(result, base, mod)
        base = mat_mult(base, base, mod)
        e >>= 1
    # result = [[F(n+1), F(n)], [F(n), F(n-1)]]
    return result[0][0] % mod


# FOLLOW-UP 2: You can take 1 to k steps (or a given set
# of step sizes).
# Answer: ways[i] = sum of ways[i - s] over allowed step
# sizes s; for 1..k keep a sliding-window sum for O(n).
def climb_steps(n, steps):
    # any allowed step sizes: sum the ways of the earlier
    # stairs
    ways = [1] + [0] * n
    for i in range(1, n + 1):
        ways[i] = sum(ways[i - s] for s in steps
                      if i >= s)
    return ways[n]


# FOLLOW-UP 2: You can take 1 to k steps (or a given set
# of step sizes).
# Answer: ways[i] = sum of ways[i - s] over allowed step
# sizes s; for 1..k keep a sliding-window sum for O(n).
def climb_up_to_k(n, k):
    # steps 1..k in O(n): keep a sliding-window sum
    ways, window = [1] + [0] * n, 1
    for i in range(1, n + 1):
        ways[i] = window
        window += ways[i]
        if i - k >= 0:
            window -= ways[i - k]
    return ways[n]


# FOLLOW-UP 3: Some steps are broken (cannot be stood on).
# Answer: Set ways[i] = 0 for broken steps while filling
# the table; everything else is unchanged.
def climb_broken(n, broken):
    # broken steps cannot be stood on: zero ways there
    bad = set(broken)
    ways = [1] + [0] * n
    for i in range(1, n + 1):
        if i not in bad:
            two_back = ways[i - 2] if i > 1 else 0
            ways[i] = ways[i - 1] + two_back
    return ways[n]


if __name__ == "__main__":
    got = [climb(n) for n in range(1, 6)]
    assert got == [1, 2, 3, 5, 8]
    assert climb(0) == 1
    assert climb_huge(5) == 8 and climb_huge(1) == 1
    assert 0 <= climb_huge(10 ** 18) < 10 ** 9 + 7
    assert climb_huge(50) == climb(50) % (10 ** 9 + 7)
    assert climb_steps(5, (1, 2)) == 8
    assert climb_steps(4, (1, 3)) == 3
    assert climb_up_to_k(5, 2) == 8
    assert climb_up_to_k(4, 3) == 7
    assert climb_broken(5, [3]) == 2
```

```
assert climb_broken(3, [1, 2]) == 0
```

**OFFICIAL PAGE TEXT**

Given an integer n, representing the number of steps in a staircase, write a function climbStairs that returns the number of distinct ways to climb to the top of the staircase. Each time you can either climb 1 step or 2 steps. You may assume that n is a non-negative integer. Examples

```
Input: n = 1
Output: 1
Explanation: There is only 1 way to climb a staircase
with 1 step.

Input: n = 2
Output: 2
Explanation: There are 2 ways to climb a staircase wit
h 2 steps:
1. Climb 1 step + 1 step
2. Climb 2 steps directly

Input: n = 3
Output: 3
Explanation: There are 3 ways to climb a staircase wit
h 3 steps:
1. Climb 1 step + 1 step + 1 step
2. Climb 1 step + 2 steps
3. Climb 2 steps + 1 step

Input: n = 4
Output: 5
Explanation: There are 5 ways to climb a staircase wit
h 4 steps:
1. Climb 1 step + 1 step + 1 step + 1 step
2. Climb 1 step + 1 step + 2 steps
3. Climb 1 step + 2 steps + 1 step
4. Climb 2 steps + 1 step + 1 step
5. Climb 2 steps + 2 steps
```

**OFFICIAL PAGE TEXT**

Challenge 1Can you come up with a solution that has a O(n) time complexity?Challenge 2Can you come up with a solution that has a O(1) space complexity?Solution 1: Dynamic Programming Approach In this solution, we use dynamic programming to compute the number of distinct ways to climb n stairs. The idea is to build up the solution incrementally by solving subproblems. We initialize a list ways where ways[i] represents the number of ways to reach the ith step. We start by setting the base cases: ways[0] = 1 (1 way to stay at the ground) ways[1] = 1 (1 way to reach the first step, which is a single 1-step) For each subsequent step i, the number of ways to reach that step is the sum of the ways to reach the previous step (ways[i - 1]) and the step before that (ways[i - 2]). This is because you can get to step i either from step i - 1 with a 1-step or from step i - 2 with a 2-step. Time Complexity: The time complexity of this solution is O(n), where n is the number of steps. This is because we iterate through the steps from 2 to n, updating the number of ways for each step. Space Complexity: The space complexity is O(n) due to the additional array ways used to store the number of ways for each step. If space optimization is needed, this can be reduced to O(1) by keeping only the last two computed values. Solution 2: Combinatorial Approach Our solution leverages combinatorial mathematics to calculate

the number of distinct ways to climb n stairs. The approach is based on counting the number of valid sequences of steps (1-step and 2-steps) that sum up to n. We use the binomial coefficient to count the number of ways to arrange a certain number of 1-steps and 2-steps. For each possible number of 2-steps (k), we compute the number of ways to choose k positions from n - k steps (the remaining steps will be 1-steps). This is done using the formula for combinations: C(n,k)C(n, k)C(n,k), where C(n,k)=n!k!*(n-k)!C(n, k) = \frac{n!}{k! \cdot (n-k)!}C(n,k)=k!*(n-k)!n!. The total number of ways is the sum of these combinations for all valid values of k. Time Complexity: The time complexity of this solution is O(n), where n is the number of steps. This is because we compute the combination for each k from 0 to n // 2, and each combination computation is efficient. Space Complexity: The space complexity is O(1) as we only use a few additional variables for storing intermediate results, regardless of the input size.No answers contributed by the community yet.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

**EASY | IN APP LC 509 | Exponent | 3 of 23**

# Fibonacci Numbers

**ASKED (ANY CO.) ~9 MONTHS AGO | OLDER**

**STUDY: TYPE** **ALL TIME 46.3** **30 DAYS 0** **PRIORITY 76/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** your app's interview approach (LeetCode #509)

**LeetCode match:** LC 509 Fibonacci Number

Last asked (any company): 9 months ago, about 2026-01 (Exponent shows only a rough age) | +26 interviews

**Original:** https://www.aced.io/questions/3631/fibonacci

## QUESTION

Write a function fib(n) that returns the nth Fibonacci number. The Fibonacci sequence is defined as F(n) = F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which produces the following pattern: 1, 1, 2, 3, 5, 8, 13 ... Examples Pythonfib(0)

**=> 1 fib(1)**

**=> 1 fib(2)**

**=> 2 fib(3)**

**=> 3 ... fib(10)**

**=> 89 fib(20)**

**=> 10946Hint 1The Fibonacci sequence**

is defined recursively since it refers to itself.Hint 2We can start with a recursive implementation and optimize it.Hint 3Each call to F(n) has two subproblems: F(n-1) and F(n-2). Can we memoize these calculations?Challenge 1Does your solution work in O(n) time?This question is a classic example of how dynamic programming, or memoization, can help us solve problems faster by reducing repetitive computations. In this first version, we recursively call the fib function twice, and then we call it twice more in every subsequent recursive call, so the total number of operations expands to O(2^n)! To improve the runtime of our program, let's save our prior calculations in a dictionary called memo and pass it into our function. Now, if we've already found fib(n) before, our function will use this stored value instead of recursing again. Time Complexity: The time complexity of the memoized version is O(n). This is because each Fibonacci number from 0 to n is computed only once and then stored in the memo dictionary. The recursive calls will now simply look up the value in the dictionary if it has already been computed, reducing redundant

## APPROACH

Iterative two-variable loop O(n), O(1) space; mention memoization and matrix exponentiation / fast doubling for O(log n).

## HOW TO THINK ABOUT IT

Each term needs only the previous two, so a loop with two variables is O(n) time and O(1) space; the naive recursion does exponential repeated work. Since the recurrence is linear it can also be turned into a matrix power, or into the fast doubling identities, for O(log n).

**Basis:** LeetCode 509, Fibonacci Number (F(0) = 0, F(1) = 1).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def fib(n):
    if n < 2:
        return 1  # fib(0) = fib(1) = 1
    return fib(n - 1) + fib(n - 2)
```

| TIME | SPACE |
|---|---|
| O(2^n) | O(n) |

**WHY IT IS SLOW**

The call tree has about 1.6^n nodes because the same values are recomputed. A loop with two variables is O(n) time and O(1) space; fast doubling is O(log n).

## OPTIMAL SOLUTION · Iteration with two variables

| TIME | SPACE |
|---|---|
| O(n) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 n is up to 10^18 and the answer is needed modulo a prime.

Fast doubling (or matrix power) with every multiplication reduced modulo the prime: O(log n) steps with small numbers.

## FOLLOW-UP 2 Why is the recursive version slow, and how do you fix it without changing the structure?

It recomputes the same values exponentially often (the call tree has about phi^n nodes). Memoisation (lru_cache) makes each value computed once, so O(n) time with O(n) memory; bottom-up with two variables drops the memory to O(1).

**FOLLOW-UP 3 Closed form (Binet's formula).**

F(n) = round(phi^n / sqrt(5)) is exact only while floating point has enough precision (up to about n = 70); beyond that it gives wrong digits, so use the integer methods.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Fibonacci Numbers
#
# Write a function fib(n) that returns the nth Fibonacci
# number. The Fibonacci sequence is defined as F(n) =
# F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which
# produces the following pattern: 1, 1, 2, 3, 5, 8, 13 ...
# Examples Pythonfib(0) # => 1 fib(1) # => 1 fib(2) # => 2
# fib(3) # => 3 ... fib(10) # => 89 fib(20) # => 10946Hint
# 1The Fibonacci sequence is defined recursively since it
# refers to itself.Hint 2We can start with a recursive
# implementation and optimize it.Hint 3Each call to F(n)
# has two subproblems: F(n-1) and F(n-2). Can we memoize
# these calculations?Challenge 1Does your solution work in
# O(n) time?This question is a classic example of how
# dynamic programming, or memoization, can help us solve
# problems faster by reducing repetitive computations.
#
# FOLLOW-UPS (answered below, in this order):
# 1. n is up to 10^18 and the answer is needed modulo a
# prime.
# 2. Why is the recursive version slow, and how do you fix
# it without changing the structure?
# 3. Closed form (Binet's formula).
#
# INTERVIEW NOTES
# Each term needs only the previous two, so a loop with
# two variables is O(n) time and O(1) space; the naive
# recursion does exponential repeated work. Since the
# recurrence is linear it can also be turned into a matrix
# power, or into the fast doubling identities, for O(log
# n).
#
# BRUTE FORCE (say it first): Plain recursion
# fib(n) = fib(n - 1) + fib(n - 2) with the two base
# cases. Correct and the oracle; the same values are
# recomputed exponentially often. Time O(2^n). Space O(n).


from functools import lru_cache
from math import sqrt


# THE DEFINITION (slow, say it first)
def fib_slow(n):
    # the definition: correct but about 1.6 ** n calls
    if n < 2:
        return 1
    return fib_slow(n - 1) + fib_slow(n - 2)


# HELPERS for follow-up 1
# this problem starts at fib(0) = 1, fib(1) = 1
MOD = 10 ** 9 + 7


def pair(n, mod):
    # follow-up: fast doubling. Returns (F(n), F(n + 1))
    # of
    # the standard sequence 0, 1, 1, 2 using
    #   F(2k)     = F(k) * (2 * F(k + 1) - F(k))
    #   F(2k + 1) = F(k) ** 2 + F(k + 1) ** 2
    if n == 0:
        return 0, 1
    a, b = pair(n // 2, mod)
    c = a * (2 * b - a) % mod
    d = (a * a + b * b) % mod
    return (d, c + d) if n % 2 else (c, d)


# BASE QUESTION (optimal): Iteration with two variables
# Idea: Keep the previous two values and advance n times.
# Memoising the recursion gives the same time with O(n)
# memory; the loop needs no memory and no recursion.
def fib(n):
    # bottom-up with two variables: O(n) time, O(1) memory
    a, b = 1, 1
    for _ in range(n):
        a, b = b, a + b
    return a


# FOLLOW-UP 1: n is up to 10^18 and the answer is needed
# modulo a prime.
# Answer: Fast doubling (or matrix power) with every
# multiplication reduced modulo the prime: O(log n) steps
# with small numbers.
def fib_mod(n, mod=MOD):
    # fib(n) here is the standard F(n + 1): O(log n) steps
    # with small numbers, so n = 10 ** 18 is easy
    return pair(n + 1, mod)[0] % mod


# FOLLOW-UP 2: Why is the recursive version slow, and how
# do you fix it without changing the structure?
# Answer: It recomputes the same values exponentially
# often (the call tree has about phi^n nodes). Memoisation
# (lru_cache) makes each value computed once, so O(n) time
# with O(n) memory; bottom-up with two variables drops the
# memory to O(1).
@lru_cache(maxsize=None)
def fib_memo(n):
    # follow-up: the same structure with a cache, so every
    # value is computed once: O(n) time and memory
    if n < 2:
        return 1
    return fib_memo(n - 1) + fib_memo(n - 2)


# FOLLOW-UP 3: Closed form (Binet's formula).
# Answer: F(n) = round(phi^n / sqrt(5)) is exact only
# while floating point has enough precision (up to about n
# = 70); beyond that it gives wrong digits, so use the
# integer methods.
def fib_binet(n):
    # follow-up: closed form. Floats run out of precision.
    phi = (1 + sqrt(5)) / 2
    return round(phi ** (n + 1) / sqrt(5))


if __name__ == "__main__":
    first = [fib(n) for n in range(7)]
    assert first == [1, 1, 2, 3, 5, 8, 13]
    assert fib(10) == 89 and fib(20) == 10946
    assert fib_slow(10) == 89
    assert fib_memo(20) == 10946
    assert fib(0) == 1 and fib(1) == 1
    for n in range(0, 200):
        assert fib_mod(n, MOD) == fib(n) % MOD
        big = 10 ** 18 + 9
```

```
    assert fib_mod(n, big) == fib(n) % big
# a huge n finishes at once
assert 0 <= fib_mod(10 ** 18) < MOD
# Binet is exact for a while, then floating point
# fails
assert all(fib_binet(n) == fib(n) for n in range(60))
```

```
late = range(60, 200)
bad = [n for n in late if fib_binet(n) != fib(n)]
assert bad and bad[0] < 100
```

**OFFICIAL PAGE TEXT**

Write a function fib(n) that returns the nth Fibonacci number. The Fibonacci sequence is defined as F(n) = F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which produces the following pattern:

```
1, 1, 2, 3, 5, 8, 13 ...
```

Examples

```
Pythonfib(0)  # => 1
fib(1)  # => 1
fib(2)  # => 2
fib(3)  # => 3
...
fib(10) # => 89
fib(20) # => 10946
```

Hint 1The Fibonacci sequence is defined recursively since it refers to itself.Hint 2We can start with a recursive implementation and optimize it.Hint 3Each call to F(n) has two subproblems: F(n-1) and F(n-2). Can we memoize these calculations?Challenge 1Does your solution work in O(n) time?This question is a classic example of how dynamic programming, or memoization, can help us solve problems faster by reducing repetitive computations. In this first version, we recursively call the fib function twice, and then we call it twice more in every subsequent recursive call, so the total number of operations expands to O(2^n)! To improve the runtime of our program, let's save our prior calculations in a dictionary called memo and pass it into our function. Now, if we've already found fib(n) before, our function will use this stored value instead of recursing again. Time Complexity: The time complexity of the memoized version is O(n). This is because each Fibonacci number from 0 to n is computed only once and then stored in the memo dictionary. The recursive calls will now simply look up the value in the dictionary if it has already been computed, reducing redundant calculations. Space Complexity: The memo dictionary stores each Fibonacci number from 0 to n, which requires O(n) space.No answers contributed by the community yet.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

Site tags: greedy, dp, quickselect, sorting, medium

**MEDIUM | 1Point3Acres | tech-screen | 4 of 23**

# Bounded Knapsack Variant

**ASKED 2026-10-16 | 4 DAYS AGO | LAST 30 DAYS**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 180/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** written and tested here for this exact question

**LeetCode match:** LC 1710 Maximum Units on a Truck

Frequency: Single report | Last asked: 2026-10-16 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/unit-weight-bounded-knapsack

## QUESTION

Each item type has a value and an available count; every item weighs 1. Given capacity W, maximize total value. DP or DFS + memo works at O(n·W), greedy by value after sorting is O(n log n), and the best answer uses quickselect for O(n).

**Input:** for each item type, its value and the number of copies available. Every item has weight 1.
**Input:** a maximum total weight W.
**Output:** the maximum total value achievable without exceeding W.

## APPROACH

Write the sort-based greedy and argue its optimality in two sentences (unit weights mean every slot is interchangeable). Implement quickselect that partitions item types by value and tracks the total count on each side to locate where W runs out. Compare the O(n × W), O(n log n), and O(n) versions out loud with their complexities.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask whether every item really weighs 1, whether values can be zero or negative (assume non-negative), whether W can exceed the total number of items, and how large the counts and W can get.

**BRUTE FORCE TO SAY OUT LOUD**

A knapsack table over the capacity, one pass per copy. O(total copies * W), correct for any weights.

**OPTIMAL IDEA**

## INTERVIEW-DAY PLAN

With unit weights every slot is interchangeable, so sort the item types by value, highest first, and take min(copies, slots left) of each until the bag is full. O(n log n). To reach O(n), run quickselect on the value: split the types into higher, equal and lower, recurse into the side that contains the W-th slot, and keep a running total of what was taken.

**DRY-RUN THESE CASES**

W = 0, W larger than all copies together, one type, equal values, a type with a huge count, all values equal.

**LIKELY FOLLOW-UPS**

Different weights (bounded knapsack with binary splitting), a huge W (greedy still works), or negative values (never take them).

**PITFALLS**

Reaching for DP because the word knapsack appears. Forgetting that a type may only be partly taken. Letting the remaining capacity go negative.

## HOW TO THINK ABOUT IT

The word knapsack points at DP, but the statement says every item weighs 1. With equal weights every slot in the bag is interchangeable, so an item that is worth more is never worse than one that is worth less: swapping a cheaper item for a dearer one always helps. Take item types by value, highest first, until W slots are used. You do not even need the full sort: quickselect finds where the W-th slot falls in O(n).

**Basis:** LeetCode 1710, Maximum Units on a Truck: every item weighs 1, so the knapsack collapses to 'take the most valuable items first'.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def knapsack(items, W):
    best = [0] * (W + 1)
    for value, count in items:
        for _ in range(count):
            for w in range(W, 0, -1):
                take = best[w - 1] + value
                best[w] = max(best[w], take)
    return best[W]
```

| TIME | SPACE |
|---|---|
| O(total copies * W) | O(W) |

**WHY IT IS SLOW**

Every copy is its own pass over the table, O(total copies * W). With unit weights the most valuable items are always the right ones, so sorting by value and taking greedily is O(n log n).

## OPTIMAL SOLUTION · Sort by value and take greedily

| TIME | SPACE |
|---|---|
| O(n log n) | O(1) extra |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Why is greedy correct here but not for a general knapsack?

With unit weights each chosen item uses exactly one slot, so the only thing that matters is value per slot, which is just the value. If an optimal bag held a cheaper item while a dearer one was available, swapping them keeps the weight and raises the value, so the optimal bag can always be rearranged into the greedy one. With different weights a cheap item can leave room for several others, so value per weight and the leftover space interact and greedy fails.

## FOLLOW-UP 2 What if the weights are not all 1?

Then it is a real bounded knapsack and greedy no longer works; use DP over the capacity. Splitting each count into 1, 2, 4, ... copies keeps the number of 0/1 items at about log(count) per type, so the cost is O(n * log(max count) * W) instead of O(total copies * W).

## FOLLOW-UP 3 W is up to 10^9. Which version still works?

Only the greedy ones: sorting is O(n log n) and quickselect O(n), and neither depends on W, while any table over the capacity needs O(W) memory. The loop simply stops when the remaining capacity reaches zero.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Bounded Knapsack Variant
#
# Each item type has a value and an available count; every
# item weighs 1. Given capacity W, maximize total value.
# DP or DFS + memo works at O(n·W), greedy by value after
# sorting is O(n log n), and the best answer uses
# quickselect for O(n). Input: for each item type, its
# value and the number of copies available. Every item has
# weight 1. Input: a maximum total weight W. Output: the
# maximum total value achievable without exceeding W.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is greedy correct here but not for a general
# knapsack?
# 2. What if the weights are not all 1?
# 3. W is up to 10^9. Which version still works?
#
# INTERVIEW NOTES
# The word knapsack points at DP, but the statement says
# every item weighs 1. With equal weights every slot in
# the bag is interchangeable, so an item that is worth
# more is never worse than one that is worth less:
# swapping a cheaper item for a dearer one always helps.
# Take item types by value, highest first, until W slots
# are used. You do not even need the full sort:
# quickselect finds where the W-th slot falls in O(n).
#
# BRUTE FORCE (say it first): DP over the capacity
# Treat every copy as its own 0/1 item and run the classic
# 1D knapsack table. It is correct for any weights, so it
# is the oracle, but it is far too slow when the counts or
# W are large. Time O(total copies * W). Space O(W).


import random


# BRUTE FORCE (say it first, it also checks the answers)
def knapsack_dp(items, W):
    # the oracle: every copy is its own 0/1 item
    best = [0] * (W + 1)
    for value, count in items:
        for _ in range(count):
            for w in range(W, 0, -1):
                take = best[w - 1] + value
                best[w] = max(best[w], take)
    return best[W]


# BASE QUESTION (optimal): Sort by value and take greedily
# Idea: Sort the item types by value, highest first. For
# each type take min(copies, slots left) and add value
# times that. Stop when the bag is full.
def max_value_sorted(items, W):
    """Unit weights: every slot is worth the same, so the
    best value always goes in first."""
    total = 0
    for value, count in sorted(items, reverse=True):
        # take as many copies as still fit in the bag
        take = min(count, W)
        total += value * take
        W -= take
        if W == 0:
            break
    return total


# SAME answer in O(n): quickselect
def max_value_select(items, W):
    """Same answer in O(n) expected: quickselect finds
    where the W-th slot falls without a full sort."""
    total, pool = 0, list(items)
    while pool and W > 0:
        pivot = random.choice(pool)[0]
        high = [it for it in pool if it[0] > pivot]
        mid = [it for it in pool if it[0] == pivot]
        low = [it for it in pool if it[0] < pivot]
        high_count = sum(c for _, c in high)
        if high_count >= W:
            # the bag fills up inside the dearer types
            pool = high
            continue
        # every dearer copy fits, then fill from the pivot
        total += sum(v * c for v, c in high)
        W -= high_count
        take = min(W, sum(c for _, c in mid))
        total += pivot * take
        W -= take
        pool = low
    return total
```

```
# FOLLOW-UP 2: What if the weights are not all 1?
# Answer: Then it is a real bounded knapsack and greedy no
# longer works; use DP over the capacity. Splitting each
# count into 1, 2, 4, ...
def knapsack_bounded(items, W):
    """Weights differ: greedy fails, so use DP. Splitting
    each count into 1, 2, 4, ... copies keeps it small."""
    best = [0] * (W + 1)
    for value, weight, count in items:
        chunk = 1
        while count > 0:
            take = min(chunk, count)
            count -= take
            cost, gain = take * weight, take * value
            for w in range(W, cost - 1, -1):
                best[w] = max(best[w],
                              best[w - cost] + gain)
            chunk *= 2
    return best[W]


if __name__ == "__main__":
    demo = [(3, 2), (5, 1), (4, 4)]
    assert max_value_sorted(demo, 4) == 17
    assert max_value_select(demo, 4) == 17
    assert max_value_sorted([], 5) == 0
    assert max_value_sorted([(7, 3)], 0) == 0
    assert max_value_sorted([(7, 3)], 10) == 21
    mixed = [(10, 5, 1), (6, 3, 2)]
    assert knapsack_bounded(mixed, 6) == 12
    # greedy is only valid when every weight is 1
    unit = [(3, 1, 2), (5, 1, 1), (4, 1, 4)]
    assert knapsack_bounded(unit, 4) == 17
    for _ in range(300):
        n = random.randint(0, 6)
        rnd = random.randint
        items = [(rnd(1, 9), rnd(1, 4)) for _ in range(n)]
        W = random.randint(0, 12)
        want = knapsack_dp(items, W)
        assert max_value_sorted(items, W) == want
        assert max_value_select(items, W) == want
```

## NOTES FROM CANDIDATES

Pitfalls: Recognizing "knapsack" and reaching for DP is the trap; with unit weights the greedy argument is simpler and faster.

## PREPARATION

Write the sort-based greedy and argue its optimality in two sentences (unit weights mean every slot is interchangeable). Implement quickselect that partitions item types by value and tracks the total count on each side to locate where W runs out. Compare the O(n × W), O(n log n), and O(n) versions out loud with their complexities.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

Site tags: array, dp, two-pointer, medium

**MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 5 of 23**

# Longest Non-Decreasing Subarray with One Replacement

**ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS**

**STUDY: READ** **NO LEETCODE FREQUENCY** **PRIORITY 209/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-04 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/longest-non-decreasing-subarray-one-replacement

## QUESTION

Find the maximum length of a contiguous non-decreasing subarray if at most one element may be changed to any value. Some rounds start from the plain longest-increasing-subarray question and add the one-replacement follow-up; those versions ask for an increasing run, so clarify strict vs non-strict.

**Input:** an integer array nums.
You may change at most one element to any value.
Return the maximum length of a contiguous subarray that is non-decreasing (nums[i] <= nums[i + 1] for every adjacent pair inside it) after the change.

**Variant:** other rounds ask for a strictly increasing run and open with the no-replacement version (longest increasing subarray) before adding the one-replacement follow-up — clarify strict vs non-strict before coding.

Expect to discuss and compare the complexity of several approaches; a brute-force first pass is usually followed by a request to optimize.

**EXAMPLES**

```
nums = [1, 0, 3, 4, 5, 2, 3]
Output: 5
Change nums[1] from 0 to 1 -> [1, 1, 3, 4, 5, 2, 3]
[1, 1, 3, 4, 5] is a non-decreasing subarray of
    length 5.
```

## APPROACH

Write the plain longest-run scan, then extend it with left-run / right-run arrays (or a two-state DP: "no change used" / "change used") so you can explain the jump from one to the other.
Test the bridge case explicitly: joining the run ending at i - 1 with the run starting at i + 1 is only valid when nums[i - 1] <= nums[i + 1]; in the strict variant there must be room for a value strictly between the neighbors, i.e. nums[i + 1] - nums[i - 1] >= 2 when the replacement must be an integer (plain < only if any real value is allowed). Otherwise the change only extends one side by one.
Repeat the exercise with the strict-increasing variant to make sure your comparisons flip correctly.

## HOW TO THINK ABOUT IT

Precompute left[i], the length of the non-decreasing run ending at i, and right[i], the length of the run starting at i. If you change element i to any value, it can join the run on its left and the run on its right when a[i-1] <= a[i+1] (a value between them exists); otherwise it can join only the longer of the two. Take the best over all i.

**Basis:** Written for this question (the one-change version of LeetCode 674, Longest Continuous Increasing Subsequence).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(a):
    best = longest_non_decreasing(a)
    for i in range(len(a)):
        for v in candidate_values(a):
            b = a[:i] + [v] + a[i + 1:]
            run = longest_non_decreasing(b)
            best = max(best, run)
    return best
```

| TIME | SPACE |
|---|---|
| O(n^3) | O(n) |

**WHY IT IS SLOW**

O(n^3). The run lengths on both sides of each position, computed once, answer every change in O(1).

## OPTIMAL SOLUTION · Left and right run lengths

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

**FOLLOW-UP 1 The run must be strictly increasing.**

Use < in the run lengths, and when the replaced value must fit between the neighbours check a[i+1] - a[i-1] >= 2 (an integer strictly between them exists) instead of a[i-1] <= a[i+1]. Clarify strict versus non-strict before coding.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Longest Non-Decreasing Subarray with One
# Replacement
#
# Find the maximum length of a contiguous non-decreasing
# subarray if at most one element may be changed to any
# value. Some rounds start from the plain longest-
# increasing-subarray question and add the one-replacement
# follow-up; those versions ask for an increasing run, so
# clarify strict vs non-strict. Input: an integer array
# nums. You may change at most one element to any value.
# Return the maximum length of a contiguous subarray that
# is non-decreasing (nums[i] <= nums[i + 1] for every
# adjacent pair inside it) after the change. Variant:
# other rounds ask for a strictly increasing run and open
# with the no-replacement version (longest increasing
# subarray) before adding the one-replacement follow-up —
# clarify strict vs non-strict before coding. Expect to
# discuss and compare the complexity of several
# approaches; a brute-force first pass is usually followed
# by a request to optimize.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The run must be strictly increasing.
#
# INTERVIEW NOTES
# Precompute left[i], the length of the non-decreasing run
# ending at i, and right[i], the length of the run
# starting at i. If you change element i to any value, it
# can join the run on its left and the run on its right
# when a[i-1] <= a[i+1] (a value between them exists);
# otherwise it can join only the longer of the two. Take
# the best over all i.
#
# BRUTE FORCE (say it first): Try every change and re-scan
# For every position and every candidate value, rewrite
# that element and find the longest non-decreasing run.
# Correct and a good oracle; cubic. Time O(n^3). Space
# O(n).


# BRUTE FORCE (say it first): Try every change and re-scan
# | time O(n^3)
def one_change_brute(a):
    best = 0
    cands = set(a) | {0}
```

```
    for i in range(len(a)):
        for v in cands | {a[i]}:
            b = a[:i] + [v] + a[i + 1 :]
            run = 0
            for k in range(len(b)):
                run = (
                    run + 1
                    if k and b[k - 1] <= b[k]
                    else 1
                )
                best = max(best, run)
    return best


# HELPERS for the base question
def run_lengths(a, ok):
    n = len(a)
    # left[i]: run ending at i, right[i]: starting at i
    left, right = [1] * n, [1] * n
    for i in range(1, n):
        if ok(a[i - 1], a[i]):
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if ok(a[i], a[i + 1]):
            right[i] = right[i + 1] + 1
    return left, right


# BASE QUESTION (optimal): Left and right run lengths
# Idea: left[i] and right[i] are the non-decreasing run
# lengths ending and starting at i. Changing a[i] gives
# left[i-1] + 1 + right[i+1] if a[i-1] <= a[i+1],
# otherwise max(left[i-1], right[i+1]) + 1.
# FOLLOW-UP 1: The run must be strictly increasing.
# Answer: Use < in the run lengths, and when the replaced
# value must fit between the neighbours check a[i+1] -
# a[i-1] >= 2 (an integer strictly between them exists)
# instead of a[i-1] <= a[i+1]. Clarify strict versus non-
# strict before coding.
def longest_with_one_change(a, strict=False):
    n = len(a)
    # nothing to fix in arrays of length 0, 1 or 2
    if n <= 2:
        return n
    # FOLLOW-UP 1: strict=True means strictly increasing
```

```
    # non-decreasing uses <=, strictly increasing uses <
    def ok(x, y):
        return x < y if strict else x <= y

    # can a value fit between the two neighbours?
    gap = 2 if strict else 0
    left, right = run_lengths(a, ok)
    best = 1
    # try changing each element a[i]
    for i in range(n):
        L = left[i - 1] if i else 0
        R = right[i + 1] if i + 1 < n else 0
        # at an end, or when a value fits between the
        # neighbours, the change joins both runs
        if not L or not R or a[i + 1] - a[i - 1] >= gap:
            best = max(best, L + 1 + R)
        # otherwise it joins only the longer run
        else:
            best = max(best, max(L, R) + 1)
    return best


# WARM-UP: no replacement allowed
def longest_without_change(a):
    # the plain longest increasing subarray warm-up
    best = run = 0
    for i, x in enumerate(a):
        run = run + 1 if i and a[i - 1] < x else 1
        best = max(best, run)
    return best


if __name__ == "__main__":
    # change 0 to 1: [1, 1, 3, 4, 5] has length 5
    nums = [1, 0, 3, 4, 5, 2, 3]
    assert longest_with_one_change(nums) == 5
    # tiny arrays
    assert longest_with_one_change([]) == 0
```

```
assert longest_with_one_change([7]) == 1
# strict version needs room between neighbours
strict = longest_with_one_change
assert strict([1, 2, 2, 3, 4], True) == 4
assert strict([1, 1, 1, 1], True) == 2
# the warm-up without any change
assert longest_without_change([1, 3, 5, 4, 7]) == 3
```

## NOTES FROM CANDIDATES

Asked as an onsite coding round in an early-career L3 loop, and as the first technical round of new-grad VOs.
Candidates who wrote a slow version first were asked to optimize, leaving little time for anything else — the problem is short, so interviewers expect the linear approach to arrive quickly.
Edge cases worth confirming: arrays of length 0–2, where the replaced element is the first or last of the run, and whether the replaced value must fit between both neighbors or only one.

## PREPARATION

Write the plain longest-run scan, then extend it with left-run / right-run arrays (or a two-state DP: "no change used" / "change used") so you can explain the jump from one to the other.
Test the bridge case explicitly: joining the run ending at i - 1 with the run starting at i + 1 is only valid when nums[i - 1] <= nums[i + 1]; in the strict variant there must be room for a value strictly between the neighbors, i.e. nums[i + 1] - nums[i - 1] >= 2 when the replacement must be an integer (plain < only if any real value is allowed). Otherwise the change only extends one side by one.
Repeat the exercise with the strict-increasing variant to make sure your comparisons flip correctly.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

Site tags: dp, grid, medium

**MEDIUM | 1Point3Acres | phone-screen | 6 of 23**

# Grid Path Count with Diagonal Moves and Checkpoints

**ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS**

**STUDY: TYPE** **ALL TIME 48.9** **30 DAYS 33** **PRIORITY 35/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** written and tested here for this exact question

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #62 Unique Paths

**LeetCode match:** LC 62 Unique Paths (related)

Frequency: Single report | Last asked: 2026-09-29 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/grid-path-count-three-moves-checkpoints

## QUESTION

Count paths in an n x m matrix from (n-1, 0) to (n-1, m-1) moving up-right, right, or down-right. Follow-ups: count paths visiting all checkpoints; then checkpoints must be visited in order, with the observation that each move changes column, and optimize space.

Matrix of size n x m. Start at (n-1, 0), end at (n-1, m-1).
Allowed moves from (r, c): up-right (r-1, c+1), right (r, c+1), down-right (r+1, c+1); stay inside the matrix.
Return the total number of distinct paths from start to end.
**Follow-up 1:** the matrix contains checkpoints; count paths that visit all checkpoints.
**Follow-up 2:** the checkpoints must be visited in order. Since every move changes the column, each column can contain at most one visited checkpoint. Count the paths and optimize space.

## APPROACH

Write the column-by-column DP where dp[c][r] depends only on dp[c-1][r-1..r+1], then reduce it to a single rolling column.
For checkpoints, practice the "segment" approach: sort by column, and between consecutive checkpoints only allow paths that pass through them (zero out other rows in a checkpoint's column).
Test: n = 1 (only straight right), m = 1, a checkpoint at the start or end, and two checkpoints in the same column.

## HOW TO THINK ABOUT IT

Every move goes exactly one column to the right, so a path visits each column exactly once and the state is just 'which row am I in at this column'. That is a DP over columns with a single rolling array. A checkpoint (r, c) simply says the path must be in row r at column c, so you zero every other row of that column.

**Basis:** LeetCode 62, Unique Paths, with up-right / right / down-right moves, and checkpoints the path must visit.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def count(r, c, seen):
    if c == m - 1:
        done = r == n - 1 and checkpoints <= seen
        return 1 if done else 0
    total = 0
    for dr in (-1, 0, 1):
        nr = r + dr
        if 0 <= nr < n:
            nxt = seen | {(nr, c + 1)}
            total += count(nr, c + 1, nxt)
    return total
answer = count(n - 1, 0, {(n - 1, 0)})
```

| TIME | SPACE |
|---|---|
| O(3^m) | O(m) |

**WHY IT IS SLOW**

Up to 3^m paths. Every move advances one column, so a dp over (column, row) with a rolling array counts them in O(n * m).

## OPTIMAL SOLUTION · Column DP with a rolling array and checkpoint filters

| TIME | SPACE |
|---|---|
| O(n * m) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Two checkpoints share a column.

No path can visit both, because a path is in exactly one row per column, so the answer is 0 (the code returns it before the DP). Checkpoints at the start or end cell are fine: they only filter rows of the first or last column.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Grid Path Count with Diagonal Moves and
# Checkpoints
#
```

```
# Count paths in an n x m matrix from (n-1, 0) to (n-1,
# m-1) moving up-right, right, or down-right. Follow-ups:
# count paths visiting all checkpoints; then checkpoints
# must be visited in order, with the observation that each
# move changes column, and optimize space. Matrix of size
# n x m. Start at (n-1, 0), end at (n-1, m-1). Allowed
# moves from (r, c): up-right (r-1, c+1), right (r, c+1),
# down-right (r+1, c+1); stay inside the matrix. Return
# the total number of distinct paths from start to end.
# Follow-up 1: the matrix contains checkpoints; count
# paths that visit all checkpoints. Follow-up 2: the
# checkpoints must be visited in order. Since every move
# changes the column, each column can contain at most one
# visited checkpoint. Count the paths and optimize space.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Two checkpoints share a column.
#
# INTERVIEW NOTES
# Every move goes exactly one column to the right, so a
# path visits each column exactly once and the state is
# just 'which row am I in at this column'. That is a DP
# over columns with a single rolling array. A checkpoint
# (r, c) simply says the path must be in row r at column
# c, so you zero every other row of that column.
#
# BRUTE FORCE (say it first): Enumerate every path
# Depth-first search over all move sequences, counting
# those that end at the bottom-right with the checkpoints
# visited. Exponential in the number of columns. Time
# O(3^m). Space O(m).


# BRUTE FORCE (say it first): Enumerate every path | time
# O(3^m)
def count_paths_brute(
    n, m, checkpoints=(), ordered=False
):
    cps = [tuple(c) for c in checkpoints]
    total = 0

    def go(r, c, seen):
```

```
        nonlocal total
        if (r, c) in cps:
            seen = seen + [(r, c)]
        if c == m - 1:
            if r == n - 1 and (
                seen == cps
                if ordered
                else sorted(seen) == sorted(cps)
            ):
                total += 1
            return
        for dr in (-1, 0, 1):
            if 0 <= r + dr < n:
                go(r + dr, c + 1, seen)

    go(n - 1, 0, [])
    return total


# HELPERS for the base question
def step(ways):
    # next column from the current column only
    n = len(ways)
    return [(ways[r - 1] if r else 0) + ways[r]
            + (ways[r + 1] if r + 1 < n else 0)
            for r in range(n)]


def keep_row(ways, row):
    # a checkpoint forces the path through one row
```

```
    return [x if r == row else 0
            for r, x in enumerate(ways)]


# FOLLOW-UP 1: Two checkpoints share a column.
# Answer: No path can visit both, because a path is in
# exactly one row per column, so the answer is 0 (the code
# returns it before the DP). Checkpoints at the start or
# end cell are fine: they only filter rows of the first or
# last column.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def checkpoint_rows(checkpoints, ordered):
    # column -> required row; None if impossible
    rows = {}
```

```
    for r, c in checkpoints:
        # two different checkpoints in one column: no path
        if rows.setdefault(c, r) != r:
            return None
    if len(rows) != len(checkpoints):
        return None
    cols = [c for _, c in checkpoints]
    # ordered: they must be listed left to right
    if ordered and cols != sorted(cols):
        return None
    return rows


# BASE QUESTION (optimal): Column DP with a rolling array
# and checkpoint filters
# Idea: ways[r] is the number of paths that are in row r
# at the current column. The next column is ways'[r] =
# ways[r - 1] + ways[r] + ways[r + 1]. At a checkpoint
# column keep only the checkpoint's row.
def count_paths(n, m, checkpoints=(), ordered=False):
    rows = checkpoint_rows(list(checkpoints), ordered)
    if rows is None:
        return 0
    # one rolling column is enough: O(n) space
    ways = [0] * n
    ways[n - 1] = 1
    if 0 in rows:
        ways = keep_row(ways, rows[0])
    for c in range(1, m):
        ways = step(ways)
        if c in rows:
            ways = keep_row(ways, rows[c])
    return ways[n - 1]


# HELPERS for the checks below
def count_paths_table(n, m):
    # full table: ways[c][r] paths reach row r, column c
    ways = [[0] * n for _ in range(m)]
    ways[0][n - 1] = 1
    for c in range(1, m):
        for r in range(n):
            # arrive from up-right, right or down-right
            for pr in (r - 1, r, r + 1):
                if 0 <= pr < n:
```

```
                    ways[c][r] += ways[c - 1][pr]
    return ways[m - 1][n - 1]


if __name__ == "__main__":
    assert count_paths(3, 3) == 2
    assert count_paths_table(3, 3) == 2
    assert count_paths(1, 5) == 1
    assert count_paths(3, 1) == 1
    assert count_paths_table(4, 6) == count_paths(4, 6)
    # force the middle column through row 1
```

```
assert count_paths(3, 3, [(1, 1)]) == 1
# two checkpoints in one column: impossible
assert count_paths(3, 3, [(0, 1), (1, 1)]) == 0
# ordered checkpoints must go left to right
assert count_paths(3, 4, [(1, 1), (1, 2)], True) == 1
assert count_paths(3, 4, [(1, 2), (1, 1)], True) == 0
```

**NOTES FROM CANDIDATES**

Asked in an L4 phone interview.
Every move advances exactly one column, so the path visits each column exactly once — this is the key structural fact the follow-ups build on.
Clarify for follow-up 1 what happens if two checkpoints share a column (no path can visit both) and whether checkpoints are given as a list or marked in the grid.

**PREPARATION**

Write the column-by-column DP where dp[c][r] depends only on dp[c-1][r-1..r+1], then reduce it to a single rolling column.
For checkpoints, practice the "segment" approach: sort by column, and between consecutive checkpoints only allow paths that pass through them (zero out other rows in a checkpoint's column).
Test: n = 1 (only straight right), m = 1, a checkpoint at the start or end, and two checkpoints in the same column.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

Site tags: tree, dp

**MEDIUM | 1Point3Acres | phone-screen | 7 of 23**

# Minimum-Cost Tree Disconnection

**ASKED 2026-09-09 | 41 DAYS AGO | LAST 90 DAYS**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 197/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-09 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/binary-tree-root-leaf-minimum-cut

## QUESTION

**EXAMPLES**

**Negative-weight follow-up**

```
A --(-1)--> B --(-1)--> C
Minimum total cut cost: -2
```

Cutting both edges costs -2, even though cutting just the first edge already separates the root from the original leaf.

## APPROACH

Implement the nonnegative postorder recurrence, then trace a chain and a branching tree while accounting for recursion depth. Implement the two-state negative-weight version and compare it with exhaustive edge-subset enumeration on small trees, including the given negative chain.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask what must be disconnected (root from every leaf), whether the tree is rooted and directed, whether weights can be negative, and what to return for an empty or single-node tree.

**BRUTE FORCE TO SAY OUT LOUD**

Try every subset of edges and keep those that separate the root from all leaves. Exponential.

**OPTIMAL IDEA**

Define best(v) as the cheapest set of cuts in v's subtree that disconnects v from every leaf below it. For each child c with edge weight w, either cut the edge, paying w, or keep it and require best(c) (not allowed if c is a leaf). Sum over children. With non-negative weights that is the answer. With negative weights, once you cut an edge the whole subtree below becomes free, and you may still cut any negative edge there, so the cut option costs w plus the sum of min(0, edge) over that subtree.

**DRY-RUN THESE CASES**

## INTERVIEW-DAY PLAN

A single edge, the negative chain A to B to C (answer -2), a leaf at different depths, a star.

**LIKELY FOLLOW-UPS**

Negative weights, many roots, or recursion depth for a chain tree.

**PITFALLS**

Treating every negative edge as a mandatory cut. Recursing deeply on a chain.

## HOW TO THINK ABOUT IT

On a tree, every decision about a subtree depends only on the edge above it, so define a function of the node that returns the best cost inside its subtree. For each child you either cut the edge above it (pay its weight) or keep it and make the child itself disconnected from the leaves below. With negative weights, a cut edge frees the whole subtree, so you may cut every negative edge in it for free gain.

**Basis:** Written. Assumed setup (confirm it): a rooted tree with edge weights (cuts cost the weight, weights may be negative); cut edges so the root is disconnected from every leaf; a leaf is a non-root node with no children; with no leaves the cost is 0.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(root, edges):
    best = INF
    for k in range(len(edges) + 1):
        for cut in combinations(edges, k):
            tree = remove_edges(edges, cut)
            if not reaches_leaf(tree, root):
                cost = sum(w for _, _, w in cut)
                best = min(best, cost)
    return best
```

| TIME | SPACE |
|---|---|
| O(2^E * n) | O(n) |

**WHY IT IS SLOW**

2^E subsets, each checked with a search: O(2^E * n). A tree DP (cut the edge above a child, or require the child to be disconnected from its leaves) is O(n).

## OPTIMAL SOLUTION · Tree DP: best(v) = cheapest cuts inside v's subtree that disconnect v from its leaves

| TIME | SPACE |
|---|---|
| O(n) | O(h) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

**FOLLOW-UP 1 Weights are non-negative.**

Then free(c) is always 0 and the recurrence collapses to best(v) = sum over children of (w if the child is a leaf, else min(w, best(child))).

**FOLLOW-UP 2 The tree is a long chain and recursion overflows.**

Compute nodes in reverse BFS order (children before parents) and store best and free in arrays, no recursion needed. State the space as O(n).

**FOLLOW-UP 3 Return which edges to cut.**

Remember at each child which option was cheaper, then walk down from the root collecting the cut edges. Cut edges also include every negative edge below any cut edge.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimum-Cost Tree Disconnection
#
# Examples: Negative-weight follow-up: A --(-1)--> B
# --(-1)--> C Minimum total cut cost: -2 Cutting both
# edges costs -2, even though cutting just the first edge
# already separates the root from the original leaf.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Weights are non-negative.
# 2. The tree is a long chain and recursion overflows.
# 3. Return which edges to cut.
#
# INTERVIEW NOTES
# On a tree, every decision about a subtree depends only
# on the edge above it, so define a function of the node
# that returns the best cost inside its subtree. For each
# child you either cut the edge above it (pay its weight)
# or keep it and make the child itself disconnected from
# the leaves below. With negative weights, a cut edge
# frees the whole subtree, so you may cut every negative
# edge in it for free gain.
#
# BRUTE FORCE (say it first): Try every subset of edges
# Enumerate each subset of edges to cut, check that the
# root can reach no leaf, and keep the cheapest.
# Exponential, but exact, so it checks the DP. Time O(2^E
# * n). Space O(n).


from collections import defaultdict


# BRUTE FORCE (say it first): Try every subset of edges |
# time O(2^E * n)
def disconnect_brute(n, edges):
    children = defaultdict(list)
    for u, v, w in edges:
        children[u].append(v)
    leaves = [
        v
        for v in range(n)
        if v != 0 and not children[v]
    ]
    best = None
    for mask in range(1 << len(edges)):
        kept = [
            (u, v)
            for i, (u, v, w) in enumerate(edges)
            if not mask >> i & 1
        ]
        adj = defaultdict(list)
        for u, v in kept:
            adj[u].append(v)
        seen, stack = {0}, [0]
        while stack:
            x = stack.pop()
            for y in adj[x]:
                if y not in seen:
                    seen.add(y)
                    stack.append(y)
        if not any(leaf in seen for leaf in leaves):
            cost = sum(
                w
                for i, (u, v, w) in enumerate(edges)
                if mask >> i & 1
            )
            best = (
                cost
                if best is None
                else min(best, cost)
            )
    return best if best is not None else 0


# HELPERS for the base question
INF = float("inf")


def tree_costs(n, edges):
    # edges are (parent, child, cost); node 0 is the root
    kids = defaultdict(list)
    for u, v, w in edges:
        kids[u].append((v, w))
    # FOLLOW-UP 2: a loop, not recursion: safe on a long
    # chain
    # breadth-first order: parents come before children
    order = [0]
    for u in order:
        for v, _ in kids[u]:
            order.append(v)
    # best[v]: cheapest way to cut v off from its leaves
    # free[v]: sum of every negative cost below v
    best, free = [0] * n, [0] * n
    for v in reversed(order):
        for c, w in kids[v]:
            # 1) cut this edge and the negatives below
            cut = w + free[c]
            # 2) keep it and cut deeper (a leaf cannot)
            keep = best[c] if kids[c] else INF
            best[v] += min(cut, keep)
            free[v] += min(0, w) + free[c]
    return kids, best, free


# BASE QUESTION (optimal): Tree DP: best(v) = cheapest
# cuts inside v's subtree that disconnect v from its
# leaves
# Idea: For a child c with edge weight w there are two
# options: cut the edge, paying w plus whatever extra
# negative-weight edges you can still cut below c (free(c)
# = sum of min(0, weight) over its subtree), or keep the
# edge and require best(c), which is impossible when c is
# a leaf.
```

```
# FOLLOW-UP 1: Weights are non-negative.
# Answer: Then free(c) is always 0 and the recurrence
# collapses to best(v) = sum over children of (w if the
# child is a leaf, else min(w, best(child))).
# FOLLOW-UP 2: The tree is a long chain and recursion
# overflows.
# Answer: Compute nodes in reverse BFS order (children
# before parents) and store best and free in arrays, no
# recursion needed. State the space as O(n).
def disconnect_cost(n, edges):
    # works for negative costs and needs no recursion
    return tree_costs(n, edges)[1][0]


# FOLLOW-UP 3: Return which edges to cut.
# Answer: Remember at each child which option was cheaper,
# then walk down from the root collecting the cut edges.
# Cut edges also include every negative edge below any cut
# edge.
def edges_to_cut(n, edges):
    kids, best, free = tree_costs(n, edges)
    cuts, stack = [], [0]
```

```
    while stack:
        v = stack.pop()
        for c, w in kids[v]:
            keep = best[c] if kids[c] else INF
            if w + free[c] <= keep:
                # cut here, plus negative edges below c
                cuts.append((v, c))
                below = [c]
                while below:
                    x = below.pop()
                    for y, wy in kids[x]:
                        if wy < 0:
                            cuts.append((x, y))
                        below.append(y)
            else:
                stack.append(c)
    return cuts


if __name__ == "__main__":
    # the question's chain A -(-1)-> B -(-1)-> C
    neg = [(0, 1, -1), (1, 2, -1)]
    assert disconnect_cost(3, neg) == -2
    cuts = sorted(edges_to_cut(3, neg))
    assert cuts == [(0, 1), (1, 2)]
    # non-negative costs: cut the cheaper edge
    chain = [(0, 1, 5), (1, 2, 3)]
    assert disconnect_cost(3, chain) == 3
    assert edges_to_cut(3, chain) == [(1, 2)]
    # a single node needs no cut
    assert disconnect_cost(1, []) == 0
    # a long chain does not overflow the stack
    big = [(i, i + 1, 1) for i in range(5000)]
    assert disconnect_cost(5001, big) == 1
```

## NOTES FROM CANDIDATES

Clarify the required result for an empty tree and a single-node tree. No final return convention is specified for these cases.
Include recursion-stack memory in space analysis. The tree can degenerate into a chain; balanced-tree depth is not a worst-case bound.
Coding took place directly in Google Docs with no supplied framework; the candidate also had to define the tree node structure.
The interviewer asked the candidate to think aloud while developing the approach.
For nonnegative weights, use postorder DP. Let F(v) be the minimum cost to disconnect node v from every original leaf below it. An original leaf has F(v) = infinity; an internal node has F(v) = sum(min(weight(v,c), F(c))) over its children. Each branch must be blocked independently. Use a safe infinity representation rather than adding to a fixed-width maximum integer.
For negative weights, cutting an ancestor edge does not remove the opportunity to cut negative edges below it. Track A(v), the minimum optional cut cost when the subtree is already disconnected: A(v) = sum(min(0, weight(v,c)) + A(c)), with A(leaf) = 0. Then use F(v) = sum(min(weight(v,c) + A(c), F(c))), retaining F(leaf) = infinity. This includes beneficial redundant negative cuts

without double counting edges.
Both recurrences process each edge once: O(n) time and O(h) recursive auxiliary space, where h can equal n. A single original root-leaf cannot be disconnected from itself by cutting edges; agree on its result convention before coding.

## PREPARATION

Implement the nonnegative postorder recurrence, then trace a chain and a branching tree while accounting for recursion depth. Implement the two-state negative-weight version and compare it with exhaustive edge-subset enumeration on small trees, including the given negative chain.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

Site tags: array, backtracking, dp, math, medium

**MEDIUM | 1Point3Acres | onsite-coding | 8 of 23**

# Target Expressions from Ordered Digits

**ASKED 2026-08-13 | 68 DAYS AGO | LAST 90 DAYS**

**STUDY: TALK** **ALL TIME 36.5** **30 DAYS 0** **PRIORITY 103/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** written and tested here for this exact question

**LeetCode match:** LC 282 Expression Add Operators

Frequency: Single report | Last asked: 2026-08-13 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/target-expressions-ordered-digits

## QUESTION

**EXAMPLES**

```
For digits [2, 3, 4] and target 14, valid outputs
    include:
2 * (3 + 4)
2 + 3 * 4
```

## APPROACH

Enumerate operator placements and valid parenthesizations for three or four ordered digits.
Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.
Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask whether digits may be concatenated into multi-digit numbers, which operators are allowed, whether parentheses are free, whether the digit order is fixed, and whether to return all expressions or a boolean.

**BRUTE FORCE TO SAY OUT LOUD**

Enumerate every operator placement and every parenthesization for the three or four digits and evaluate each. That is fine for a handful of digits and is what the first stage asks for.

**OPTIMAL IDEA**

Interval DP: for each interval of digits compute a map from value to the expressions that achieve it by splitting at every point and combining left and right with plus, minus and times. Memoize by interval. If only reachability is needed store just the set of values per interval, which is much smaller than the strings.

**DRY-RUN THESE CASES**

[2, 3, 4] with target 14 gives 2*(3+4) and 2+3*4. A target that is unreachable. A single digit. Zeros.

## INTERVIEW-DAY PLAN

**LIKELY FOLLOW-UPS**

N digits (the DP above), division with fractions, or concatenation.

**PITFALLS**

Missing parentheses in the output string. Evaluating with floats.

## HOW TO THINK ABOUT IT

Any expression over a run of digits is two smaller expressions joined by an operator at some split point, so the set of values for an interval is built from the sets of its two halves. Compute values bottom-up per interval and only build text for the target. Be explicit about parentheses: they come from the split structure, so every split is a candidate.

**Basis:** Related to LeetCode 282, Expression Add Operators, but with parentheses allowed and digits not concatenated.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def values(digits):
    if len(digits) == 1:
        return {digits[0]}
    result = set()
    for cut in range(1, len(digits)):
        for a in values(digits[:cut]):
            for b in values(digits[cut:]):
                result |= {a + b, a - b, a * b}
    return result
reachable = target in values(all_digits)
```

| TIME | SPACE |
|---|---|
| **exponential (Catalan(n) *...** exponential (Catalan(n) * 3^(n-1) expressions) | **O(n)** recursion plus the value set |

**WHY IT IS SLOW**

The same sub-interval of digits is recomputed from scratch for every split above it, giving Catalan-number growth. Remembering the values per interval (memoisation) makes it polynomial in the number of distinct values.

## OPTIMAL SOLUTION · Interval DP: all expressions for digits[l:r]

| TIME | SPACE |
|---|---|
| **exponential in the number...** exponential in the number of results; polynomial in the number of distinct values | **O(distinct values per interval)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

**FOLLOW-UP 1 How do you handle N digits where listing every expression is too much?**

Agree on the contract: if you only need to know whether a target is reachable, store just the set of values per interval (a boolean DP over (l, r, value)), which is far smaller than the set of strings. If you need one witness, store one expression string per value. Values are bounded by the digits, so the table stays practical for moderate N.

**FOLLOW-UP 2 What if concatenating digits (23) or division is allowed?**

Concatenation adds base cases: for every l and r the number int(digits[l:r]) is also an atom of the interval. Division adds a fourth operator with a zero check and, if exactness matters, Fraction values instead of floats.

**FOLLOW-UP 3 Only a yes/no answer is needed for many digits.**

Store just the set of reachable values per interval (as target_reachable does) instead of the expression strings; the strings blow up long before the value sets do. Build the expression only for the winning target by remembering one split per value.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Target Expressions from Ordered Digits
#
# Examples: For digits [2, 3, 4] and target 14, valid
# outputs include: 2 * (3 + 4) 2 + 3 * 4
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you handle N digits where listing every
# expression is too much?
# 2. What if concatenating digits (23) or division is
# allowed?
# 3. Only a yes/no answer is needed for many digits.
#
# INTERVIEW NOTES
# Any expression over a run of digits is two smaller
# expressions joined by an operator at some split point,
# so the set of values for an interval is built from the
# sets of its two halves. Compute values bottom-up per
# interval and only build text for the target. Be explicit
# about parentheses: they come from the split structure,
# so every split is a candidate.
#
# BRUTE FORCE (say it first): Recursively try every split
# and operator, without remembering sub-results
# Split the digits at every point, combine all value pairs
# of the two parts with +, - and *, and see whether the
# target appears. Correct, but the same sub-interval is
# recomputed many times; memoising by interval is the fix.
# Time exponential (Catalan(n) * 3^(n-1) expressions).
# Space O(n) recursion plus the value set.
```

```
from collections import defaultdict


# BRUTE FORCE (say it first): Recursively try every split
# and operator, without remembering sub-results | time
# exponential (Catalan(n) * 3^(n-1) expressions)
def _all_values(digits):
    if len(digits) == 1:
        return {digits[0]}
    out = set()
    for i in range(1, len(digits)):
        for a in _all_values(digits[:i]):
            for b in _all_values(digits[i:]):
                out |= {a + b, a - b, a * b}
    return out


def target_reachable_brute(digits, target):
    return target in _all_values(digits)


# HELPERS for the base question
def build(digits):
    # table[(l, r)] = {value: {(text, precedence)}}
    table = {}

    def solve(l, r):
        if (l, r) in table:
            return table[(l, r)]
        res = defaultdict(set)
        if r - l == 1:
            # a single digit is an atom (precedence 3)
            res[digits[l]].add((str(digits[l]), 3))
        # try every split point
        for m in range(l + 1, r):
            for lv, ls in solve(l, m).items():
                for rv, rs in solve(m, r).items():
                    combine(res, lv, ls, rv, rs)
        table[(l, r)] = res
        return res

    return solve(0, len(digits))


def combine(res, lv, ls, rv, rs):
    ops = (("+", 1, lv + rv), ("-", 1, lv - rv),
           ("*", 2, lv * rv))
    for lt, lp in ls:
        for rt, rp in rs:
            for op, prec, val in ops:
                # brackets only where precedence needs
                left = f"({lt})" if lp < prec else lt
                need = rp < prec
                if rp == prec and op == "-":
                    need = True
                right = f"({rt})" if need else rt
                res[val].add((left + op + right, prec))
```

```
# BASE QUESTION (optimal): Interval DP: all expressions
# for digits[l:r]
# Idea: Every expression over digits[l:r] is two sub-
# expressions over a split point joined by +, - or *.
# Memoize, for each interval, a map from numeric value to
# the set of expression strings that produce it.
def expressions_for_target(digits, target):
    found = build(digits).get(target, ())
    return sorted({text for text, _ in found})


# FOLLOW-UP 3: Only a yes/no answer is needed for many
# digits.
```

```
# Answer: Store just the set of reachable values per
# interval (as target_reachable does) instead of the
# expression strings; the strings blow up long before the
# value sets do.
def target_reachable(digits, target):
    # yes/no only: keep just the set of values per range
    values = {}

    def solve(l, r):
        if (l, r) in values:
            return values[(l, r)]
        out = {digits[l]} if r - l == 1 else set()
        for m in range(l + 1, r):
            for a in solve(l, m):
                for b in solve(m, r):
                    out |= {a + b, a - b, a * b}
        values[(l, r)] = out
        return out

    return target in solve(0, len(digits))


if __name__ == "__main__":
    ans = expressions_for_target([2, 3, 4], 14)
    assert "2*(3+4)" in ans and "2+3*4" in ans
    assert target_reachable([2, 3, 4], 14)
    assert not target_reachable([2, 3, 4], 100)
    # subtraction needs brackets on the right
    sub = expressions_for_target([5, 3, 1], 3)
    assert "5-(3-1)" in sub
    assert target_reachable([1] * 8, 8)
```

## NOTES FROM CANDIDATES

The base case is small enough for direct enumeration, but a complicated ad hoc implementation becomes difficult to extend. Keep expression construction separate from numeric evaluation while negotiating the N-digit contract.

If the agreed contract uses every digit exactly once in order, memoize each interval [l, r) as a map from attainable numeric values to the expression strings producing them. For every split point, combine left and right entries with + and *, wrapping composite children as needed; the full-interval target bucket is the answer. Because every distinct expression must be retained, both time and storage are output-sensitive and can grow exponentially even though interval states are memoized.

## PREPARATION

Enumerate operator placements and valid parenthesizations for three or four ordered digits.

Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.

Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

Site tags: string, dp, medium

**MEDIUM | 1Point3Acres | phone-screen | 9 of 23**

# Count Word Decompositions into Chemical Element Symbols

**ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS**

**STUDY: TYPE** **ALL TIME 43.5** **30 DAYS 0** **PRIORITY 81/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** written and tested here for this exact question

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #139 Word Break

**LeetCode match:** LC 139 Word Break (related)

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/periodic-table-word-decomposition

## QUESTION

**EXAMPLES**

**Physics has four stated decompositions**

P - H - Y - Si - C - S P - H - Y - Si - Cs P - H - Y - S - I - C - S P - H - Y - S - I - Cs The returned count is 4.

## APPROACH

Implement a suffix-count or prefix-count formulation and state what each state represents.

Test a word with no decomposition, one decomposition, multiple decompositions, and overlapping one- and two-letter symbols.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask whether matching is case-insensitive, whether to count decompositions or just test for one, and for the dictionary (the periodic table symbols of one and two letters).

**BRUTE FORCE TO SAY OUT LOUD**

Recursive search that tries both a one-letter and a two-letter symbol at each position. Exponential in the worst case.

**OPTIMAL IDEA**

Suffix-count DP: ways[i] is the number of decompositions of word[i:], with ways[n] = 1. ways[i] adds ways[i + 1] if word[i] is a symbol and ways[i + 2] if word[i:i + 2] is a symbol. O(n). Physics gives 4.

**DRY-RUN THESE CASES**

Physics (4), a word with no decomposition (hello), one with exactly one, the empty string, overlapping one- and two-letter symbols such as Si versus S + I.

**LIKELY FOLLOW-UPS**

List the decompositions, handle a larger dictionary with a trie, or count modulo a prime.

## INTERVIEW-DAY PLAN

**PITFALLS**

Forgetting to lowercase before the lookup. Starting the DP with ways[n] = 0.

## HOW TO THINK ABOUT IT

Counting the ways to cut a string into dictionary pieces is Word Break with a count instead of a yes/no. Define ways[i] as the number of decompositions of the suffix starting at i; it depends only on ways[i + 1] and ways[i + 2], because symbols have one or two letters. Compute from the end so every value you need already exists.

**Basis:** A counting version of LeetCode 139, Word Break, with the periodic-table symbols as the dictionary.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def ways(i):
    if i == len(word):
        return 1
    total = 0
    if word[i] in SYMBOLS:
        total += ways(i + 1)
    if word[i:i + 2] in SYMBOLS:
        total += ways(i + 2)
    return total
```

| TIME | SPACE |
|---|---|
| exponential (Fibonacci gr... exponential (Fibonacci growth) | O(n) |

**WHY IT IS SLOW**

The same suffix is reached by many different splits, so the calls grow like the Fibonacci numbers. Computing ways from the end of the word once per position is O(n).

## OPTIMAL SOLUTION · Suffix-count DP

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 How would you list the decompositions, not just count them?

Keep the same suffix recursion but return the list of splits instead of a number, memoized by start index. The count version is O(n); the list version is output-sensitive because there can be exponentially many splits, so agree on whether the interviewer wants a count, one witness, or all of them.

**FOLLOW-UP 2 What if the dictionary is large and symbols can be long?**

Put the dictionary in a trie (or a set plus the maximum word length L) and only look at substrings up to length L from each index. That makes the DP O(n * L) instead of O(n^2), which is the standard Word Break optimization.

**FOLLOW-UP 3 List the decompositions instead of counting them.**

Same recursion, but carry the parts chosen so far and record them at the end. The number of results can be exponential, so ask whether a count or a list is wanted.

**FOLLOW-UP 4 A larger dictionary with long entries.**

Store the dictionary in a set plus the maximum word length L (only look at substrings up to L), or in a trie, so each position costs O(L) lookups instead of O(n).

**FOLLOW-UP 5 Count modulo a prime, or only test whether any decomposition exists.**

Replace addition by addition modulo the prime, or by logical or for a yes/no answer. The DP structure is the same.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Word Decompositions into Chemical
# Element Symbols
#
# Examples: Physics has four stated decompositions: P - H
# - Y - Si - C - S P - H - Y - Si - Cs P - H - Y - S - I -
# C - S P - H - Y - S - I - Cs The returned count is 4.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How would you list the decompositions, not just count
# them?
# 2. What if the dictionary is large and symbols can be
# long?
# 3. List the decompositions instead of counting them.
# 4. A larger dictionary with long entries.
# 5. Count modulo a prime, or only test whether any
# decomposition exists.
#
# INTERVIEW NOTES
# Counting the ways to cut a string into dictionary pieces
# is Word Break with a count instead of a yes/no. Define
# ways[i] as the number of decompositions of the suffix
# starting at i; it depends only on ways[i + 1] and ways[i
# + 2], because symbols have one or two letters. Compute
# from the end so every value you need already exists.
#
# BRUTE FORCE (say it first): Recursion that tries a one-
# letter and a two-letter symbol at every position
# At each position take either the next letter or the next
# two letters if they are symbols, and add up the ways to
# finish. Correct and the oracle; the same suffix is
# solved many times. Time exponential (Fibonacci growth).
# Space O(n).


# HELPERS for the brute force
ELEMENTS = set("""h he li be b c n o f ne na mg al si
p s cl ar k ca sc ti v cr mn fe co ni cu zn ga ge as
se br kr rb sr y zr nb mo tc ru rh pd ag cd in sn sb
te i xe cs ba la ce pr nd pm sm eu gd tb dy ho er tm
yb lu hf ta w re os ir pt au hg tl pb bi po at rn fr
ra ac th pa u np pu am cm bk cf es fm md no lr rf db
sg bh hs mt ds rg cn nh fl mc lv ts og""".split())


# BRUTE FORCE (say it first): Recursion that tries a one-
# letter and a two-letter symbol at every position | time
# exponential (Fibonacci growth)
def decompositions_recursive(word, symbols=ELEMENTS):
    w = word.lower()

    def go(i):
        if i == len(w):
            return 1
        total = 0
        if w[i] in symbols:
            total += go(i + 1)
        if i + 2 <= len(w) and w[i : i + 2] in symbols:
            total += go(i + 2)
        return total

    return go(0)


# BASE QUESTION (optimal): Suffix-count DP
# Idea: ways[i] is the number of decompositions of
# word[i:]. A symbol has one or two letters, so ways[i] =
# ways[i + 1] (if word[i] is a symbol) + ways[i + 2] (if
# word[i:i + 2] is a symbol).
def count_decompositions(word, symbols=ELEMENTS,
                         mod=None):
    w, n = word.lower(), len(word)
    # ways[i]: ways to split the suffix starting at i
    ways = [0] * (n + 2)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        # a one-letter symbol, then a two-letter symbol
        if w[i] in symbols:
            ways[i] += ways[i + 1]
        if i + 2 <= n and w[i:i + 2] in symbols:
            ways[i] += ways[i + 2]
        if mod:
            ways[i] %= mod
    return ways[0]


# FOLLOW-UP 3: List the decompositions instead of counting
# them.
# Answer: Same recursion, but carry the parts chosen so
# far and record them at the end. The number of results
# can be exponential, so ask whether a count or a list is
# wanted.
def list_decompositions(word, symbols=ELEMENTS):
    w, n, memo = word.lower(), len(word), {}

    def splits(i):
        # every way to split the suffix starting at i
        if i == n:
            return [[]]
        if i in memo:
            return memo[i]
        out = []
        for size in (1, 2):
```

```
            part = w[i:i + size]
            if len(part) == size and part in symbols:
                for rest in splits(i + size):
                    out.append([part] + rest)
        memo[i] = out
        return out

    return splits(0)


# FOLLOW-UP 4: A larger dictionary with long entries.
# Answer: Store the dictionary in a set plus the maximum
# word length L (only look at substrings up to L), or in a
# trie, so each position costs O(L) lookups instead of
# O(n).
def count_with_max_len(word, symbols):
    # big dictionary: only look at pieces up to its
    # longest
    longest = max(map(len, symbols))
    w, n = word.lower(), len(word)
    ways = [0] * (n + 1)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        for size in range(1, min(longest, n - i) + 1):
            if w[i:i + size] in symbols:
                ways[i] += ways[i + size]
    return ways[0]


# FOLLOW-UP 5: Count modulo a prime, or only test whether
# any decomposition exists.
# Answer: Replace addition by addition modulo the prime,
# or by logical or for a yes/no answer. The DP structure
```

```
# is the same.
def exists_decomposition(word, symbols=ELEMENTS):
    # yes / no only: the same DP with or instead of +
    return count_decompositions(word, symbols) > 0


if __name__ == "__main__":
    assert count_decompositions("Physics") == 4
    parts = list_decompositions("Physics")
    assert len(parts) == 4
    assert ["p", "h", "y", "si", "c", "s"] in parts
    assert exists_decomposition("Physics")
    # no element starts with x then q
    assert not exists_decomposition("xq")
    assert count_decompositions("Physics", mod=3) == 1
    assert count_with_max_len("Physics", ELEMENTS) == 4
```

## NOTES FROM CANDIDATES

Confirm whether matching is case-sensitive and whether the caller supplies the symbol set or the implementation may treat the periodic table as fixed.
The result counts decompositions, not merely whether at least one decomposition exists.

## PREPARATION

Implement a suffix-count or prefix-count formulation and state what each state represents.
Test a word with no decomposition, one decomposition, multiple decompositions, and overlapping one- and two-letter symbols.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

Site tags: dp, recursion, medium

**MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 10 of 23**

# Fence Painting Minimum Operations (Gemini Team)

**ASKED 2026-04-27 | 176 DAYS AGO | LAST 6 MONTHS**

**STUDY: READ** **NO LEETCODE FREQUENCY** **PRIORITY 200/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-27 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/fence-painting-min-ops

## QUESTION

**EXAMPLES**

```
n = 5, a = [2, 2, 1, 2, 1]
Optimal:
  1) Horizontal stroke at h=1 across [1..5]
      (covers row 1 across whole fence)
  2) Horizontal stroke at h=2 across [1..2]
  3) Vertical stroke at plank 4 (height 2 still
      missing row 2)
Optimal answer: 3.
```

## APPROACH

Drill The canonical largest-rectangle-in-histogram problem — the geometric setup is the same.
Practice the recursive painter pattern on small inputs by hand; mistakes around the band height (min - base) are common.
Be ready for the follow-up: "the brush has width W instead of 1" — extends to bounded horizontal width.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask whether vertical strokes paint one plank fully, whether horizontal strokes must cover consecutive planks at one height, and the sizes involved.

**BRUTE FORCE TO SAY OUT LOUD**

Try all combinations of strokes. Exponential.

**OPTIMAL IDEA**

Divide and conquer on the minimum height: for a range with base height h, painting every plank vertically costs the number of planks; otherwise paint one horizontal band up to the minimum height m (cost 1) and recurse on the segments above m, split where planks are exactly m. The answer is the minimum of the two. O(n^2) worst case, O(n log n) with a sparse table or a min-stack for range minimum queries.

**DRY-RUN THESE CASES**

## INTERVIEW-DAY PLAN

The example giving 3. All equal heights (1). Strictly increasing heights. A single plank.

**LIKELY FOLLOW-UPS**

A brush of width W, or return the strokes.

**PITFALLS**

Forgetting the vertical option, so a tall single plank is overcounted. Off-by-one on the band height.

## HOW TO THINK ABOUT IT

The shortest plank in a range decides the shape: a horizontal band along the bottom, up to that plank's height, is free of regret because every plank in the range needs it. That splits the range at the shortest planks, so recurse on the pieces above the band. The other choice for a range is to paint every plank vertically. Take whichever is cheaper.

**Basis:** Codeforces 448C, Painting Fence. Strokes: a vertical stroke paints one whole plank; a horizontal stroke paints one row across consecutive planks that all reach that height.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def paint(l, r, base):
    if l > r:
        return 0
    m = min(height[l:r + 1])
    vertical = r - l + 1
    horizontal = m - base
    for part in split_at_height(l, r, m):
        horizontal += paint(part.l, part.r, m)
    return min(vertical, horizontal)
```

| TIME | SPACE |
|---|---|
| O(n^2) worst case | O(n) recursion |

**WHY IT IS SLOW**

Finding the minimum by scanning makes a sorted or constant fence quadratic, O(n^2). A sparse table for range-minimum gives O(n log n).

## OPTIMAL SOLUTION · Same recursion with a sparse table for range minimum

| TIME | SPACE |
|---|---|
| O(n log n) | O(n log n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

**FOLLOW-UP 1 The horizontal brush is only W planks wide.**

A band over a range needs ceil(width / W) strokes per row instead of one, so the band cost becomes (m - base) * ceil(len / W). The vertical option and the splitting are unchanged.

**FOLLOW-UP 2 Return the strokes, not just the count.**

Record which option won in each range and rebuild the list by walking the recursion again: horizontal bands as (row, l, r), vertical strokes as (plank).

**FOLLOW-UP 3 Is it the same as largest rectangle in a histogram?**

It shares the idea of splitting at the minimum (a Cartesian tree on the heights), but the objective differs: here you minimise strokes, there you maximise area. **Cost:** the fence recursion is O(n log n) with a sparse table (O(n^2) worst case with a plain scan); the histogram problem has an O(n) stack solution.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Fence Painting Minimum Operations (Gemini
# Team)
#
# Examples: n = 5, a = [2, 2, 1, 2, 1] Optimal: 1)
# Horizontal stroke at h=1 across [1..5] (covers row 1
# across whole fence) 2) Horizontal stroke at h=2 across
# [1..2] 3) Vertical stroke at plank 4 (height 2 still
# missing row 2) Optimal answer: 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The horizontal brush is only W planks wide.
# 2. Return the strokes, not just the count.
# 3. Is it the same as largest rectangle in a histogram?
#
# INTERVIEW NOTES
# The shortest plank in a range decides the shape: a
# horizontal band along the bottom, up to that plank's
# height, is free of regret because every plank in the
# range needs it. That splits the range at the shortest
# planks, so recurse on the pieces above the band. The
# other choice for a range is to paint every plank
# vertically. Take whichever is cheaper.
#
# BRUTE FORCE (say it first): Divide and conquer, scanning
# for the minimum each time
# For a range with base height b: either paint every plank
# vertically (cost = number of planks) or paint a
# horizontal band up to the range minimum (cost = min - b)
# and recurse on the pieces split at the minimum planks.
# Finding the minimum by scanning makes a sorted or
# constant fence quadratic. Time O(n^2) worst case. Space
# O(n) recursion.


# BRUTE FORCE (say it first): Divide and conquer, scanning
# for the minimum each time | time O(n^2) worst case
def paint_brute(a):
    def solve(l, r, base):
        if l > r:
            return 0
        m = min(a[l : r + 1])
        cost = m - base
        i = l
        while i <= r:
            if a[i] == m:
                i += 1
                continue
            j = i
            while j <= r and a[j] != m:
                j += 1
            cost += solve(i, j - 1, m)
            i = j
        return min(r - l + 1, cost)

    return solve(0, len(a) - 1, 0)


# HELPERS for the base question
def sparse_table(a):
    # table[j][i] = index of the smallest plank in a[i : i
    # + 2^j]
    table = [list(range(len(a)))]
    j = 1
    while (1 << j) <= len(a):
        prev, row = table[-1], []
        for i in range(len(a) - (1 << j) + 1):
            x, y = prev[i], prev[i + (1 << (j - 1))]
            row.append(x if a[x] <= a[y] else y)
        table.append(row)
        j += 1
    return table


# FOLLOW-UP 2: Return the strokes, not just the count.
# Answer: Record which option won in each range and
# rebuild the list by walking the recursion again:
# horizontal bands as (row, l, r), vertical strokes as
# (plank).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def paint_strokes(a, width=None):
    # returns (number of strokes, the list of strokes)
    n = len(a)
    if n == 0:
        return 0, []
    table = sparse_table(a)

    def argmin(l, r):
        # smallest plank in a[l..r] in O(1)
        k = (r - l + 1).bit_length() - 1
        x, y = table[k][l], table[k][r - (1 << k) + 1]
        return x if a[x] <= a[y] else y

    def solve(l, r, base):
        # paint planks l..r whose bottom `base` rows are
        # done
        if l > r:
            return 0, []
        m = argmin(l, r)
        size = r - l + 1
        # a band of rows over the whole range needs one
        # stroke per row (or per `width` planks)
        per_row = 1
        if width is not None:
            per_row = -(-size // width)
        band = (a[m] - base) * per_row
        lc, ls = solve(l, m - 1, a[m])
        rc, rs = solve(m + 1, r, a[m])
        horizontal = band + lc + rc
        # or paint every plank by itself, top to bottom
```

```
        if size <= horizontal:
            planks = range(l, r + 1)
            return size, [("vertical", i) for i in planks]
        rows = [("horizontal", h, l, r)
                for h in range(base + 1, a[m] + 1)]
        return horizontal, rows + ls + rs

    return solve(0, n - 1, 0)


# BASE QUESTION (optimal): Same recursion with a sparse
# table for range minimum
# Idea: The recursion is correct; only the minimum lookup
# is slow. A sparse table answers range-minimum-index
# queries in O(1), and each plank becomes the split point
# at most once, so the recursion visits n ranges.
def paint(a, width=None):
    # FOLLOW-UP 1: width= limits the horizontal brush
    return paint_strokes(a, width)[0]


if __name__ == "__main__":
    # the example: 3 strokes
    assert paint([2, 2, 1, 2, 1]) == 3
    assert paint([]) == 0
```

```
    assert paint([3]) == 1
    # a flat fence is one stroke per row
    assert paint([4, 4, 4, 4]) == 4
    # tall thin planks are cheaper to paint upright
    assert paint([5, 0, 5]) == 2
    count, strokes = paint_strokes([2, 2, 1, 2, 1])
    assert count == len(strokes) == 3
    # a narrow brush needs several strokes per row
    assert paint([2, 2, 2, 2], width=2) == 4
    assert paint([2, 2, 2, 2], width=4) == 2
```

## NOTES FROM CANDIDATES

Classic recursive divide-and-conquer: for range [l..r] with base height h:
Option A: paint every plank vertically -> cost r - l + 1.
Option B: paint a horizontal band of height min - h, then recurse on the sub-ranges above the band split by planks at height min.
Take the minimum.
Equivalent to Cartesian-tree DP; runs in O(N log N) with min-stack or sparse table, or $O(N^2)$ with the recursive form.
Verbalize the recursion's correctness: the band optimally covers all planks >= min in the range, then each "taller column block" is solved independently.

## PREPARATION

Drill The canonical largest-rectangle-in-histogram problem — the geometric setup is the same.
Practice the recursive painter pattern on small inputs by hand; mistakes around the band height (min - base) are common.
Be ready for the follow-up: "the brush has width W instead of 1" — extends to bounded horizontal width.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

**MEDIUM | IN APP LC 198 | PracHub | Technical Screen | 11 of 23**

**★ TOP 75: rank 61 in the Medium and Hard priority list**

# ★ Maximize sum without choosing adjacent elements

**POSTED 2026-01-06 | 287 DAYS AGO | OLDER**

**STUDY: TYPE** **ALL TIME 57.2** **30 DAYS 0** **PRIORITY 50/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #198 House Robber

**LeetCode match:** LC 198 House Robber (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-26)

**Original:** https://prachub.com/coding-questions/maximize-sum-without-choosing-adjacent-elements

## QUESTION

Overview: This question evaluates a candidate's understanding of array-based optimization and dynamic programming concepts for computing a maximum sum under non-adjacent selection constraints.

## APPROACH

dp: take = skip_prev + x, skip = max(take_prev, skip_prev); O(n) time, O(1) space.

## HOW TO THINK ABOUT IT

At each element you either take it (then the previous one is off limits) or skip it. Only two numbers matter from the past: the best total if you took the previous element and the best if you did not. Update them together and the whole array is one pass with no table.

**Basis:** LeetCode 198, House Robber: pick elements with no two adjacent to maximise the sum.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def best(i):
    if i >= len(nums):
        return 0
    skip = best(i + 1)
    take = nums[i] + best(i + 2)
    return max(skip, take)
```

| TIME | SPACE |
|---|---|
| O(2^n) | O(n) |

**WHY IT IS SLOW**

## BRUTE FORCE

Two calls per index and the same suffix is solved repeatedly: O(2^n). Keeping only the best total with and without the previous element is O(n) time and O(1) space.

## OPTIMAL SOLUTION · Two running values (take, skip)

| TIME | SPACE |
|---|---|
| O(n) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

## FOLLOW-UP 2 Return which houses to rob.

Keep the table dp[i] = best from i onward (filled from the right) and walk forward: take house i when nums[i] + dp[i + 2] equals dp[i], then jump two; otherwise move one.

## FOLLOW-UP 3 Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Maximize sum without choosing adjacent
# elements
#
# This question evaluates a candidate's understanding of
# array-based optimization and dynamic programming
# concepts for computing a maximum sum under non-adjacent
# selection constraints.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The houses form a circle (LeetCode 213).
# 2. Return which houses to rob.
# 3. Robbing a house also alerts the next k houses.
#
# INTERVIEW NOTES
# At each element you either take it (then the previous
# one is off limits) or skip it. Only two numbers matter
# from the past: the best total if you took the previous
# element and the best if you did not. Update them
# together and the whole array is one pass with no table.
#
# BRUTE FORCE (say it first): Recursion over take or skip
# At index i either skip it (go to i + 1) or take it and
# jump to i + 2, and keep the larger total. Exact and the
# oracle; the same suffixes are solved over and over. Time
# O(2^n). Space O(n).
```

```
# BRUTE FORCE (say it first): Recursion over take or skip
# | time O(2^n)
def _house_brute(nums):
    def go(i):
        if i >= len(nums):
            return 0
        return max(go(i + 1), nums[i] + go(i + 2))

    return go(0)


# BASE QUESTION (optimal): Two running values (take, skip)
# Idea: take is the best total that includes the current
# element, skip the best that excludes it. For the next
# element: new take = skip + x (the previous element must
# have been skipped), new skip = max(take, skip).
def rob_line(nums):
    # take: best when the current element is taken,
```

```
    # skip: best when it is not
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)


# FOLLOW-UP 1: The houses form a circle (LeetCode 213).
# Answer: The first and last are adjacent, so you cannot
# take both: the answer is the better of 'ignore the first
# house' and 'ignore the last house', each solved with the
# linear version.
def rob_circle(nums):
    # the first and last are neighbours: ignore one of
    # them
    if len(nums) == 1:
        return nums[0]
    return max(rob_line(nums[1:]), rob_line(nums[:-1]))


# FOLLOW-UP 2: Return which houses to rob.
# Answer: Keep the table dp[i] = best from i onward
# (filled from the right) and walk forward: take house i
# when nums[i] + dp[i + 2] equals dp[i], then jump two;
# otherwise move one.
def rob_with_choice(nums):
    # dp[i] = best from house i onward, filled from the
    # right
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + 2])
    # walk forward: take house i when it explains dp[i]
    chosen, i = [], 0
    while i < n:
        if nums[i] + dp[i + 2] == dp[i] and nums[i] > 0:
            chosen.append(i)
            i += 2
        else:
            i += 1
    return dp[0], chosen


# FOLLOW-UP 3: Robbing a house also alerts the next k
# houses.
```

```
# Answer: The recurrence becomes dp[i] = max(dp[i + 1],
# nums[i] + dp[i + k + 1]); the O(1) space trick needs a
# window of k + 1 values (a deque), or just keep the O(n)
# table.
def rob_gap(nums, k):
    # taking a house blocks the next k houses (k = 1 is
    # the
    # classic problem): dp[i] = max(dp[i + 1], nums[i] +
    # dp[i + k + 1])
    n = len(nums)
    dp = [0] * (n + k + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1])
    return dp[0]


if __name__ == "__main__":
    assert rob_line([2, 7, 9, 3, 1]) == 12
    assert rob_line([]) == 0
    assert rob_line([5]) == 5
    assert rob_circle([2, 3, 2]) == 3
    assert rob_circle([1, 2, 3, 1]) == 4
    assert rob_circle([4]) == 4
    total, houses = rob_with_choice([2, 7, 9, 3, 1])
    assert total == 12 and houses == [0, 2, 4]
    values = [2, 7, 9, 3, 1]
    assert sum(values[i] for i in houses) == total
    assert rob_gap([2, 7, 9, 3, 1], 1) == 12
    # blocking two houses after each pick
    assert rob_gap([5, 1, 1, 5, 1, 1, 5], 2) == 15
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(nums: list[int]) -> int:
```

### Sample call

```
solution([2, 7, 9, 3, 1])
```

### Test cases

| Input | Expected |
|---|---|
| ([2, 7, 9, 3, 1],) | 12 |
| ([1, 2, 3, 1],) | 4 |

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

Site tags: array, dp, medium

**MEDIUM | 1Point3Acres | onsite-coding | 12 of 23**

# Jump Game with Profit

**STUDY: TYPE** **ALL TIME 56** **30 DAYS 0** **PRIORITY 58/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** written and tested here for this exact question

This question has several parts, so each part has its own answer below.

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #55 Jump Game

**LeetCode match:** LC 55 Jump Game (related)

Frequency: Single report | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/jump-game-with-profit

## QUESTION

**EXAMPLES**

Variant-dependent; the principal recurrence is the one above.

## APPROACH

Drill the reachability-jump, min-jumps, and coin-change family together so the DP shape is automatic.
Practice clarifying "can I skip without consuming a turn" before coding.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

This is underspecified, so ask: is arr[i] the maximum jump (a cap) or an exact jump, is profit collected on landing or when leaving, can you skip a cell for free, are revisits allowed, and what is the goal (reach the end, or maximize profit)?

**BRUTE FORCE TO SAY OUT LOUD**

Recursively try every jump from every cell. Exponential.

**OPTIMAL IDEA**

Pick a reading and say it. Cap reading: best[i] is the best profit standing on i; best[i] = profit[i] + max best[j] over j that can reach i; O(n * max jump), or O(n log n) with a heap and lazy deletion. Exact jump with optional skipping: dp[i] = max(dp[i + 1], profit[i] + dp[i + arr[i]]) from the right, O(n). Plain reachability (no profit) is a greedy farthest-reach scan in O(n).

**DRY-RUN THESE CASES**

A cell with jump 0, an unreachable end, negative profits, a single cell.

**LIKELY FOLLOW-UPS**

Minimum jumps (LeetCode 45), profits that can be negative, or a limit on the number of jumps.

**PITFALLS**

## INTERVIEW-DAY PLAN

Treating the jump as a cap when it is exact, or the reverse. Not defining what skipping costs.

## HOW TO THINK ABOUT IT

This statement is underspecified, so pin it down first (cap or exact jump, when profit is collected, can you skip, what is the goal). The reading with a maximum jump and profit on landing is a DP over positions: the best profit at i is its own profit plus the best of all positions that can reach it. The reading with an exact jump and a free skip is a one-line DP from the right.

**Basis:** Variant of LeetCode 55 (Jump Game) and LeetCode 45 (Jump Game II). The prompt is underspecified, so two readings are given; confirm which one with the interviewer.

## BRUTE FORCE · Reading A: recursion over every jump length

**BRUTE FORCE · PSEUDO-CODE**

```
best = -INF
def go(i, total):
    global best
    if i == n - 1:
        best = max(best, total)
        return
    for step in range(1, arr[i] + 1):
        if i + step < n:
            gain = profit[i + step]
            go(i + step, total + gain)
```

| TIME | SPACE |
|---|---|
| O(max jump ^ n) | O(n) |

**WHY IT IS SLOW**

Every cell branches into up to max-jump choices, so the paths multiply (max jump to the power n). A table of the best profit per cell, filled left to right, is O(n * max jump).

## BRUTE FORCE · Reading B: recursion over 'jump or skip' choices

**BRUTE FORCE · PSEUDO-CODE**

```
def best(i):
    if i >= n:
        return 0
    skip = best(i + 1)
    jump = 0
    if arr[i] > 0:
        jump = profit[i] + best(i + arr[i])
    return max(skip, jump)
```

| TIME | SPACE |
|---|---|
| O(2^n) | O(n) |

**WHY IT IS SLOW**

### BRUTE FORCE · Reading B: recursion over 'jump or skip' choices

Two branches at every cell and the same suffix is solved repeatedly: O(2^n). Computing best(i) once per index from the right is O(n).

### OPTIMAL SOLUTION · Reading A: arr[i] is a maximum jump, profit[i] is gained on landing, maximize total profit to the last index

| TIME | SPACE |
|---|---|
| O(n * maxJump) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

### OPTIMAL SOLUTION · Reading B: arr[i] is an exact jump; at each index you may jump (gaining profit[i]) or skip one cell for free

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

### FOLLOW-UP 1 Speed up reading A to near-linear time.

The inner loop asks for the max of best[j] over all j that can still reach i. Maintain a max-heap of (best[j], j + arr[j]) and pop entries whose reach is below i (lazy deletion): O(n log n). If the jump limit is the same for all cells, a monotonic deque over a sliding window makes it O(n).

### FOLLOW-UP 2 What if you want to know whether the end is reachable at all (plain Jump Game)?

Greedy: scan left to right keeping the farthest reachable index; if i exceeds it you are stuck. O(n) time, O(1) space. Use the DP when profits matter, and the greedy when only reachability matters.

### FOLLOW-UP 3 Only reachability, no profit (LeetCode 55).

Greedy: keep the farthest index reachable so far; if the current index is beyond it you are stuck. O(n), O(1).

### FOLLOW-UP 4 Make the profit DP faster.

best[i] needs the maximum of best[j] over j that can reach i. A max-heap with lazy deletion of positions that can no longer reach i makes it O(n log n); with a constant maximum jump a monotonic deque gives O(n).

### FOLLOW-UP 5 Fewest jumps (LeetCode 45).

BFS by levels or a greedy that tracks the end of the current jump range and the farthest reach, O(n).

### FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Jump Game with Profit
#
# Examples: Variant-dependent; the principal recurrence is
# the one above.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Speed up reading A to near-linear time.
# 2. What if you want to know whether the end is reachable
# at all (plain Jump Game)?
# 3. Only reachability, no profit (LeetCode 55).
# 4. Make the profit DP faster.
# 5. Fewest jumps (LeetCode 45).
#
# INTERVIEW NOTES
# This statement is underspecified, so pin it down first
# (cap or exact jump, when profit is collected, can you
# skip, what is the goal). The reading with a maximum jump
# and profit on landing is a DP over positions: the best
# profit at i is its own profit plus the best of all
# positions that can reach it. The reading with an exact
# jump and a free skip is a one-line DP from the right.
#
# BRUTE FORCE (say it first): Reading A: recursion over
# every jump length
# From each cell try every allowed jump length and add the
# profit of the landing cell; take the best total that
# reaches the last index. Exact and the oracle, and
# exponential. Time O(max jump ^ n). Space O(n).


import heapq


# HELPERS for the brute force
NEG = float("-inf")


# SLOWER VERSION (say it first)
def max_profit_slow(arr, profit):
    # best[i]: most profit when landing on i (arr[j] is
    # the
    # farthest jump from j); O(n^2)
    n = len(arr)
    best = [NEG] * n
    best[0] = profit[0]
```

```
    for i in range(1, n):
        for j in range(i):
            # can j reach i? then landing on i is possible
            if best[j] != NEG and j + arr[j] >= i:
                total = best[j] + profit[i]
                best[i] = max(best[i], total)
    return -1 if best[-1] == NEG else best[-1]


# BASE QUESTION (optimal): Reading A: arr[i] is a maximum
# jump, profit[i] is gained on landing, maximize total
# profit to the last index
# Idea: Let best[i] be the largest profit achievable when
# standing on i. Landing on i from any j < i with j +
```

```
# arr[j] >= i gives best[i] = profit[i] + max(best[j]).
# Unreachable cells stay at minus infinity.
def max_profit_fast(arr, profit):
    # a max-heap of (-best, reach) with lazy deletion
    n = len(arr)
    best = [NEG] * n
    best[0] = profit[0]
    heap = [(-best[0], arr[0])]
    for i in range(1, n):
        # forget cells that can no longer reach i
        while heap and heap[0][1] < i:
            heapq.heappop(heap)
        if heap:
            best[i] = -heap[0][0] + profit[i]
            heapq.heappush(heap, (-best[i], i + arr[i]))
    return -1 if best[-1] == NEG else best[-1]


# VARIANT: reachability only (LeetCode 55)
# FOLLOW-UP 3: Only reachability, no profit (LeetCode 55).
# Answer: Greedy: keep the farthest index reachable so
# far; if the current index is beyond it you are stuck.
# O(n), O(1).
def can_reach_end(arr):
    # LeetCode 55: only reachability, greedy O(n)
    farthest = 0
    for i, jump in enumerate(arr):
        if i > farthest:
            return False
        farthest = max(farthest, i + jump)
    return True
```

```
# VARIANT: fewest jumps (LeetCode 45)
# FOLLOW-UP 5: Fewest jumps (LeetCode 45).
# Answer: BFS by levels or a greedy that tracks the end of
# the current jump range and the farthest reach, O(n).
def fewest_jumps(arr):
    # LeetCode 45: levels of a BFS, greedy O(n)
    jumps = end = farthest = 0
    for i in range(len(arr) - 1):
        farthest = max(farthest, i + arr[i])
        if i == end:
            # we must jump now: extend to the farthest
            # reach
            jumps += 1
            end = farthest
    return jumps


if __name__ == "__main__":
    arr, profit = [2, 3, 1, 1, 4], [1, 5, 2, 3, 4]
    # best path: 0 -> 1 -> 2 -> 3 -> 4 = 15
    assert max_profit_slow(arr, profit) == 15
    assert max_profit_fast(arr, profit) == 15
    # the end cannot be reached
    assert max_profit_slow([1, 0, 1], [1, 1, 1]) == -1
    assert max_profit_fast([1, 0, 1], [1, 1, 1]) == -1
    assert can_reach_end(arr)
    assert not can_reach_end([1, 0, 1])
    assert fewest_jumps(arr) == 2
    # the fast version agrees with the slow one
    import itertools
    for jumps in itertools.product(range(4), repeat=5):
        pr = [1, 2, 3, 2, 1]
        assert max_profit_slow(jumps, pr) == (
            max_profit_fast(jumps, pr))
```

## NOTES FROM CANDIDATES

Standard intern coding round, single source — clarification with interviewer is key (does "skip" advance by 1 or 0? are revisits allowed?).
Common bug: treating arr[i] as range cap rather than exact jump.

## PREPARATION

Drill the reachability-jump, min-jumps, and coin-change family together so the DP shape is automatic.
Practice clarifying "can I skip without consuming a turn" before coding.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

Site tags: grid, dp, medium

**MEDIUM | 1Point3Acres | onsite-coding | 13 of 23**

# Count Same-Color Squares in Matrix

**STUDY: TYPE** **ALL TIME 51.2** **30 DAYS 0** **PRIORITY 68/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** written and tested here for this exact question

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #221 Maximal Square

**LeetCode match:** LC 221 Maximal Square (related)

Frequency: Low | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/count-same-color-squares

## QUESTION

LeetCode 221 variant — count maximal same-color square sub-grids. DP on each cell as the bottom-right corner.

**Input:** m × n grid of colored cells.
**Output:** count of square sub-grids whose cells are all the same color (or maximum side length, depending on prompt).
DP: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) if grid[i][j] == grid[i-1][j] == grid[i][j-1] == grid[i-1][j-1] else 1.
Sum dp[i][j] across the grid for the count variant; max for the side variant.

**EXAMPLES**
The plain "count all-1 squares" DP generalizes to same-color directly.

## APPROACH

Write the all-1-squares DP from memory in under 8 min. Generalize to multi-color in <2 min — only the equality check changes.

## HOW TO THINK ABOUT IT

Let dp[i][j] be the side of the largest same-colour square whose bottom-right corner is (i, j). That square exists only if the three neighbours above, left and above-left have the same colour as the cell and can themselves carry a square, so dp is 1 plus the smallest of their dp values. Summing dp over all cells counts every square once (each cell contributes one square per side length up to dp).

**Basis:** LeetCode 221, Maximal Square, and LeetCode 1277, Count Square Submatrices with All Ones, generalised from 1s to any one colour.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
count = 0
for i, j in all_cells(grid):
    for k in range(1, min(m - i, n - j) + 1):
        cells = square(grid, i, j, k)
        if all(c == grid[i][j] for c in cells):
            count += 1
```

| TIME | SPACE |
|---|---|
| O(m * n * min(m, n)^3) | O(1) |

**WHY IT IS SLOW**

Re-checks the same cells again and again, O(m * n * min(m, n)^3). A dp on the bottom-right corner reuses the neighbours' results in O(m * n).

## OPTIMAL SOLUTION · dp on the bottom-right corner

| TIME | SPACE |
|---|---|
| O(m * n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Return only the largest side instead of the count.

Keep the same dp and take the maximum instead of adding. The common bug is checking only the diagonal neighbour: the up and left neighbours must also match, otherwise a different colour can sit inside the square.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Same-Color Squares in Matrix
#
# LeetCode 221 variant — count maximal same-color square
# sub-grids. DP on each cell as the bottom-right corner.
# Input: m × n grid of colored cells. Output: count of
# square sub-grids whose cells are all the same color (or
# maximum side length, depending on prompt). DP: dp[i][j]
# = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) if
# grid[i][j] == grid[i-1][j] == grid[i][j-1] ==
# grid[i-1][j-1] else 1. Sum dp[i][j] across the grid for
# the count variant; max for the side variant. Examples:
# The plain "count all-1 squares" DP generalizes to same-
# color directly.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the largest side instead of the count.
#
# INTERVIEW NOTES
# Let dp[i][j] be the side of the largest same-colour
# square whose bottom-right corner is (i, j). That square
# exists only if the three neighbours above, left and
# above-left have the same colour as the cell and can
```

```
# themselves carry a square, so dp is 1 plus the smallest
# of their dp values. Summing dp over all cells counts
# every square once (each cell contributes one square per
# side length up to dp).
#
# BRUTE FORCE (say it first): Check every square directly
# For every corner and every side length look at all cells
# in the square. Correct; far too slow for a large grid.
# Time O(m * n * min(m, n)^3). Space O(1).


# BRUTE FORCE (say it first): Check every square directly
# | time O(m * n * min(m, n)^3)
def count_squares_brute(g):
    m, n = len(g), len(g[0]) if g else 0
    total = 0
    for i in range(m):
        for j in range(n):
            for k in range(1, min(m - i, n - j) + 1):
                if all(
                    g[a][b] == g[i][j]
                    for a in range(i, i + k)
                    for b in range(j, j + k)
                ):
                    total += 1
    return total


# HELPERS for the base question
def square_sides(g):
    # side[i][j]: the biggest same-colour square whose
    # bottom-right corner is (i, j); one row at a time
    if not g:
        return []
    prev, out = [0] * len(g[0]), []
    for i, row in enumerate(g):
        cur = [1] * len(row)
        for j, c in enumerate(row):
            # all three neighbours must match this cell
            if i and j and g[i - 1][j] == c \
                    and g[i][j - 1] == c \
                    and g[i - 1][j - 1] == c:
                cur[j] = 1 + min(prev[j], cur[j - 1],
                                 prev[j - 1])
        out.append(cur)
        prev = cur
    return out


# BASE QUESTION (optimal): dp on the bottom-right corner
# Idea: dp[j] holds the best square side ending in the
# current row at column j.
def count_same_color_squares(g):
    # every cell contributes one square per side up to its
    # side
    return sum(sum(row) for row in square_sides(g))


# FOLLOW-UP 1: Return only the largest side instead of the
# count.
# Answer: Keep the same dp and take the maximum instead of
# adding. The common bug is checking only the diagonal
# neighbour: the up and left neighbours must also match,
# otherwise a different colour can sit inside the square.
def largest_side(g):
    # the variant that asks for the biggest square only
    return max((max(row) for row in square_sides(g)),
               default=0)
```

```
if __name__ == "__main__":
    # a 3 x 3 block of one colour: 9 + 4 + 1 squares
    block = ["RRR", "RRR", "RRR"]
    assert count_same_color_squares(block) == 14
    assert largest_side(block) == 3
    # a different colour inside blocks the bigger squares
    holes = ["RRR", "RGR", "RRR"]
    assert largest_side(holes) == 1
    assert count_same_color_squares(holes) == 9
    assert count_same_color_squares([]) == 0
    assert largest_side([]) == 0
```

## NOTES FROM CANDIDATES

Variant reported in both intern VO and an additional NG round. Common bug: assuming all four corners equal is sufficient — need all three neighbors (top, left, top-left) to match the current cell.

## PREPARATION

Write the all-1-squares DP from memory in under 8 min. Generalize to multi-color in <2 min — only the equality check changes.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

Site tags: array, dp, binary-search, sorting, medium

**MEDIUM | IN APP LC 354 | 1Point3Acres | phone-screen | 14 of 23**

# Russian Doll Envelopes

**STUDY: TYPE** **ALL TIME 52.8** **30 DAYS 33** **PRIORITY 28/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** your app's interview approach (LeetCode #354)

**LeetCode match:** LC 354 Russian Doll Envelopes

Frequency: Single report | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/russian-doll-envelopes

## QUESTION

LeetCode 354. Maximum chain of envelopes that strictly fit inside each other. Sort + LIS reduction.

**Input:** list of (width, height) envelopes.
**Output:** longest chain where each envelope strictly fits in the next.
Trick: sort by width ascending, height descending (when widths tie); run LIS (Longest Increasing Subsequence) on heights.
LIS with patience-sort gives O(n log n).

**EXAMPLES**

```
[[5,4],[6,4],[6,7],[2,3]] -> 3.
```

## APPROACH

Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.

## HOW TO THINK ABOUT IT

Sort by width ascending so that only the heights matter, then find the longest strictly increasing subsequence of heights. Envelopes with equal width cannot nest, so sort those by height descending: then a strictly increasing run of heights can never use two envelopes of the same width.

**Basis:** LeetCode 354, Russian Doll Envelopes.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

## BRUTE FORCE

```
envelopes.sort()  # by width
best = [1] * len(envelopes)
for i in range(len(envelopes)):
    for j in range(i):
        w1, h1 = envelopes[j]
        w2, h2 = envelopes[i]
        if w1 < w2 and h1 < h2:
            best[i] = max(best[i], best[j] + 1)
answer = max(best)
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

**WHY IT IS SLOW**

Each envelope is compared with all earlier ones, O(n^2). After sorting, a longest increasing subsequence of the heights with binary search is O(n log n).

## OPTIMAL SOLUTION · Sort, then LIS with binary search

| TIME | SPACE |
|---|---|
| O(n log n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Why sort heights descending when widths tie?

If two envelopes have the same width they cannot nest, but an ascending height order would let the LIS pick both. Descending order makes a strictly increasing height run use at most one envelope per width.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Russian Doll Envelopes
#
# LeetCode 354. Maximum chain of envelopes that strictly
# fit inside each other. Sort + LIS reduction. Input: list
# of (width, height) envelopes. Output: longest chain
# where each envelope strictly fits in the next. Trick:
# sort by width ascending, height descending (when widths
# tie); run LIS (Longest Increasing Subsequence) on
# heights. LIS with patience-sort gives O(n log n).
# Examples: [[5,4],[6,4],[6,7],[2,3]] -> 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why sort heights descending when widths tie?
#
# INTERVIEW NOTES
# Sort by width ascending so that only the heights matter,
# then find the longest strictly increasing subsequence of
# heights. Envelopes with equal width cannot nest, so sort
# those by height descending: then a strictly increasing
# run of heights can never use two envelopes of the same
# width.
#
```

```
# BRUTE FORCE (say it first): O(n^2) DP over sorted
# envelopes
# Sort by width and for each envelope take the best chain
# of an earlier envelope that is strictly smaller in both
# dimensions. Correct; quadratic is too slow for 10^5
# envelopes. Time O(n^2). Space O(n).


from bisect import bisect_left


# CHECKER: the O(n^2) version
def max_envelopes_slow(envelopes):
    # O(n^2) check used for testing
    ordered = sorted(envelopes)
    best = [1] * len(ordered)
    for i in range(len(ordered)):
        for j in range(i):
            wj, hj = ordered[j]
            wi, hi = ordered[i]
            if wj < wi and hj < hi:
                best[i] = max(best[i], best[j] + 1)
    return max(best, default=0)
```

```
# BRUTE FORCE (say it first): O(n^2) DP over sorted
# envelopes | time O(n^2)
def doll_brute(envs):
    envs = sorted(envs)
    best = [1] * len(envs)
    for i in range(len(envs)):
        for j in range(i):
            if (
                envs[j][0] < envs[i][0]
                and envs[j][1] < envs[i][1]
            ):
                best[i] = max(best[i], best[j] + 1)
    return max(best, default=0)


# BASE QUESTION (optimal): Sort, then LIS with binary
# search
# Idea: Sort by (width ascending, height descending).
def max_envelopes(envelopes):
    # sort by width; for equal widths put TALLER first so
    # a
    # strictly increasing height run can use only one of
    # them
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    # tails[k]: smallest ending height of a chain of k + 1
    tails = []
    for _, h in ordered:
        i = bisect_left(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)


if __name__ == "__main__":
    envs = [[5, 4], [6, 4], [6, 7], [2, 3]]
    assert max_envelopes(envs) == 3
    assert max_envelopes([]) == 0
    # equal widths cannot nest, however the heights go
    assert max_envelopes([[1, 1], [1, 2], [1, 3]]) == 1
    assert max_envelopes([[4, 5], [4, 6], [6, 7], [2, 3],
```

```
                          [1, 1]]) == 4
    import itertools
    pts = [(w, h) for w in range(1, 4)
           for h in range(1, 4)]
    for combo in itertools.combinations(pts, 4):
        assert max_envelopes(list(combo)) == (
            max_envelopes_slow(list(combo)))
```

**NOTES FROM CANDIDATES**

Sorting heights descending on width-ties prevents same-width envelopes from chaining — easy to miss.
LIS via bisect_left on a tails array is the standard O(n log n) implementation.

**PREPARATION**

Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

**MEDIUM | IN APP LC 53 | Exponent | 15 of 23**

**★ TOP 75: rank 20 in the Medium and Hard priority list**

# ★ Find the maximum subarray sum.

**ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS**

**STUDY: TYPE** **ALL TIME 64.4** **30 DAYS 41.1** **PRIORITY 14/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** your app's interview approach (LeetCode #53)

**LeetCode match:** LC 53 Maximum Subarray

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +1522 interviews

**Original:** https://www.aced.io/questions/973/maximum-subarray-sum

## QUESTION

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples Input: nums = [2, 3, -2, 4]
Output: 7 Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.
Input: nums = [1, -1, -5, -4]
Output: 1 Explanation: The maximum sum is 1, which is the single element with the highest value.Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?

## APPROACH

Kadane's: cur = max(x, cur + x); best = max(best, cur). O(n).

## HOW TO THINK ABOUT IT

The best subarray ending at position i either extends the best one ending at i - 1 or starts fresh at i; it extends only if the previous best is positive. So keep one number, the best sum ending here, and one answer: cur = max(x, cur + x), best = max(best, cur).

**Basis:** LeetCode 53, Maximum Subarray (Kadane's algorithm). The subarray must be non-empty.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

## BRUTE FORCE

```
def brute(nums):
    best = -INF
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            best = max(best, total)
    return best
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(1) |

**WHY IT IS SLOW**

O(n^2) subarrays. Kadane's algorithm keeps the best sum ending at each position (extend or restart) in O(n) time and O(1) space.

## OPTIMAL SOLUTION · Kadane's algorithm

| TIME | SPACE |
|---|---|
| O(n) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Return the subarray itself.

Remember where the current run started: when starting fresh at i set start = i, and when cur beats best record (start, i). Same single pass.

## FOLLOW-UP 2 The array is circular (LeetCode 918).

The best circular subarray is either an ordinary one or the whole array minus the worst ordinary one (the minimum subarray): max(kadane max, total - kadane min), taking care of the all-negative case.

## FOLLOW-UP 3 Divide and conquer instead of Kadane.

Split the array; the best subarray is entirely left, entirely right, or crosses the middle (best suffix of the left plus best prefix of the right). T(n) = 2T(n/2) + O(n) = O(n log n); useful when the data is split across machines.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find the maximum subarray sum.
#
# Given an array of integers nums, write a function
# maxSubarraySum to find the maximum sum of a contiguous
# subarray within the array and return that maximum sum.
# The subarray must be contiguous, meaning that the
```

```
# elements must appear consecutively in the original
# array. Examples Input: nums = [2, 3, -2, 4] Output: 7
# Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7. Input:
# nums = [1, -1, -5, -4] Output: 1 Explanation: The
# maximum sum is 1, which is the single element with the
# highest value.Challenge 1Can you come up with a solution
# with a O(n) time complexity?Challenge 2Can you come up
# with a solution with a O(1) space complexity?
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subarray itself.
# 2. The array is circular (LeetCode 918).
# 3. Divide and conquer instead of Kadane.
#
# INTERVIEW NOTES
# The best subarray ending at position i either extends
# the best one ending at i - 1 or starts fresh at i; it
# extends only if the previous best is positive. So keep
# one number, the best sum ending here, and one answer:
# cur = max(x, cur + x), best = max(best, cur).
#
# BRUTE FORCE (say it first): Try every subarray
# For each start extend the end with a running sum and
# keep the maximum. Correct and the oracle; quadratic.
# Time O(n^2). Space O(1).


# BRUTE FORCE (say it first): Try every subarray | time
# O(n^2)
def max_subarray_brute(nums):
    best = nums[0]
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            best = max(best, total)
    return best
```

```
# BASE QUESTION (optimal): Kadane's algorithm
# Idea: cur is the best sum of a subarray ending at the
# current element: either the element alone or the element
# added to the previous cur. best is the maximum cur seen.
def max_subarray(nums):
    # Kadane: the best sum of a run ending here either
    # extends
    # the previous run or starts again at this element
    cur = best = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best


# FOLLOW-UP 1: Return the subarray itself.
# Answer: Remember where the current run started: when
# starting fresh at i set start = i, and when cur beats
# best record (start, i). Same single pass.
def max_subarray_range(nums):
    # also say where it is: remember where the run started
    cur, start = nums[0], 0
    best, span = nums[0], (0, 0)
    for i in range(1, len(nums)):
        if nums[i] > cur + nums[i]:
            cur, start = nums[i], i
        else:
            cur += nums[i]
        if cur > best:
            best, span = cur, (start, i)
    return best, span


# FOLLOW-UP 2: The array is circular (LeetCode 918).
```

```
# Answer: The best circular subarray is either an ordinary
# one or the whole array minus the worst ordinary one (the
# minimum subarray): max(kadane max, total - kadane min),
# taking care of the all-negative case.
def max_circular(nums):
    # circular array (LeetCode 918): the best run either
    # does not
    # wrap, or wraps and equals total minus the WORST run
    best = max_subarray(nums)
    total = sum(nums)
    worst = -max_subarray([-x for x in nums])
```

```
    # all negative: the wrap would be empty, so use best
    if total == worst:
        return best
    return max(best, total - worst)


# FOLLOW-UP 3: Divide and conquer instead of Kadane.
# Answer: Split the array; the best subarray is entirely
# left, entirely right, or crosses the middle (best suffix
# of the left plus best prefix of the right). T(n) =
# 2T(n/2) + O(n) = O(n log n); useful when the data is
# split across machines.
def max_subarray_dc(nums, lo=0, hi=None):
    # divide and conquer: left, right, or crossing the
    # middle
    hi = len(nums) if hi is None else hi
    if hi - lo == 1:
        return nums[lo]
    mid = (lo + hi) // 2
    left_best = right_best = float("-inf")
    run = 0
    for i in range(mid - 1, lo - 1, -1):
        run += nums[i]
        left_best = max(left_best, run)
    run = 0
    for i in range(mid, hi):
        run += nums[i]
        right_best = max(right_best, run)
    return max(max_subarray_dc(nums, lo, mid),
               max_subarray_dc(nums, mid, hi),
               left_best + right_best)


if __name__ == "__main__":
    assert max_subarray([2, 3, -2, 4]) == 7
    assert max_subarray([-3, -1, -2]) == -1
    assert max_subarray([5]) == 5
    mixed = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
    assert max_subarray_range(mixed) == (6, (3, 6))
    assert max_circular([5, -3, 5]) == 10
    assert max_circular([-3, -2, -3]) == -2
    assert max_circular([1, -2, 3, -2]) == 3
    samples = ([2, 3, -2, 4], [-1, -2], [1, -1, 1, -1, 5])
    for arr in samples:
        assert max_subarray_dc(arr) == max_subarray(arr)
```

**OFFICIAL PAGE TEXT**

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples

**OFFICIAL PAGE TEXT**

```
Input: nums = [2, 3, -2, 4]
Output: 7
Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.

Input: nums = [1, -1, -5, -4]
Output: 1
Explanation: The maximum sum is 1, which is the single
 element with the highest value.
```

Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?Our solution for finding the maximum subarray sum processes the input array by iterating through it while maintaining two variables: max_current and max_global. The max_current variable keeps track of the maximum sum of the subarray ending at the current position, while max_global stores the maximum sum encountered so far. For each element in the array, we update max_current by taking the maximum of the current element alone or the sum of the current element and max_current. This decision ensures that max_current always holds the maximum sum of the subarray that ends at the current element. We then update max_global to be the maximum of max_global and max_current. If the input array is empty, the function returns 0 since there are no subarrays to consider. This approach uses a simple iteration and a couple of variables, making it both straightforward and efficient. Time Complexity: The solution has a time complexity of O(n), where n is the number of elements in the array. This is because we iterate through the array once, performing constant-time operations (comparison and addition) for each element. Space Complexity: The solution uses O(1) space since it only requires a fixed amount of extra space

for the max_current and max_global variables, regardless of the input size. This makes the solution very efficient in terms of space usage. NoteThis algorithm is known as Kadane's Algorithm.No answers contributed by the community yet.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

**MEDIUM | IN APP LC 416 | Exponent | 16 of 23**

# Partition an array into two sub-arrays with equal sum.

**ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS**

**STUDY: TYPE** **ALL TIME 46.6** **30 DAYS 0** **PRIORITY 75/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** your app's interview approach (LeetCode #416)

**LeetCode match:** LC 416 Partition Equal Subset Sum

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +26 interviews

**Original:** https://www.aced.io/questions/969/partition-array-equal-sum

## QUESTION

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples Input: [2, 4, 11, 5]
Output: True Explanation: The array can be partitioned into two subsets [2, 4, 5] and [11] with equal sum.
Input: [2, 7, 3, 1]
Output: False Explanation: The array cannot be partitioned into two subsets with equal sums.

## APPROACH

Subset-sum DP to total/2 (boolean array or bitset); odd totals are impossible. O(n * sum/2).

## HOW TO THINK ABOUT IT

Two equal parts mean each sums to total / 2, so if the total is odd the answer is no. Otherwise the question is whether some subset reaches total / 2, which is subset-sum. A bitset where bit s says 'sum s is reachable' updates with one shift and one OR per number.

**Basis:** LeetCode 416, Partition Equal Subset Sum (the two parts are subsets, not contiguous sub-arrays; the statement says sub-arrays, so the contiguous reading is given as a follow-up).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

## BRUTE FORCE

```
def brute(nums):
    total = sum(nums)
    if total % 2:
        return False
    for size in range(len(nums) + 1):
        for subset in combinations(nums, size):
            if sum(subset) == total // 2:
                return True
    return False
```

| TIME | SPACE |
|---|---|
| O(2^n * n) | O(n) |

**WHY IT IS SLOW**

2^n subsets. Subset-sum as a reachability table (or a bitset shifted by each number) is O(n * sum).

## OPTIMAL SOLUTION · Subset-sum as a bitset

| TIME | SPACE |
|---|---|
| O(n * sum / 64) with big-integer shifts (O(n * sum) as a boolean table) | O(sum) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Return the two subsets.

Keep, for each reachable sum, the list of indices (or a parent pointer) that first reached it; read off the subset for total / 2 and give the rest to the other side.

## FOLLOW-UP 2 Minimise the difference between the two parts (LeetCode 1049).

Compute all reachable subset sums with the same bitset and choose the reachable s closest to total / 2: the answer is total - 2s.

## FOLLOW-UP 3 The parts must be contiguous sub-arrays (a split point).

Then only one split can work: compute prefix sums and check whether some prefix equals total / 2. O(n), O(1) space; no DP is needed.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Partition an array into two sub-arrays
# with equal sum.
#
# Given an array of integers, determine if it can be
```

```
# partitioned into two subsets such that the sum of the
# elements in each subset is equal. A partition is
# considered valid if it divides the array into two
# subsets where both subsets have the same sum. Examples
# Input: [2, 4, 11, 5] Output: True Explanation: The array
# can be partitioned into two subsets [2, 4, 5] and [11]
# with equal sum. Input: [2, 7, 3, 1] Output: False
# Explanation: The array cannot be partitioned into two
# subsets with equal sums.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the two subsets.
# 2. Minimise the difference between the two parts
# (LeetCode 1049).
# 3. The parts must be contiguous sub-arrays (a split
# point).
#
# INTERVIEW NOTES
# Two equal parts mean each sums to total / 2, so if the
# total is odd the answer is no. Otherwise the question is
# whether some subset reaches total / 2, which is subset-
# sum. A bitset where bit s says 'sum s is reachable'
# updates with one shift and one OR per number.
#
# BRUTE FORCE (say it first): Try every subset
# Enumerate all subsets and check whether any sums to half
# the total. Exact and the oracle; exponential. Time O(2^n
# * n). Space O(n).


import itertools


# BRUTE FORCE (say it first): Try every subset | time
# O(2^n * n)
def can_partition_brute(nums):
    total = sum(nums)
    if total % 2:
        return False
    return any(
        sum(c) == total // 2
```

```
        for r in range(len(nums) + 1)
        for c in itertools.combinations(nums, r)
    )


# BASE QUESTION (optimal): Subset-sum as a bitset
# Idea: reach is an integer whose bit s is set when some
# subset sums to s; start with bit 0. For each number x do
# reach |= reach << x. The answer is whether bit total / 2
# is set.
def can_partition(nums):
    # reachable subset sums as the bits of one integer
    total = sum(nums)
    if total % 2:
        return False
    reach = 1
    for x in nums:
        # every old sum s also gives s + x
        reach |= reach << x
    return bool(reach >> (total // 2) & 1)


# FOLLOW-UP 1: Return the two subsets.
# Answer: Keep, for each reachable sum, the list of
# indices (or a parent pointer) that first reached it;
# read off the subset for total / 2 and give the rest to
# the other side.
def partition_subsets(nums):
    # also return the two subsets: remember which item
    # first
    # reached each sum
```

```
    total = sum(nums)
    if total % 2:
        return None
    half = total // 2
    first = {0: -1}
    for i, x in enumerate(nums):
        for s in list(first):
            if s + x <= half and s + x not in first:
                first[s + x] = i
    if half not in first:
        return None
    chosen, s = [], half
    while s:
        i = first[s]
```

```
        chosen.append(i)
        s -= nums[i]
    left = sorted(chosen)
    rest = [i for i in range(len(nums))
            if i not in chosen]
    return ([nums[i] for i in left],
            [nums[i] for i in rest])


# FOLLOW-UP 2: Minimise the difference between the two
# parts (LeetCode 1049).
# Answer: Compute all reachable subset sums with the same
# bitset and choose the reachable s closest to total / 2:
# the answer is total - 2s.
def min_difference(nums):
    # LeetCode 1049: make the two sums as close as
    # possible
    total = sum(nums)
    reach = 1
    for x in nums:
        reach |= reach << x
    sums = range(total // 2 + 1)
    best = max(s for s in sums if reach >> s & 1)
    return total - 2 * best


# FOLLOW-UP 3: The parts must be contiguous sub-arrays (a
# split point).
# Answer: Then only one split can work: compute prefix
# sums and check whether some prefix equals total / 2.
# O(n), O(1) space; no DP is needed.
def can_split_contiguous(nums):
    # the parts must be a prefix and a suffix: no DP
    # needed
    total, prefix = sum(nums), 0
    if total % 2:
        return False
    for x in nums:
        prefix += x
        if prefix * 2 == total:
            return True
    return False


if __name__ == "__main__":
```

```
    # 2 + 4 + 5 = 11 = the other part
    assert can_partition([2, 4, 11, 5]) is True
    assert can_partition([1, 2, 5]) is False
    assert can_partition([]) is True
    assert can_partition([1]) is False
    left, right = partition_subsets([1, 5, 11, 5])
    assert sum(left) == sum(right) == 11
    assert sorted(left + right) == [1, 5, 5, 11]
    assert partition_subsets([1, 2, 5]) is None
    assert min_difference([1, 6, 11, 5]) == 1
    assert min_difference([2, 4, 11, 5]) == 0
    assert can_split_contiguous([1, 2, 3, 6])
```

```
assert not can_split_contiguous([1, 5, 11, 5])
```

**OFFICIAL PAGE TEXT**

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples

```
Input: [2, 4, 11, 5]
Output: True
Explanation: The array can be partitioned into two sub
sets [2, 4, 5] and [11] with equal sum.

Input: [2, 7, 3, 1]
Output: False
Explanation: The array cannot be partitioned into two
subsets with equal sums.
```

Our solution processes the input array by first calculating the total sum of the array elements and then determining if it can be partitioned into two subsets with equal sums. The core idea is based on dynamic programming. Calculate Total Sum: We start by computing the total sum of the array elements. If the total sum is odd, it is immediately impossible to partition it into two subsets with equal sums, so we return false. Define Target Sum: If the total sum is even, we calculate the target sum for each subset, which is half of the total sum. Dynamic Programming Array: We use a boolean array dp where dp[i] indicates whether a subset sum of i is achievable. We initialize dp[0] to true because a subset sum of 0 is always achievable (by taking no elements). Iterate Through the Array: For each number in the array, we update the dp array in reverse order (from target down to the current number). This reverse iteration ensures that each number is only used once in the current update. If a subset sum i - num was achievable, then adding the current number num makes dp[i] achievable. We update dp[i] to true accordingly. Check Final Result: After processing all numbers, if dp[target] is true, it means there is a subset with the required sum equal to target, indicating that the array can be partitioned into two subsets with equal sums. Otherwise, we return false. The use of a boolean array ensures that we efficiently keep

track of achievable subset sums. Time Complexity: The solution has a time complexity of O(n * target), where n is the number of elements in the array and target is half of the total sum. This is because we iterate through the array and update the dp array for each element. Space Complexity: The solution uses O(target) space for the dp array, where target is half of the total sum. This space is needed to store the achievable subset sums.No answers contributed by the community yet.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

**MEDIUM | IN APP LC 198 | Exponent | 17 of 23**

**★ TOP 75: rank 74 in the Medium and Hard priority list**

# ★ Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.

**ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS**

**STUDY: TYPE** **ALL TIME 57.2** **30 DAYS 0** **PRIORITY 51/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** your app's interview approach (LeetCode #198)

**LeetCode match:** LC 198 House Robber (inferred from short statement)

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age)

**Original:** https://www.aced.io/questions/4646/house-robber

## QUESTION

Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.

## APPROACH

dp: rob = skip_prev + x, skip = max(rob_prev, skip_prev); O(n) time, O(1) space.

## HOW TO THINK ABOUT IT

At each element you either take it (then the previous one is off limits) or skip it. Only two numbers matter from the past: the best total if you took the previous element and the best if you did not. Update them together and the whole array is one pass with no table.

**Basis:** LeetCode 198, House Robber: pick elements with no two adjacent to maximise the sum.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def best(i):
    if i >= len(nums):
        return 0
    skip = best(i + 1)
    take = nums[i] + best(i + 2)
    return max(skip, take)
```

| TIME | SPACE |
|---|---|
| O(2^n) | O(n) |

## BRUTE FORCE

**WHY IT IS SLOW**

Two calls per index and the same suffix is solved repeatedly: O(2^n). Keeping only the best total with and without the previous element is O(n) time and O(1) space.

## OPTIMAL SOLUTION · Two running values (take, skip)

| TIME | SPACE |
|---|---|
| O(n) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

## FOLLOW-UP 2 Return which houses to rob.

Keep the table dp[i] = best from i onward (filled from the right) and walk forward: take house i when nums[i] + dp[i + 2] equals dp[i], then jump two; otherwise move one.

## FOLLOW-UP 3 Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Given n houses in a line with money in
# each, find the maximum amount a robber can steal,
# without stealing from two adjacent houses.
#
# Given n houses in a line with money in each, find the
# maximum amount a robber can steal, without stealing from
# two adjacent houses.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The houses form a circle (LeetCode 213).
# 2. Return which houses to rob.
# 3. Robbing a house also alerts the next k houses.
#
# INTERVIEW NOTES
# At each element you either take it (then the previous
# one is off limits) or skip it. Only two numbers matter
# from the past: the best total if you took the previous
# element and the best if you did not. Update them
# together and the whole array is one pass with no table.
#
# BRUTE FORCE (say it first): Recursion over take or skip
# At index i either skip it (go to i + 1) or take it and
# jump to i + 2, and keep the larger total. Exact and the
# oracle; the same suffixes are solved over and over. Time
```

```
# O(2^n). Space O(n).


# BRUTE FORCE (say it first): Recursion over take or skip
# | time O(2^n)
def _house_brute(nums):
    def go(i):
        if i >= len(nums):
            return 0
        return max(go(i + 1), nums[i] + go(i + 2))

    return go(0)


# BASE QUESTION (optimal): Two running values (take, skip)
# Idea: take is the best total that includes the current
# element, skip the best that excludes it. For the next
# element: new take = skip + x (the previous element must
# have been skipped), new skip = max(take, skip).
def rob_line(nums):
    # take = best sum if we take this house; skip = if we
    # do not
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)


# FOLLOW-UP 1: The houses form a circle (LeetCode 213).
# Answer: The first and last are adjacent, so you cannot
# take both: the answer is the better of 'ignore the first
# house' and 'ignore the last house', each solved with the
# linear version.
def rob_circle(nums):
    # the first and last houses touch: ignore one of them
    if len(nums) == 1:
        return nums[0]
    return max(rob_line(nums[1:]), rob_line(nums[:-1]))


# FOLLOW-UP 2: Return which houses to rob.
# Answer: Keep the table dp[i] = best from i onward
# (filled from the right) and walk forward: take house i
# when nums[i] + dp[i + 2] equals dp[i], then jump two;
# otherwise move one.
def rob_with_choice(nums):
    # dp[i] = best from house i to the end (filled from
    # the right)
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + 2])
    # walk forward: take house i when it explains dp[i]
    houses, i = [], 0
    while i < n:
        if nums[i] > 0 and nums[i] + dp[i + 2] == dp[i]:
            houses.append(i)
            i += 2
        else:
            i += 1
    return dp[0], houses


# FOLLOW-UP 3: Robbing a house also alerts the next k
# houses.
# Answer: The recurrence becomes dp[i] = max(dp[i + 1],
# nums[i] + dp[i + k + 1]); the O(1) space trick needs a
# window of k + 1 values (a deque), or just keep the O(n)
# table.
def rob_alarm(nums, k):
    # robbing a house alerts the next k houses
    n = len(nums)
    dp = [0] * (n + k + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1])
    return dp[0]


if __name__ == "__main__":
    assert rob_line([1, 2, 3, 1]) == 4
    assert rob_line([2, 7, 9, 3, 1]) == 12
    assert rob_line([]) == 0
    assert rob_circle([2, 3, 2]) == 3
    assert rob_circle([1, 2, 3, 1]) == 4
    assert rob_circle([5]) == 5
    total, houses = rob_with_choice([2, 7, 9, 3, 1])
    assert total == 12 and houses == [0, 2, 4]
    assert rob_alarm([2, 7, 9, 3, 1], 1) == 12
    assert rob_alarm([5, 1, 1, 5, 1, 1, 5], 2) == 15
```

**OFFICIAL PAGE TEXT**

You are given an array representing the value in the houses e.g. [100, 20, 40, 70, 80]. Return 100 + 40 + 80 = 220

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

**MEDIUM | IN APP LC 516, 5 | Exponent | 18 of 23**

**★ TOP 75: rank 45 in the Medium and Hard priority list**

# ★ Find the longest palindromic subsequence using dynamic programming.

**ASKED (ANY CO.) ~7 MONTHS AGO | OLDER**

**STUDY: TYPE** **ALL TIME 69.4** **30 DAYS 0** **PRIORITY 26/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** your app's interview approach (LeetCode #516)

**LeetCode match:** LC 516 Longest Palindromic Subsequence (title) / LC 5 Longest Palindromic Substring (statement)

Last asked (any company): 7 months ago, about 2026-03 (Exponent shows only a rough age) | +37 interviews

**Original:** https://www.aced.io/questions/1169/longest-palindromic-subsequence-dynamic-programming

## QUESTION

Given a string s, determine the longest palindromic substring in s. A substring is considered a palindrome if it reads the same forward and backward. A substring is a contiguous sequence of characters within a string. The longest palindromic substring is the substring with the maximum length that satisfies this condition. Examples Input: "abacc" Output: "aba" Explanation: "aba" is longer than "cc"

## APPROACH

Subsequence: dp[i][j] over substrings (LCS of s and reverse(s)). Substring: expand around centers O(n^2) or Manacher O(n).

## HOW TO THINK ABOUT IT

Look at the two ends of a range. If they are equal they both belong to the palindrome and you add 2 to the answer for the inside; if not, one of them is dropped, so take the better of dropping the left or the right end. The state is a range (i, j), giving an O(n^2) table filled from short ranges to long ones.

**Basis:** LeetCode 516, Longest Palindromic Subsequence (not a substring: letters may be skipped).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

## BRUTE FORCE

```
def lps(i, j):
    if i > j:
        return 0
    if i == j:
        return 1
    if s[i] == s[j]:
        return 2 + lps(i + 1, j - 1)
    skip_left = lps(i + 1, j)
    skip_right = lps(i, j - 1)
    return max(skip_left, skip_right)
```

| TIME | SPACE |
|---|---|
| O(2^n) | O(n) |

**WHY IT IS SLOW**

Two branches per step and the same (i, j) range recurs: O(2^n). A table over all ranges is O(n^2) time, with a rolling row for O(n) memory.

## OPTIMAL SOLUTION · Interval DP with a rolling row

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Return the subsequence itself.

Keep the full table and walk from (0, n - 1): equal ends are taken (both go into the answer), otherwise move toward the side with the larger value. O(n^2) memory is needed for the walk back.

## FOLLOW-UP 2 Minimum number of insertions (or deletions) to make the string a palindrome.

It is n - LPS(s): keep the longest palindromic subsequence and add (or remove) the other characters.

## FOLLOW-UP 3 Longest palindromic substring instead (contiguous).

Expand around each of the 2n - 1 centres in O(n^2), or use Manacher's algorithm in O(n). A subsequence DP is not needed.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find the longest palindromic subsequence
# using dynamic programming.
#
# Given a string s, determine the longest palindromic
```

```
# substring in s. A substring is considered a palindrome
# if it reads the same forward and backward. A substring
# is a contiguous sequence of characters within a string.
# The longest palindromic substring is the substring with
# the maximum length that satisfies this condition.
# Examples Input: "abacc" Output: "aba" Explanation: "aba"
# is longer than "cc"
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subsequence itself.
# 2. Minimum number of insertions (or deletions) to make
# the string a palindrome.
# 3. Longest palindromic substring instead (contiguous).
#
# INTERVIEW NOTES
# Look at the two ends of a range. If they are equal they
# both belong to the palindrome and you add 2 to the
# answer for the inside; if not, one of them is dropped,
# so take the better of dropping the left or the right
# end. The state is a range (i, j), giving an O(n^2) table
# filled from short ranges to long ones.
#
# BRUTE FORCE (say it first): Recursion on the two ends
# lps(i, j): if the ends match add 2 to lps(i + 1, j - 1),
# otherwise take the max of lps(i + 1, j) and lps(i, j -
# 1). Exact and the oracle; the same ranges recur
# exponentially often. Time O(2^n). Space O(n).


from itertools import combinations


# BRUTE FORCE (say it first, it also checks the answers)
def brute_lps(s):
    # oracle: the longest palindrome among all
    # subsequences
    for size in range(len(s), 0, -1):
        for picks in combinations(s, size):
            if picks == picks[::-1]:
                return size
    return 0
```

```
# HELPERS for the base question
def lps_table(s):
    # dp[i][j] = longest palindromic SUBSEQUENCE of
    # s[i..j].
    # Equal ends are both used; otherwise drop one end.
    n = len(s)
    dp = [[0] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        dp[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = dp[i + 1][j - 1] + 2
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
    return dp


# BASE QUESTION (optimal): Interval DP with a rolling row
# Idea: dp[i][j] is the length of the longest palindromic
# subsequence of s[i..j]. Fill i from the end to the start
# and j from i + 1 up: equal ends give dp[i + 1][j - 1] +
# 2, otherwise max(dp[i + 1][j], dp[i][j - 1]).
def lps_length(s):
    return lps_table(s)[0][-1] if s else 0


# FOLLOW-UP 1: Return the subsequence itself.
# Answer: Keep the full table and walk from (0, n - 1):
# equal ends are taken (both go into the answer),
# otherwise move toward the side with the larger value.
# O(n^2) memory is needed for the walk back.
def lps_string(s):
    # follow-up: the subsequence itself. Walk the table
    # from
    # the full range and rebuild the left half as we go.
    if not s:
        return ""
    dp = lps_table(s)
    i, j = 0, len(s) - 1
    left, middle = [], ""
    while i <= j:
        if i == j:
            middle = s[i]
            break
        if s[i] == s[j]:
            left.append(s[i])
            i += 1
            j -= 1
        elif dp[i + 1][j] >= dp[i][j - 1]:
            i += 1
        else:
            j -= 1
    half = "".join(left)
    return half + middle + half[::-1]


# FOLLOW-UP 2: Minimum number of insertions (or deletions)
# to make the string a palindrome.
# Answer: It is n - LPS(s): keep the longest palindromic
# subsequence and add (or remove) the other characters.
def min_insertions(s):
    # follow-up: insertions (or deletions) to make a
    # palindrome: everything outside the longest one
    return len(s) - lps_length(s)


# FOLLOW-UP 3: Longest palindromic substring instead
# (contiguous).
# Answer: Expand around each of the 2n - 1 centres in
# O(n^2), or use Manacher's algorithm in O(n). A
# subsequence DP is not needed.
def longest_palindromic_substring(s):
    # follow-up: CONTIGUOUS version. Expand around each of
    # the
    # 2n - 1 centres (a letter, or the gap between two).
    best = ""
    for centre in range(2 * len(s) - 1):
        i = centre // 2
        j = i + centre % 2
        while i >= 0 and j < len(s) and s[i] == s[j]:
            i -= 1
            j += 1
        if j - i - 1 > len(best):
            best = s[i + 1:j]
    return best


if __name__ == "__main__":
```

```
    assert lps_length("bbbab") == 4
    assert lps_length("cbbd") == 2
    assert lps_length("") == 0
    assert lps_string("bbbab") == "bbbb"
    assert lps_string("a") == "a"
    assert min_insertions("abca") == 1
    assert min_insertions("race") == 3
    assert min_insertions("aba") == 0
    assert longest_palindromic_substring("abacc") == "aba"
    assert longest_palindromic_substring("cbbd") == "bb"
    assert longest_palindromic_substring("") == ""
    import random
```

```
    for _ in range(300):
        s = "".join(random.choice("abc")
                    for _ in range(random.randint(0, 10)))
        want = brute_lps(s)
        assert lps_length(s) == want
        found = lps_string(s)
        assert len(found) == want and found == found[::-1]
        # the answer really is a subsequence of s
        it = iter(s)
        assert all(ch in it for ch in found)
```

**OFFICIAL PAGE TEXT**

Given a string s, determine the longest palindromic substring in s. A substring is considered a palindrome if it reads the same forward and backward. A substring is a contiguous sequence of characters within a string. The longest palindromic substring is the substring with the maximum length that satisfies this condition. Examples

```
Input: "abacc"
Output: "aba"
Explanation: "aba" is longer than "cc"
```

Our solution finds the longest palindromic substring by expanding around potential centers in the input string. For each character in the string, we consider it as the center of a palindrome and expand outwards to determine the maximum length of the palindrome centered at that position. We repeat this for both odd-length and even-length palindromes. For each character, we use a helper function expandAroundCenter that takes two indices (left and right) and expands outwards as long as the characters at these indices are equal and within the bounds of the string. We calculate the lengths of palindromes centered at the current character (odd-length palindromes) and between the current character and the next one (even-length palindromes). We track the starting and ending indices of the longest palindromic substring found so far. By comparing the lengths of the palindromes found in each iteration, we can determine the longest palindromic substring. The use of expansion around centers ensures that we examine all potential palindromes efficiently. Time Complexity: The solution has a time complexity of O(n^2), where n is the length of the input string. This is because, for each character in the string, we potentially expand outwards in both directions, resulting in quadratic time complexity in the worst case. Space Complexity: The solution uses O(1) space beyond the

input string. The only extra space used is for a few variables, making it very space-efficient. NoteCheck out our lesson on the two-pointer approach to learn more about what it is and how to use it!No answers contributed by the community yet.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

**MEDIUM | Exponent | 19 of 23**

# Count the number of ways to split a string.

**ASKED (ANY CO.) ~3 YEARS AGO | OLDER**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 178/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** written and tested here for this exact question

**LeetCode match:** LC 1573 Number of Ways to Split a String (likely) (inferred from short statement)

Last asked (any company): 3 years ago, about 2023-10 (Exponent shows only a rough age)

**Original:** https://www.aced.io/questions/3256/count-ways-to-split-string

## QUESTION

Count the number of ways to split a string.

## APPROACH

Count the target character; split positions are products of the gaps between required groups. If it is a generic split count, use DP on prefixes. Clarify the problem first.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

This prompt is vague, so ask first. The common version is LeetCode 1573: split a binary string into three non-empty parts with the same number of ones. Others are splits into equal parts, or splits into palindromes. State which you are solving.

**BRUTE FORCE TO SAY OUT LOUD**

Try every pair of cut positions and count ones in each part. O(n^3), or O(n^2) with prefix sums.

**OPTIMAL IDEA**

For the three-part binary version count the total ones. If it is not divisible by 3 the answer is 0; if there are none the answer is C(n - 1, 2). Otherwise let k be ones per part; the number of zeros between the k-th and (k+1)-th one (gap1) and between the 2k-th and (2k+1)-th one (gap2) give (gap1 + 1) * (gap2 + 1) ways, taken modulo 10^9 + 7. O(n).

**DRY-RUN THESE CASES**

All zeros of length 3, 4 and 5. A string with 3 ones. Ones not divisible by 3. Large n for modulo.

**LIKELY FOLLOW-UPS**

Split into k parts, or count palindromic splits with DP.

**PITFALLS**

Forgetting the all-zeros special case. Not applying the modulus to the product.

## HOW TO THINK ABOUT IT

Ask which split question it is before solving. For 'equal ones' the answer depends only on where the ones are: the cuts have to fall inside the stretches of zeros that separate the groups of ones, so the count is a product of (zeros in the gap + 1). All-zero strings are a special case (choose any two cut points).

**Basis:** The prompt is vague; the usual version is LeetCode 1573, Number of Ways to Split a String (a binary string into three non-empty parts with equal numbers of ones, modulo 10^9 + 7).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(s):
    n, count = len(s), 0
    for a in range(1, n - 1):
        for b in range(a + 1, n):
            parts = (s[:a], s[a:b], s[b:])
            ones = {p.count("1") for p in parts}
            if len(ones) == 1:
                count += 1
    return count
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

**WHY IT IS SLOW**

O(n^2) pairs. The cuts only depend on where the ones are, so the answer is a product of two gap sizes, computed in O(n).

## OPTIMAL SOLUTION · Count the ones, multiply the gap sizes

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Split into m parts with equal ones.

The same argument: each of the m - 1 cut points sits in the gap after a multiple of k ones, so multiply the gap sizes; with no ones at all it is C(n - 1, m - 1).

## FOLLOW-UP 2 Count ways to split into palindromes.

That is a different problem: dp[i] counts splits of the prefix, adding dp[j] for every palindromic s[j:i]; precompute the palindromic substrings with expand-around-center or a DP table in O(n^2).

**FOLLOW-UP 3 The numbers get huge.**

Return the answer modulo 10^9 + 7 and apply the modulus after every multiplication; in languages with fixed-width integers use 64-bit intermediates. **Cost:** O(n) with modular arithmetic; no big integers are needed.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count the number of ways to split a
# string.
#
# Count the number of ways to split a string.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Split into m parts with equal ones.
# 2. Count ways to split into palindromes.
# 3. The numbers get huge.
#
# INTERVIEW NOTES
# Ask which split question it is before solving. For
# 'equal ones' the answer depends only on where the ones
# are: the cuts have to fall inside the stretches of zeros
# that separate the groups of ones, so the count is a
# product of (zeros in the gap + 1). All-zero strings are
# a special case (choose any two cut points).
#
# BRUTE FORCE (say it first): Try every pair of cut
# positions
# With prefix counts of ones, check all pairs (i, j) of
# cut points and count those giving three equal parts.
# Correct and fine for short strings. Time O(n^2). Space
# O(n).


from itertools import combinations
from math import comb


# BRUTE FORCE (say it first, it also checks the answers)
def brute(s, parts):
    # oracle: try every choice of cut points
    count = 0
    for cuts in combinations(range(1, len(s)), parts - 1):
        edges = [0, *cuts, len(s)]
        spans = zip(edges, edges[1:])
        pieces = [s[a:b] for a, b in spans]
        if len({p.count("1") for p in pieces}) == 1:
            count += 1
    return count


def brute_palindromes(s):
    # oracle: try every set of cut points
```

```
    count = 0
    for size in range(len(s)):
        for cuts in combinations(range(1, len(s)), size):
            edges = [0, *cuts, len(s)]
            spans = zip(edges, edges[1:])
            pieces = [s[a:b] for a, b in spans]
            if all(p == p[::-1] for p in pieces):
                count += 1
    return count
```

```
# HELPERS for the base question
MOD = 10 ** 9 + 7


# BASE QUESTION (optimal): Count the ones, multiply the
# gap sizes
# Idea: Let ones be the total number of ones. If it is not
# divisible by 3 the answer is 0; if it is 0, any two of
# the n - 1 cut positions work, C(n - 1, 2).
# FOLLOW-UP 1: Split into m parts with equal ones.
# Answer: The same argument: each of the m - 1 cut points
# sits in the gap after a multiple of k ones, so multiply
# the gap sizes; with no ones at all it is C(n - 1, m -
# 1).
# FOLLOW-UP 3: The numbers get huge.
# Answer: Return the answer modulo 10^9 + 7 and apply the
# modulus after every multiplication; in languages with
# fixed-width integers use 64-bit intermediates. Cost:
# O(n) with modular arithmetic; no big integers are
# needed.
def ways_to_split(s, parts=3, mod=MOD):
    # a binary string cut into `parts` non-empty pieces
    # that
    # hold the same number of 1s (LeetCode 1573 for
    # parts=3)
    n = len(s)
    if n < parts:
        return 0
    # FOLLOW-UP 1: parts=m: m pieces with equal ones
    ones = [i for i, ch in enumerate(s) if ch == "1"]
    if len(ones) % parts:
        return 0
    if not ones:
        # no ones: any choice of parts - 1 cut points
```

```
        # works
        total = comb(n - 1, parts - 1)
        return total % mod if mod else total
    each = len(ones) // parts
    answer = 1
    for cut in range(1, parts):
        # the cut lies in the run of zeros between the
        # last
        # 1 of piece `cut` and the first 1 of the next
        # piece
        gap = ones[cut * each] - ones[cut * each - 1]
        answer *= gap
        # FOLLOW-UP 3: huge counts: answer modulo 10^9 + 7
        if mod:
            answer %= mod
    return answer


# FOLLOW-UP 2: Count ways to split into palindromes.
# Answer: That is a different problem: dp[i] counts splits
# of the prefix, adding dp[j] for every palindromic
# s[j:i]; precompute the palindromic substrings with
# expand-around-center or a DP table in O(n^2).
def count_palindrome_splits(s):
    # follow-up: split into palindromes. dp[i] = ways to
    # cut
    # the first i letters; add dp[j] for every palindrome
    # s[j:i]. The table of palindromes is built first.
    n = len(s)
    pal = [[False] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        for j in range(i, n):
            inner = j - i < 2 or pal[i + 1][j - 1]
            pal[i][j] = s[i] == s[j] and inner
    dp = [1] + [0] * n
    for i in range(1, n + 1):
        for j in range(i):
            if pal[j][i - 1]:
```

```
                dp[i] += dp[j]
    return dp[n]


if __name__ == "__main__":
    assert ways_to_split("10101") == 4
    assert ways_to_split("1001") == 0
```

```
    assert ways_to_split("0000") == 3
    assert ways_to_split("100100010100110") == 12
    assert ways_to_split("1100", parts=2) == 1
    assert ways_to_split("0000", parts=2) == 3
    assert ways_to_split("", parts=3) == 0
    # the modulus keeps the answer small for long strings
    big = "0" * 5000
    assert ways_to_split(big) == comb(4999, 2) % MOD
    assert ways_to_split(big, mod=None) == comb(4999, 2)
    assert count_palindrome_splits("aab") == 2
    assert count_palindrome_splits("aaa") == 4
    assert count_palindrome_splits("") == 1
    import random
    for _ in range(300):
        s = "".join(random.choice("0011")
                    for _ in range(random.randint(3, 10)))
        for parts in (2, 3, 4):
            want = brute(s, parts)
            got = ways_to_split(s, parts, mod=None)
            assert got == want
        t = "".join(random.choice("ab")
                    for _ in range(random.randint(1, 8)))
        want = brute_palindromes(t)
        assert count_palindrome_splits(t) == want
```

**OFFICIAL PAGE TEXT**

Log in or sign up to answer this question.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

**MEDIUM | HackTheRounds | Onsite | 20 of 23**

# Reverse Coin Change

**STUDY: TALK** **ALL TIME 37.9** **30 DAYS 0** **PRIORITY 100/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #518 Coin Change II

**LeetCode match:** Inverse of LC 518

## QUESTION

In standard coin change, given a coins array, we build dp where dp[i] = number of ways to make amount i. Reverse problem: given a dp array, find the original coins array that produced it.

**EXAMPLES**

| Example 1 | |
|---|---|
| **Input** | `dp = [1, 1, 2, 2, 3]` |
| **Output** | `[1, 2]` |
| **Why** | `Coins [1, 2] produce this dp array.` |

| Example 2 | |
|---|---|
| **Input** | `dp = [1, 1, 1, 2, 2, 2, 3]` |
| **Output** | `[1, 3]` |

**CONSTRAINTS**

- 1 <= dp.length <= 1000
- dp[0] = 1

## APPROACH

Rebuild coins left to right: the first amount whose dp exceeds the dp built from coins so far is a new coin

## HOW TO THINK ABOUT IT

Think of building the ways table forward: adding a coin c changes dp[c] by exactly one (the single coin c). So scanning amounts upward, the first amount where the target has more ways than your table can only be explained by a new coin of exactly that value. Add it, rebuild, and continue; at the end compare with the target to detect an impossible input.

**Basis:** The inverse of LeetCode 518, Coin Change II (number of ways to make each amount with unlimited coins).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(dp):
    n = len(dp) - 1
    for k in range(n + 1):
        values = range(1, n + 1)
        for coins in combinations(values, k):
            if ways_table(coins, n) == dp:
                return list(coins)
    return None
```

| TIME | SPACE |
|---|---|
| O(2^n * n * coins) | O(n) |

**WHY IT IS SLOW**

2^n candidate sets, each with an O(n * coins) table. Scanning amounts upward and adding a coin whenever the target has one more way than the table is O(n^2).

## OPTIMAL SOLUTION · Scan upward and add a coin whenever the table falls short

| TIME | SPACE |
|---|---|
| O(n * coins) = O(n^2) worst case | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The array might not come from any set of coins.

Return None (or raise) when dp[0] != 1, when an amount needs more than one extra way, or when the rebuilt table differs from the input at the end; the code does all three checks.

## FOLLOW-UP 2 Order of the coins matters (the dp counts permutations, not combinations).

That is a different recurrence: dp[a] = sum of dp[a - c] over coins. Inverting it is a linear-algebra style problem: at each amount the new coins explain the excess; ask which counting is used.

## FOLLOW-UP 3 Each coin may be used at most once.

The forward recurrence iterates amounts downward. The inverse still scans upward adding a coin when the target exceeds the table, but the table update must use the 0/1 form.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Reverse Coin Change
#
# In standard coin change, given a coins array, we build
# dp where dp[i] = number of ways to make amount i.
# Reverse problem: given a dp array, find the original
# coins array that produced it.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array might not come from any set of coins.
# 2. Order of the coins matters (the dp counts
# permutations, not combinations).
# 3. Each coin may be used at most once.
#
# INTERVIEW NOTES
# Think of building the ways table forward: adding a coin
# c changes dp[c] by exactly one (the single coin c). So
# scanning amounts upward, the first amount where the
# target has more ways than your table can only be
# explained by a new coin of exactly that value. Add it,
# rebuild, and continue; at the end compare with the
# target to detect an impossible input.
#
# BRUTE FORCE (say it first): Try every subset of coin
# values 1..n
# Enumerate each subset of {1..n} as a candidate coin set,
# build its ways table and compare with the target. Exact,
# and the oracle for the fast method. Time O(2^n * n *
# coins). Space O(n).


# HELPERS for the brute force
def ways_table(coins, n):
    # forward: dp[a] = number of combinations making a
    dp = [1] + [0] * n
    for c in coins:
        for a in range(c, n + 1):
            dp[a] += dp[a - c]
    return dp


# BRUTE FORCE (say it first): Try every subset of coin
# values 1..n | time O(2^n * n * coins)
def coins_from_dp_brute(dp):
    n = len(dp) - 1
    for mask in range(1 << n):
        coins = [
            i + 1 for i in range(n) if mask >> i & 1
        ]
        if ways_table(coins, n) == list(dp):
            return coins
    return None


# BASE QUESTION (optimal): Scan upward and add a coin
# whenever the table falls short
# Idea: Keep the ways table for the coins found so far.
# FOLLOW-UP 1: The array might not come from any set of
# coins.
# Answer: Return None (or raise) when dp[0] != 1, when an
# amount needs more than one extra way, or when the
# rebuilt table differs from the input at the end; the
# code does all three checks.
def coins_from_dp(dp):
    # backward: rebuild the table with the coins found so
    # far. The first amount where the real dp is exactly
    # one
    # bigger must be a new coin (it adds the single way
    # "just that coin"). Anything else is impossible.
    n = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    cur, coins = [1] + [0] * n, []
    for a in range(1, n + 1):
        if dp[a] == cur[a]:
            continue
        if dp[a] != cur[a] + 1:
            return None
        coins.append(a)
        for x in range(a, n + 1):
            cur[x] += cur[x - a]
    # the table rebuilt from the coins must equal the
    # input
    return coins if cur == list(dp) else None


# FOLLOW-UP 2: Order of the coins matters (the dp counts
# permutations, not combinations).
# Answer: That is a different recurrence: dp[a] = sum of
# dp[a - c] over coins. Inverting it is a linear-algebra
# style problem: at each amount the new coins explain the
# excess; ask which counting is used.
def ordered_table(coins, n):
    # follow-up: the order of coins matters
    # (permutations):
    # dp[a] = sum of dp[a - c] over the coins
    dp = [1] + [0] * n
    for a in range(1, n + 1):
        dp[a] = sum(dp[a - c] for c in coins if c <= a)
    return dp


# FOLLOW-UP 2: Order of the coins matters (the dp counts
# permutations, not combinations).
# Answer: That is a different recurrence: dp[a] = sum of
# dp[a - c] over coins. Inverting it is a linear-algebra
# style problem: at each amount the new coins explain the
# excess; ask which counting is used.
def coins_from_ordered(dp):
    n = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    coins = []
    for a in range(1, n + 1):
        # what the known coins already explain at amount a
        known = sum(dp[a - c] for c in coins)
        if dp[a] == known + 1:
            coins.append(a)
        elif dp[a] != known:
            return None
    again = ordered_table(coins, n)
    return coins if again == list(dp) else None


# FOLLOW-UP 3: Each coin may be used at most once.
# Answer: The forward recurrence iterates amounts
# downward. The inverse still scans upward adding a coin
# when the target exceeds the table, but the table update
# must use the 0/1 form.
def once_table(coins, n):
    # follow-up: each coin used at most once; amounts are
    # visited downward in the forward step
    dp = [1] + [0] * n
    for c in coins:
        for a in range(n, c - 1, -1):
            dp[a] += dp[a - c]
```

```
    return dp


# FOLLOW-UP 3: Each coin may be used at most once.
# Answer: The forward recurrence iterates amounts
# downward. The inverse still scans upward adding a coin
# when the target exceeds the table, but the table update
# must use the 0/1 form.
def coins_from_once(dp):
    n = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    cur, coins = [1] + [0] * n, []
    for a in range(1, n + 1):
        if dp[a] == cur[a]:
            continue
        if dp[a] != cur[a] + 1:
            return None
        coins.append(a)
        for x in range(n, a - 1, -1):
            cur[x] += cur[x - a]
    return coins if cur == list(dp) else None


if __name__ == "__main__":
    assert coins_from_dp([1, 1, 2, 2, 3]) == [1, 2]
    assert coins_from_dp([1, 1, 1, 2, 2, 2, 3]) == [1, 3]
    assert coins_from_dp([1]) == []
    assert coins_from_dp([]) is None
    # not a table: dp[0] must be 1
    assert coins_from_dp([2, 1]) is None
    # an amount that jumps by more than one new way
    assert coins_from_dp([1, 3]) is None
    # looks fine early, but the whole table does not match
    assert coins_from_dp([1, 1, 2, 2, 5]) is None
    table = ordered_table([1, 2], 6)
    assert coins_from_ordered(table) == [1, 2]
    assert coins_from_ordered([1, 1, 2, 3, 5]) == [1, 2]
    table = once_table([1, 2, 3], 6)
    assert coins_from_once(table) == [1, 2, 3]
    import random
    for _ in range(300):
        size = random.randint(0, 4)
        coins = sorted(random.sample(range(1, 10), size))
        n = random.randint(0, 14)
```

```
        small = [c for c in coins if c <= n]
        table = ways_table(coins, n)
        assert coins_from_dp(table) == small
        table = ordered_table(coins, n)
        assert coins_from_ordered(table) == small
        table = once_table(coins, n)
        assert coins_from_once(table) == small
        junk = [1]
        junk += [random.randint(0, 3) for _ in range(n)]
        found = coins_from_dp(junk)
        if found is not None:
            assert ways_table(found, n) == junk
```

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

Site tags: dp, bit-manipulation, hard

**HARD | 1Point3Acres | phone-screen, onsite-coding | 21 of 23**

# Count Perfect Wake Numbers up to N

**ASKED 2026-05-07 | 166 DAYS AGO | LAST 6 MONTHS**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 187/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-05-07 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/perfect-wake-number-count

## QUESTION

**EXAMPLES**

```
196   -> perfect (digits 1,9,6 all distinct, no 0, 9
    is not a valley)
23    -> perfect
12463 -> perfect (1<2, 2<4, 4>6? No, 4<6, 6>3 — 6 is
    not a valley because its right neighbor 3 is
    smaller)
1546  -> not perfect (4 is adjacent to 5 and 6, both
    strictly larger)
320   -> not perfect (contains 0)
34321 -> not perfect (digit 3 repeats)
```

## APPROACH

Drill digit DP templates: classic digit-DP problems — "count digit one", "non-negative integers without consecutive ones", "numbers at most N given digit set", "numbers with repeated digits" — cover "count integers <= n satisfying digit predicate." Practice bitmask-over-digits + tight-flag state machines. Memorize the standard skeleton: recurse over positions, accept any digit <= n's digit when tight, else any digit 1..9.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask for the exact definition: no zero digit, all digits distinct, no interior digit smaller than both neighbors. Ask for the range of N and whether to count 1..N inclusive.

**BRUTE FORCE TO SAY OUT LOUD**

Test every number from 1 to N with a digit check. O(N * d), fine for small N.

**OPTIMAL IDEA**

Digit DP over positions with state (used-digit mask, last digit, whether the digits have started falling, tight flag). Numbers with fewer digits than N are counted without a tight constraint, and zero is never allowed, so there are no leading-zero cases. Digits must rise then fall, which is exactly the no-valley rule. There is also a closed form of C(9, k) * 2^(k - 1) for k-digit numbers.

## INTERVIEW-DAY PLAN

**DRY-RUN THESE CASES**

196, 23, 12463 perfect; 1546, 320, 34321 not perfect. N = 9, N = 10, N a power of ten.

**LIKELY FOLLOW-UPS**

A range [A, B] (inclusion-exclusion), or different digit rules.

**PITFALLS**

Forgetting the zero rule. Letting a digit repeat across the tight and free branches.

## HOW TO THINK ABOUT IT

Distinct digits with no zero and no valley means the digits go up and then down (they can only change direction once). That structure is both a closed form for each length and a state for a digit DP: position, which digits are used, the last digit, and whether the digits have started falling. The tight flag handles the bound N.

**Basis:** Digit DP, like LeetCode 1012 (Numbers With Repeated Digits) and 233. Definition from the report: no zero digit, all digits distinct, and no interior digit is a valley (strictly smaller than both neighbours).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def is_perfect(x):
    d = list(map(int, str(x)))
    if 0 in d or len(set(d)) < len(d):
        return False
    for i in range(1, len(d) - 1):
        if d[i] < d[i - 1] and d[i] < d[i + 1]:
            return False  # a valley
    return True
count = 0
for x in range(1, N + 1):
    count += is_perfect(x)
```

| TIME | SPACE |
|---|---|
| O(N * d) | O(1) |

**WHY IT IS SLOW**

O(N * digits): hopeless for N around 10^9. A digit DP counts by position, used digits, last digit and direction in time independent of N's size.

## OPTIMAL SOLUTION · Part 1: test a single number

| TIME | SPACE |
|---|---|
| O(d) | O(d) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## OPTIMAL SOLUTION · Part 2: digit DP with tight flag, used-digit mask, last digit and direction

| TIME | SPACE |
|---|---|
| O(d * 2^9 * 9 * 9) | O(2^9 * 9) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Why does the DP only need a 'falling' flag instead of the full history?

A valley needs a digit that is lower than the digit before it and then higher than the one after it. Once any digit has decreased, a later increase would create exactly that valley, so the only information to carry is whether we have already gone down. The used mask and last digit cover the other two rules.

## FOLLOW-UP 2 How do you count the numbers in a range [A, B]?

Use inclusion-exclusion: count(B) - count(A - 1). The DP is a function of the upper bound only, and it is cheap enough (a few thousand states) to call twice.

## FOLLOW-UP 3 What is the closed form without a DP?

Since digits are distinct and unimodal, pick which k digits (from 1..9) appear and the peak is the largest; each of the other k - 1 digits goes either left or right of the peak: 2^(k-1) arrangements per set. So the count of perfect numbers with k digits is C(9, k) * 2^(k-1). Only the 'at most N' bound needs the DP.

## FOLLOW-UP 4 Is there a closed form for k-digit numbers?

Pick the k distinct digits from 1..9 (C(9, k) ways); the largest must be the peak, and each of the other k - 1 digits goes either left or right of it in sorted order, so 2^(k-1) arrangements: C(9, k) * 2^(k-1). Only the bound N needs the DP.

## FOLLOW-UP 5 Count perfect numbers in a range [A, B].

Compute count(B) - count(A - 1); the DP depends on the upper bound only.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Perfect Wake Numbers up to N
#
# Examples: 196 -> perfect (digits 1,9,6 all distinct, no
# 0, 9 is not a valley) 23 -> perfect 12463 -> perfect (1<
2,
# 2<4, 4>6? No, 4<6, 6>3 — 6 is not a valley because its
# right neighbor 3 is smaller) 1546 -> not perfect (4 is
# adjacent to 5 and 6, both strictly larger) 320 -> not
# perfect (contains 0) 34321 -> not perfect (digit 3
# repeats)
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the DP only need a 'falling' flag instead of
# the full history?
# 2. How do you count the numbers in a range [A, B]?
# 3. What is the closed form without a DP?
# 4. Is there a closed form for k-digit numbers?
# 5. Count perfect numbers in a range [A, B].
#
# INTERVIEW NOTES
# Distinct digits with no zero and no valley means the
# digits go up and then down (they can only change
# direction once). That structure is both a closed form
# for each length and a state for a digit DP: position,
# which digits are used, the last digit, and whether the
# digits have started falling. The tight flag handles the
# bound N.
#
# BRUTE FORCE (say it first): Test every number from 1 to
# N
# Apply the three rules to each number. Correct and the
# oracle; it is hopeless for N around 10^9. Time O(N * d).
# Space O(1).


from functools import lru_cache
from math import comb


# BRUTE FORCE (say it first): Test every number from 1 to
# N | time O(N * d)
def count_perfect_brute(N):
    return sum(
        1 for x in range(1, N + 1) if is_perfect(x)
    )
```

```
# HELPERS for the base question
# FOLLOW-UP 4: Is there a closed form for k-digit numbers?
# Answer: Pick the k distinct digits from 1..9 (C(9, k)
# ways); the largest must be the peak, and each of the
# other k - 1 digits goes either left or right of it in
# sorted order, so 2^(k-1) arrangements: C(9, k) *
# 2^(k-1).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def with_k_digits(k):
    # choose the k digits; the largest is the peak and
    # each
    # other digit goes left or right of it
    return comb(9, k) * 2 ** (k - 1) if k >= 1 else 0


# BASE QUESTION (optimal): test a single number
# Idea: Check the three rules directly on the digit list:
# no '0', all digits distinct, and for each interior digit
# not (left > digit < right). The first and last digits
# are never valleys.
def is_perfect(n):
    # distinct digits, no zero, and no valley: no digit is
    # smaller than both of its neighbours
    d = list(map(int, str(n)))
    if 0 in d or len(set(d)) != len(d):
        return False
    return all(not (d[i - 1] > d[i] < d[i + 1])
               for i in range(1, len(d) - 1))
```

```
# PART 2: count the perfect numbers up to N
# FOLLOW-UP 2: How do you count the numbers in a range [A,
# B]?
# Answer: Use inclusion-exclusion: count(B) - count(A -
# 1). The DP is a function of the upper bound only, and it
# is cheap enough (a few thousand states) to call twice.
def count_up_to(n):
    if n < 1:
        return 0
    digits = list(map(int, str(n)))
    size = len(digits)
    # all shorter numbers: closed form
    total = sum(with_k_digits(k) for k in range(1, size))
```

```
    @lru_cache(None)
    def go(pos, mask, last, falling, tight):
        # count completions of a number of full length
        if pos == size:
            return 1
        out = 0
        limit = digits[pos] if tight else 9
        for d in range(1, limit + 1):
            if mask >> d & 1:
                continue
            fall = falling
            if pos:
                if d > last and falling:
                    # going up after going down: a valley
                    continue
                if d < last:
                    fall = True
            out += go(pos + 1, mask | 1 << d, d, fall,
                      tight and d == digits[pos])
        return out

    # numbers with the same length as n, not above n
    return total + go(0, 0, 0, False, True)


# FOLLOW-UP 5: Count perfect numbers in a range [A, B].
# Answer: Compute count(B) - count(A - 1); the DP depends
# on the upper bound only.
def count_between(a, b):
    # inclusion-exclusion over the upper bounds
    return count_up_to(b) - count_up_to(a - 1)


if __name__ == "__main__":
    for ok in (196, 23, 12463):
        assert is_perfect(ok)
    for bad in (1546, 320, 34321):
        assert not is_perfect(bad)
    # the DP agrees with checking every number
    for n in (0, 9, 10, 99, 100, 999, 5000, 20000):
        every = range(1, n + 1)
        brute = sum(is_perfect(i) for i in every)
        assert count_up_to(n) == brute
    assert count_between(10, 99) == (
```

```
        count_up_to(99) - count_up_to(9))
    # 1-digit and 2-digit counts from the closed form
    assert with_k_digits(1) == 9
    assert with_k_digits(2) == 72
```

## NOTES FROM CANDIDATES

Part 1 is a linear scan; Part 2 is digit DP with state (position, mask of used digits, last digit, is_tight against n). Track whether the previous step went up or down to detect a future valley.
Multiple candidates report not finishing Part 2 in the 45-minute window. The interviewer is generally patient and dispenses hints ("think DP" / "think digit DP") when you stall.
Watch out: the "no valley" rule is about interior digits, so the first and last digit are never themselves the valley; they can still create one for their neighbor.
The full prompt is short and recent — expect the follow-up to vary (e.g. count exactly-K-digit Perfect Wake Numbers).

## PREPARATION

Drill digit DP templates: classic digit-DP problems — "count digit one", "non-negative integers without consecutive ones", "numbers at most N given digit set", "numbers with repeated digits" — cover "count integers <= n satisfying digit predicate."
Practice bitmask-over-digits + tight-flag state machines.
Memorize the standard skeleton: recurse over positions, accept any digit <= n's digit when tight, else any digit 1..9.

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

Site tags: tree, dp, dfs, hard

**HARD | IN APP LC 834 | 1Point3Acres | onsite-coding | 22 of 23**

# Tree Distance Sum (Re-root DP)

**ASKED 2026-01-19 | 274 DAYS AGO | OLDER**

**STUDY: TALK** **ALL TIME 30.5** **30 DAYS 0** **PRIORITY 124/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** your app's interview approach (LeetCode #834)

**LeetCode match:** LC 834 Sum of Distances in Tree

Frequency: Single report | Last asked: 2026-01-19 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/tree-distance-sum-reroot

## QUESTION

Onsite coding round 1: given an undirected tree on N nodes, return for every node the sum of distances to all other nodes. Equivalent to LC 834. Interviewer expects baseline O(n²) DFS followed by the O(n) re-root DP.

**Input:** undirected tree with N nodes and N-1 edges.
**Output:** array ans where ans[u] is the sum of shortest-path distances from u to every other node.
Walk through baseline (BFS/DFS from every node, O(n²)) before optimizing.
Final solution should be O(n) via two-pass re-root DP.

**EXAMPLES**

```
Edges: [[0,1],[0,2],[2,3],[2,4],[2,5]]
ans:   [8, 12, 6, 10, 10, 10]
```

## APPROACH

Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes. Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.
Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).

## HOW TO THINK ABOUT IT

Running a BFS from every node costs O(n^2). Notice that moving the root from a parent p to its child v changes every distance by exactly one: the nodes inside v's subtree get closer by 1, all the others get farther by 1. So compute the answer for one root, then derive each child's answer from its parent's in O(1) using the subtree sizes.

**Basis:** LeetCode 834, Sum of Distances in Tree.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(n, adj):
    ans = [0] * n
    for s in range(n):
        dist = bfs(adj, s)
        ans[s] = sum(dist)
    return ans
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

**WHY IT IS SLOW**

n BFS runs of O(n) each, O(n^2). Moving the root across one edge changes every distance by exactly 1, so two passes over the tree give all answers in O(n).

## OPTIMAL SOLUTION · Two passes: subtree sizes, then re-root

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Why is the re-root formula correct?

Compare the distance from every node w to p and to its child v. If w is inside v's subtree the path to p goes through v, so dist(w, p) = dist(w, v) + 1. If w is outside it, dist(w, v) = dist(w, p) + 1. There are size[v] nodes of the first kind and n - size[v] of the second, so ans[v] = ans[p] - size[v] + (n - size[v]).

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Tree Distance Sum (Re-root DP)
#
# Onsite coding round 1: given an undirected tree on N
# nodes, return for every node the sum of distances to all
# other nodes. Equivalent to LC 834. Interviewer expects
# baseline O(n²) DFS followed by the O(n) re-root DP.
# Input: undirected tree with N nodes and N-1 edges.
# Output: array ans where ans[u] is the sum of shortest-
# path distances from u to every other node. Walk through
# baseline (BFS/DFS from every node, O(n²)) before
# optimizing. Final solution should be O(n) via two-pass
# re-root DP. Examples: Edges:
# [[0,1],[0,2],[2,3],[2,4],[2,5]] ans: [8, 12, 6, 10, 10,
# 10]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the re-root formula correct?
#
# INTERVIEW NOTES
# Running a BFS from every node costs O(n^2). Notice that
# moving the root from a parent p to its child v changes
```

```
# every distance by exactly one: the nodes inside v's
# subtree get closer by 1, all the others get farther by
# 1. So compute the answer for one root, then derive each
# child's answer from its parent's in O(1) using the
# subtree sizes.
#
# BRUTE FORCE (say it first): BFS from every node
# Run a BFS from each node and add up the distances.
# Correct and the right baseline to state in the first
# minutes; quadratic is too slow for 10^5 nodes. Time
# O(n^2). Space O(n).


from collections import deque


# BRUTE FORCE (say it first): BFS from every node | time
# O(n^2)
def tree_sums_brute(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    out = []
```

```
    for s in range(n):
        dist, q = {s: 0}, deque([s])
        while q:
            u = q.popleft()
            for v in adj[u]:
                if v not in dist:
                    dist[v] = dist[u] + 1
                    q.append(v)
        out.append(sum(dist.values()))
    return out


# HELPERS for the base question
def build(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    return adj


# BASELINE (say it first): BFS from every node
def distance_sums_bfs(n, edges):
    # the baseline to say first: a BFS from every node,
    # O(n^2)
    adj, out = build(n, edges), []
    for s in range(n):
        dist, queue = {s: 0}, deque([s])
        while queue:
            u = queue.popleft()
            for v in adj[u]:
                if v not in dist:
                    dist[v] = dist[u] + 1
                    queue.append(v)
        out.append(sum(dist.values()))
    return out


# BASE QUESTION (optimal): Two passes: subtree sizes, then
# re-root
# Idea: Root the tree at 0 and get a BFS order. In reverse
# order compute size[u] and the sum of distances inside
# u's subtree. For the root that sum is already the
# answer.
def tree_distance_sums(n, edges):
```

```
    if n == 0:
        return []
    adj = build(n, edges)
    # order: parents before children (no recursion)
    parent, order = [-1] * n, [0]
    for u in order:
        for v in adj[u]:
            if v != parent[u]:
                parent[v] = u
                order.append(v)
    # size[u]: nodes in u's subtree
    # inside[u]: distances from u to its own subtree
    size, inside = [1] * n, [0] * n
    for u in reversed(order):
        p = parent[u]
        if p >= 0:
            size[p] += size[u]
            inside[p] += inside[u] + size[u]
    # the root's answer is already complete
    ans = [0] * n
    ans[0] = inside[0]
    # moving the root from p to child u: the size[u] nodes
    # below get 1 closer, the other n - size[u] get 1
    # farther
    for u in order[1:]:
        ans[u] = ans[parent[u]] + n - 2 * size[u]
    return ans


if __name__ == "__main__":
    edges = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]
    want = [8, 12, 6, 10, 10, 10]
    assert tree_distance_sums(6, edges) == want
    assert distance_sums_bfs(6, edges) == want
    assert tree_distance_sums(1, []) == [0]
    assert tree_distance_sums(0, []) == []
    # a chain of 100000 nodes: no recursion trouble
    n = 100000
    chain = [(i, i - 1) for i in range(1, n)]
    first = tree_distance_sums(n, chain)[0]
    assert first == n * (n - 1) // 2
```

## NOTES FROM CANDIDATES

The interviewer explicitly liked the "baseline first, optimize later" structure — name the O(n²) and move on within the first ~10 minutes.

Two-pass technique to memorize:

Post-order DFS rooted at any node: compute subSize[u] and subSum[u] (sum of distances inside u's subtree). Recurrence: subSum[u] += subSum[v] + subSize[v].

Pre-order DFS doing re-root: when moving root from parent p to child v, ans[v] = ans[p] + (n - subSize[v]) - subSize[v].

Verbalize the geometric intuition for the re-root formula: moving root toward v shortens distance to the subSize[v] nodes inside v's subtree by 1 and lengthens the other n - subSize[v] by 1.

## PREPARATION

Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes.

Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.

Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).

**STUDY HERE: topic 5, Dynamic Programming | main topic: Dynamic programming**

**HARD | PracHub | Technical Screen | 23 of 23**

# Determine winner in stack-merging game

**POSTED 2026-01-06 | 287 DAYS AGO | OLDER**

**STUDY: TALK** **ALL TIME 31.2** **30 DAYS 0** **PRIORITY 121/228**

## ABOUT THIS QUESTION

**Topic:** Dynamic Programming

**Optimal answer from:** the interview site's own solution

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #877 Stone Game

**LeetCode match:** Related: LC 292 Nim Game, LC 464 Can I Win, LC 877 Stone Game (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-14)

**Original:** https://prachub.com/coding-questions/determine-winner-in-stack-merging-game

## QUESTION

Overview: This question evaluates understanding of combinatorial game theory, impartial game analysis, state-space encoding, and efficient state-space search techniques.

## APPROACH

Impartial game: encode the state (tuple of stack sizes), memoize win/lose with DFS over all legal merges, or use Sprague-Grundy values. Prune with symmetry (sorted state).

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

My copy of this question has only the overview, so before anything ask for the exact rules: what a move is (for example merge two stacks, possibly adjacent ones, with a size limit), who moves first, whether the player who cannot move loses (normal play) or wins, and the number and size of stacks.

**BRUTE FORCE TO SAY OUT LOUD**

Recursive search over every legal move with no memory. Exponential.

**OPTIMAL IDEA**

Treat it as an impartial game. Encode the state as the multiset of stack sizes (a sorted tuple, which removes symmetric duplicates). A state is winning if some move leads to a losing state, and a state with no moves is losing for the player to move under normal play. Memoize on the sorted tuple. If the game splits into independent parts, use Sprague-Grundy numbers (mex of reachable values) and XOR them instead of searching the product.

**DRY-RUN THESE CASES**

## INTERVIEW-DAY PLAN

No legal moves, one stack, two equal stacks, a state reachable by two move orders (checks memoization).

**LIKELY FOLLOW-UPS**

Larger states (find a pattern from small cases with a brute-force table, then prove it), misere play, or counting winning first moves.

**PITFALLS**

Starting to code before the rules are confirmed. Memoizing on an unsorted state and blowing up the state space.

## HOW TO THINK ABOUT IT

Two-player game with perfect play and no draws: define win(state) = some move leads to a state where the opponent loses. A state with no moves is a loss for whoever must move. If a move does not care about stack order, sort the state so equal positions share one cache entry. If the game splits into independent parts, use Grundy numbers and XOR them.

**Basis:** Written. The statement is not available in my copy, so this uses an example rule (confirm the real one): any two stacks may be merged if their combined size is at most a cap; a player who cannot move loses. The method (memoized win/lose search, Sprague-Grundy for independent games) is the part that carries over.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def wins(state):
    for nxt in all_merges(state):
        if not wins(nxt):
            return True  # opponent loses
    return False  # no move wins
```

| TIME | SPACE |
|---|---|
| **exponential in the number...** exponential in the number of stacks | **O(depth)** |

**WHY IT IS SLOW**

The same position is reached through many different move orders, so the work is exponential. Remembering the result per position (sorted state as key) solves each position once.

## OPTIMAL SOLUTION · Memoize on the sorted state

| TIME | SPACE |
|---|---|
| **O(number of distinct sorted states * k^2)** | **O(number of states)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

**FOLLOW-UP 1 The game is a sum of independent games (several separate boards or piles).**

Compute each part's Grundy number (the mex of the Grundy numbers of its moves) and XOR them. The first player wins exactly when the XOR is not zero. This turns an exponential search over the product of states into a sum of small searches.

**FOLLOW-UP 2 Misere play (the player who cannot move wins).**

Flip the base case in the memoized search: no moves means the player to move wins. Grundy numbers no longer compose by XOR in general, so search the whole state or use the known special-case theory.

**FOLLOW-UP 3 Count the number of winning first moves.**

Keep the same memoized win/lose function and count the moves whose resulting state is a loss for the opponent.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Determine winner in stack-merging game
#
# This question evaluates understanding of combinatorial
# game theory, impartial game analysis, state-space
# encoding, and efficient state-space search techniques.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The game is a sum of independent games (several
# separate boards or piles).
# 2. Misere play (the player who cannot move wins).
# 3. Count the number of winning first moves.
#
# INTERVIEW NOTES
# Two-player game with perfect play and no draws: define
# win(state) = some move leads to a state where the
# opponent loses. A state with no moves is a loss for
# whoever must move. If a move does not care about stack
# order, sort the state so equal positions share one cache
# entry. If the game splits into independent parts, use
# Grundy numbers and XOR them.
#
# BRUTE FORCE (say it first): Plain recursion over every
# move
# Try every merge, and the current player wins if some
# merge leaves the opponent in a losing position. Correct,
# but the same positions are explored again and again
# through different move orders. Time exponential in the
# number of stacks. Space O(depth).


from functools import lru_cache


# BRUTE FORCE (say it first): Plain recursion over every
# move | time exponential in the number of stacks
def merge_game_brute(stacks, cap):
    def wins(state):
        for i in range(len(state)):
            for j in range(i + 1, len(state)):
                if state[i] + state[j] <= cap:
                    nxt = (
                        state[:i]
                        + state[i + 1 : j]
                        + state[j + 1 :]
                        + [state[i] + state[j]]
                    )
                    if not wins(nxt):
                        return True
        return False

    return wins(list(stacks))


# HELPERS for the base question
def moves(state, cap):
    # merge two stacks if the new stack fits under the cap
    seen = set()
    for i in range(len(state)):
        for j in range(i + 1, len(state)):
            if state[i] + state[j] <= cap:
                rest = [x for k, x in enumerate(state)
                        if k not in (i, j)]
                merged = state[i] + state[j]
                new = tuple(sorted(rest + [merged]))
                # equal results are the same move
                if new not in seen:
                    seen.add(new)
                    yield new


# BASE QUESTION (optimal): Memoize on the sorted state
# Idea: Merging does not depend on order, so a sorted
# tuple identifies the position. Each state is solved once
# and then looked up. This is the standard fix for 'many
# move orders reach the same position'.
def first_player_wins(stacks, cap, misere=False):
    # the player who cannot move loses (misere: wins)
    @lru_cache(None)
    def wins(state):
        options = list(moves(state, cap))
        if not options:
            return misere
        # a winning move leaves the opponent in a losing
        # state
        return any(not wins(nxt) for nxt in options)

    return wins(tuple(sorted(stacks)))


# FOLLOW-UP 1: The game is a sum of independent games
# (several separate boards or piles).
# Answer: Compute each part's Grundy number (the mex of
# the Grundy numbers of its moves) and XOR them. The first
# player wins exactly when the XOR is not zero.
# FOLLOW-UP 2: Misere play (the player who cannot move
# wins).
# Answer: Flip the base case in the memoized search: no
# moves means the player to move wins. Grundy numbers no
# longer compose by XOR in general, so search the whole
# state or use the known special-case theory.
def grundy(stacks, cap):
    # Sprague-Grundy number of one game (mex of its moves)
    @lru_cache(None)
    def g(state):
        reach = {g(nxt) for nxt in moves(state, cap)}
        number = 0
        while number in reach:
            number += 1
        return number
```

```
    return g(tuple(sorted(stacks)))


# FOLLOW-UP 1: The game is a sum of independent games
# (several separate boards or piles).
# Answer: Compute each part's Grundy number (the mex of
# the Grundy numbers of its moves) and XOR them. The first
# player wins exactly when the XOR is not zero.
# FOLLOW-UP 2: Misere play (the player who cannot move
# wins).
# Answer: Flip the base case in the memoized search: no
# moves means the player to move wins. Grundy numbers no
# longer compose by XOR in general, so search the whole
# state or use the known special-case theory.
def sum_of_games_wins(boards, cap):
    # independent boards: XOR their Grundy numbers
    total = 0
    for board in boards:
        total ^= grundy(board, cap)
    return total != 0


# FOLLOW-UP 3: Count the number of winning first moves.
# Answer: Keep the same memoized win/lose function and
# count the moves whose resulting state is a loss for the
```

```
# opponent.
def winning_first_moves(stacks, cap):
    # how many different first moves win?
    state = tuple(sorted(stacks))
    return sum(1 for nxt in moves(state, cap)
               if not first_player_wins(nxt, cap))


if __name__ == "__main__":
    # one stack: no move, so the first player loses
    assert not first_player_wins([3], 5)
    # two stacks that can merge: one move ends it, so we
    # win
    assert first_player_wins([1, 1], 5)
    # three stacks: after one merge the other two may
    # merge too
    assert first_player_wins([1, 1, 1], 2) is True
    # in misere play the result flips for a single stack
    assert first_player_wins([3], 5, misere=True)
    assert winning_first_moves([1, 1, 1], 2) >= 1
    # normal play: a position wins exactly when Grundy !=
    # 0
    for state in ([1, 2, 3], [2, 2, 2, 2], [1, 1, 4]):
        win = first_player_wins(state, 6)
        assert win == (grundy(state, 6) != 0)
    # two separate boards: XOR of their numbers
    both = sum_of_games_wins([[1, 1], [1, 1]], 5)
    assert both is False
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(colors: list[str]) -> bool:
SAMPLE INPUT
['Y']
```

### Test cases

| Input | Expected |
|---|---|
| `['Y']` | `False` |
| `['Y', 'W']` | `True` |

**ALSO REPORTED · 1 more**

**Babylon Tile-Stack Merge Game | 1Point3Acres | onsite-coding | asked 2025-12-29 | Single report**

https://www.1point3acres.com/interview/problems/company/google/babylon-stack-merge-game

This is closer to an implementation/simulation interview than a clever-algorithm problem — the interviewer wants a clean game state class, legal-move enumeration, and a play(move) API.
Represent state as a List[Stack] where Stack = (height, topColor, fullSequence?). Whether you keep the full color sequence depends on whether the interviewer asks about "undo" or "re-color from top".
Add a minimax / negamax for "who wins from this state with optimal play" as the natural follow-up — branching factor is small (at most C(12, 2) = 66 initially, drops fast).
Reported as "too hard to finish coding in 45 minutes"; the interviewer gives hints throughout. Pacing matters.

Preparation: Drill clean game-state implementations: e.g. Connect 4, Othello move generators, in 30 minutes.
Practice writing a generic minimax framework so you can plug in this game's legalMoves(state) and terminal(state) quickly.
Watch the merge invariant carefully: "same height OR same top color" — fail mode is forgetting the alternative branch.